



2026

Decoding **Compilers**

First Edition

Published by:
Code Blue Publications

DECODING COMPILERS



DECODING COMPILERS

THE JOURNEY FROM CODE TO EXECUTION



FIRST EDITION

APRIL 6, 2026

CODE BLUE PUBLICATIONS

Savar, Dhaka

Preface

This book, *Decoding Compilers*, is written for students, beginners, and anyone who wants to understand the basic concepts of compiler design. The main goal of this book is to explain compiler topics in a simple and clear way so that readers can easily understand how a compiler works.

The book covers important topics such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and code generation. Each topic is explained step by step, with examples to help readers understand both the theory and practical aspects of compiler design.

This book is written as part of an academic project by students of the Department of Computer Science and Engineering, Daffodil International University. While writing this book, we focused on making the explanations simple, organized, and easy to follow.

Prerequisites: To get the most from this book, readers should have some basic knowledge of any programming languages and the theory of computation. You do not need to be an expert, but understanding concepts like variables, loops, functions, basic logic, and objects will help you follow the examples and explanations more easily.

We hope this book will help students learn compiler design in an easy way and develop a better understanding of how programming languages are processed and executed by a computer.

-----Authors

Acknowledgement

We would like to give our deepest and sincerest thanks to our parents. Their love, care, and support have always been the strongest foundation of our lives. From the beginning, they have guided us, encouraged us, and believed in us even when we doubted ourselves. Their patience and sacrifices have made it possible for us to follow our dreams and continue our education without fear or worry.

Every success we have achieved is because of their constant guidance and support. They have given us not only the opportunity to study but also the confidence to explore new ideas and learn new things. Their example has inspired us to work hard, stay focused, and never give up, no matter how difficult the path may seem. They have shown us the value of kindness, patience, and understanding, which we hope to carry forward in our own lives.

Even in times when life was not easy, their support never wavered. They have sacrificed so much of their time, energy, and comfort to make sure we have everything we need to grow and succeed. Their dedication has been the light that guided us through every challenge, and their encouragement has been the strength that kept us moving forward.

We dedicate this work and all our efforts to our parents. This is a small way to show our respect, gratitude, and love for everything they have done for us. Words cannot fully express how thankful we are for their support, their guidance, and their constant belief in us. We hope that through our hard work and achievements, we can make them proud and honor all their sacrifices.

Thank you, Mom and Dad, for being our pillars, our inspiration, and our guiding light. Your love and support are the reason we are here today, and we will always be grateful for everything you have done for us. May He bless our parents with health, happiness, and the highest place in Jannatul Ferdous. Ameen.

About The Authors

This book is written by a team of five members who are passionate about compiler design, programming languages, and software development:

Md. Sazzad Sabbir Sun

ID: 241-15-882

Department of Computer Science and Engineering
Daffodil International University

Md. Didarul Islam

ID: 241-15-214

Department of Computer Science and Engineering
Daffodil International University

Md. Sanjidor Rahman Shimul

ID: 241-15-956

Department of Computer Science and Engineering
Daffodil International University

Md. Tushar Abdullah

ID: 241-15-130

Department of Computer Science and Engineering
Daffodil International University

Md. Sajib Ahmed

ID: 241-15-832

Department of Computer Science and Engineering
Daffodil International University

Table of Contents

Chapter 1: INTRODUCTION TO COMPILER	1
Introduction	1
Compiler	2
Purposes of Compiler	2
Interpreter	3
Key Characteristics of Interpreter	3
Advantages of an Interpreter	3
Disadvantages of an Interpreter	3
Compiler vs Interpreter	3
Language Processing System	4
Source Program and Target Program	4
Components of a Language Processing System	4
Phases of a Compiler	5
Overview of Compiler Phases	6
Bootstrapping	6
Compiler Tools	7
Chapter 1 Exercises	8
Part A: Define / Explain	8
Part B: True/False:	8
Part C: Short Answer Questions	8
Part D: Comparison / Analytical Questions	8
Part E: Scenario / Story-Based Questions	9
Part F: Challenge Questions	9
 Chapter 2: STRUCTURE OF A COMPILER	 11
Introduction	11
Compiler Architecture	12
Frontend vs Backend	13

The Frontend (Customer Area).....	13
The Backend (Kitchen Area)	14
Data Flow in a Compiler	14
Step-by-Step Flow	14
Phases Of Compiler (Expression)	16
Tokens and Lexemes	16
Lexical Analyzer.....	17
Syntax Analyzer.....	18
Semantic Analyzer.....	18
Intermediate Code Generation.....	18
Code Optimization.....	19
Target Code Generation.....	19
Chapter 2 Exercises	23
Part A: Define / Explain	23
Part B: True / False	23
Part C: Short Answer Questions	23
Part D: Comparison / Analytical Questions.....	24
Part E: Scenario / Story-Based Questions	24
Part F: Challenge Questions	24
Chapter 3: INTRODUCTION TO LEXICAL ANALYSIS	27
Introduction	27
Role of Lexical Analyzer.....	27
Tokens and Patterns	28
Tokens.....	28
Patterns	28
Lexemes, Tokens, Attributes	29
Lexeme	29
Token	29

Attribute.....	29
Regular Expressions (RE)	30
RE To Strings.....	30
RE To Language	31
RE To Token Pattern.....	32
Finite Automata (NFA and DFA).....	32
NFA (Non-deterministic Finite Automata)	32
DFA (Deterministic Finite Automata)	33
Difference between NFA and DFA.....	34
RE to NFA to DFA Conversion	35
Lex Tools (Flex)	35
Input Buffering	35
Error Handling in Lexical Analysis.....	36
Chapter 3 Exercises	37
Part A: Define / Explain	37
Part B: True / False	37
Part C: Short Answer Questions	37
Part D: Comparison / Analytical Questions.....	38
Part E: Scenario / Story-Based Questions	38
Part F Challenge Questions:	38
Chapter 4: INTRODUCTION TO SYNTAX ANALYSIS	39
Introduction	39
Syntax Analysis	40
Roler of Parser.....	40
Top-Down Parsing.....	41
Bottom-Up Parsing.....	41
Top-Down vs Bottom-Up Parsing	41
Context-Free Grammar (CFG)	42

Formal definition of CFG	42
LMD & RMD	43
Parse Tree	44
Ambiguity in Grammar	46
Steps to determine grammar ambiguity.....	46
Problems of Ambiguity.....	48
Operator Precedence and Associativity	49
Grammar Transformations.....	49
Left Recursion Removal.....	50
Left factoring	52
FIRST	53
FOLLOW	54
LL(1) Parsing.....	55
Advantages of LL(1) parser.....	56
Conditions for LL(1) Grammar	56
LL(1) Parsing Table.....	56
Recursive Descent Parser	59
Key features of RDP.....	59
Structure of a Recursive Descent Parser	59
Bottom-Up Parsing.....	62
Shift-Reduce Parsing	63
Steps in Shift-Reduce Parsing:	63
Operator Precedence Parsing.....	65
Steps in Operator Precedence Parsing	65
LR(0) Parsing	66
CFG to LR(0) Parsing Table.....	66
Parser Generators.....	69
Yacc (Yet Another Compiler-Compiler)	70
Bison.....	70

Working principle of Yacc/Bison.....	70
Chapter 4 Exercises	71
Part A: Short Questions	71
Part B: Explanation Questions.....	72
Part C: Challenge Questions.....	72
Part D: Practice Questions.....	73
Chapter 5: INTRODUCTION TO SEMANTIC ANALYSIS.....	77
Introduction	77
Semantic Checking.....	77
Symbol Table.....	78
Type Checking.....	78
Scope and Binding.....	78
Semantic Errors	79
Chapter 5 Exercises	80
Part A: Define / Explain	80
Part B: True / False	80
Part C: Short Answer Questions	80
Part D: Comparison / Analytical Questions.....	81
Chapter 6: INTRODUCTION TO ICG	83
Introduction	83
Intermediate Representation (IR)	83
Why IR is Needed	84
Advantages of IR.....	85
High-level vs Low-level IR.....	85
Types of IR	86
Linear IR (TAC)	86

Simple TAC	87
Quadruples.....	87
Triples	88
Indirect Triples.....	88
Boolean Expressions in TAC.....	91
Short-Circuit Evaluation (AND, OR).....	91
Backpatching	92
Jumping Code vs Boolean Value Code.....	92
Tree IR (Syntax Tree)	94
Key Properties of Tree IR.....	95
Importance of Tree IR	96
Tree IR vs Linear IR.....	96
Prase Tree vs Syntax Tree.....	96
Graph IR (DAG).....	97
Properties of DAG	97
Importance of DAG	98
Common Subexpression Elimination using DAG.....	98
Syntax Tree vs DAG.....	99
Construction of DAG	99
Chapter 6 Exercises	102
Part A: Theory Questions	102
Part B: Practice Questions	102
Chapter 7: RUNTIME ENVIRONMENT	105
Introduction	105
Storage Organization	105
Stack, Heap, Static Memory	106
Activation Records	109
Parameter Passing.....	110

Recursion Handling	111
Chapter 7 Exercises	113
Part A: Define / Explain	113
Part B: True / False	113
Part C: Short Answer Questions	113
4. Comparison / Analytical Questions	114
5. Scenario / Story-Based Questions	114
6. Challenge Questions	114
Chapter 8: INTRODUCTION TO CODE OPTIMIZATION.....	115
Introduction	115
Goals of Optimization (Performance, Space, Energy)	115
Levels of Optimization	116
Scope of Optimization (Local vs Global).....	116
Basic Blocks and Flow Graphs.....	117
Definition of Basic Blocks	117
Control Flow Graph (CFG)	117
Leader Statements and Block Boundaries	118
Use in Optimization.....	119
Local Optimization Techniques	119
Constant Folding	119
Algebraic Simplification	120
Strength Reduction	120
Copy Propagation	121
Global Optimization Techniques	121
Common Subexpression Elimination	122
Dead Code Elimination	122
Code Motion (Loop-Invariant Code Motion).....	123
Instruction Scheduling.....	123

Local Optimization Techniques.....	124
Importance of Loops in Performance	124
Loop Unrolling	124
Loop Fusion and Fission	125
Induction Variable Analysis.....	125
Loop Interchange.....	126
Data Flow Analysis for Optimization.....	127
Concept of Data Flow Analysis	127
Reaching Definitions	128
Live Variable Analysis.....	128
Available Expressions.....	128
Advanced Optimization Techniques.....	129
Peephole Optimization	130
Inline Expansion	130
Tail Recursion Elimination.....	130
Trade-offs and Limitations	131
Space vs Time Trade-off.....	131
Complexity of Analysis	131
Machine-Specific Considerations.....	132
Chapter 8 Exercises	133
Part A: Define / Explain	133
Part B: True / False	133
Part C: Short Answer Questions	134
Part D: Comparison / Analytical Questions.....	134
 Chapter 9: INTRODUCTION TO TCG	135
Introduction	135
Target Code Generation.....	136
Role of Target Code Generator.....	137

Input to Code Generator (IR, Symbol Table)	138
Output of Code Generator	139
Issues in Code Generation	140
Phases of Target Code Generation.....	141
Instruction Selection	142
Translating TAC to Target Code	142
Cost of Instructions.....	143
Register Allocation	145
Need for Register Allocation	145
Register Assignment.....	145
Register Spilling	146
Local Register Allocation	147
Addressing Modes	147
Immediate Addressing	148
Direct Addressing	148
Indirect Addressing.....	148
Register Addressing.....	148
Indexed / Base + Offset Addressing	149
Choosing Addressing Modes	149
Instruction Scheduling.....	149
The Problem: Pipeline Hazards	150
Instruction Scheduling Solution	151
Scheduling in Loops	151
Peephole Optimization	151
Code Generation for Control Statements	152
Chapter 9 Exercises	157
Part A: Conceptual Questions.....	157
Part B: Practice Problems	157
Part C: Control Statement Exercises	158

Chapter 10: ERROR HANDLING	159
Introduction	159
Error Types	159
Lexical, Syntax, Semantic Errors	160
Types of Errors Table.....	162
Panic Mode Recovery.....	162
Phrase-Level Recovery.....	163
Chapter 10 Exercises	164
Part A: Define / Explain	164
Part B: True / False	164
Part C: Short Answer Questions	164
Part D: Comparison / Analytical Questions.....	165
 Chapter 11: SYMBOL TABLE MANAGEMENT	 167
Introduction	167
Symbol Table Design.....	167
Hash Table and Tree	168
Scope Handling	169
Insert, Lookup, Delete	170
Chapter 11 Exercises	171
Part A: Define / Explain	171
Part B: True / False	171
Part C: Short Answer Questions	171
Part D: Comparison / Analytical Questions.....	172
Part E: Scenario / Story-Based Questions	172
Part F: Challenge Questions	172

Chapter 12: CODE TO COMPILE	173
Introduction	173
Basic Code.....	174
Count String Length	174
Remove white space	174
Count Vowels.....	175
Multiple Line Input.....	176
Handling Comments.....	177
Regex	179
Regex Pattern Code	182
Max Frequency Word	182
Creating a New Language	184
Demo Code for SimpleScript	185
Main Function	186
Tokenization (Lexer)	187
Parser	189
Semantic analysis	192
Evaluation.....	194
 Final Words.....	 197

Chapter 1

INTRODUCTION TO COMPILER

Introduction

In the modern world, computers are used to solve many problems. But computers do not understand human languages or high-level programming languages directly. Computers only understand machine language, which is written in binary (0s and 1s). Therefore, we need a translator that can convert high-level programming languages into machine language so that the computer can understand and execute the program. This translator is called a compiler.

A compiler is an important part of computer science and software development because it helps programmers write programs in easy and readable languages like C, C++, Java, and Python, and then converts those programs into machine code. Without compilers or interpreters, programming would be very difficult because programmers would have to write programs in machine language.

In this chapter, we will learn the basic concepts of compiler design. We will discuss what a compiler is, the purpose of a compiler, and how an interpreter works. We will also learn the characteristics, advantages, and disadvantages of interpreters. Then we will compare compilers and interpreters to understand their differences.

After that, we will study the language processing system, source program, and target program. We will also learn the components of a language processing system and the different phases of a compiler. An overview of all compiler phases will help us understand how a program is translated step by step. Finally, we will discuss bootstrapping and compiler tools, which are important concepts in compiler design.

This chapter will build the basic foundation of compiler design, which will help you understand the more advanced topics in the later chapters.

Compiler

Suppose you are from Bangladesh and your friend is from Saudi Arabia. You speak Bangla, and your friend speaks Arabic. If you want to communicate with your friend, you need a translator to convert your Bangla into Arabic so that your friend can understand.

This is exactly what a compiler does. Computers cannot understand human languages. They only understand true and false values (represented as 0s and 1s). We use programming languages like C++, Java, Python, and others to give instructions to a computer. However, these instructions are written in a human-readable form, so the computer still cannot execute them directly.

A compiler acts as the middleman. It translates the high-level code that humans can understand into machine code that a computer can execute.

In short, a compiler is a special program that converts a program written in a high-level language into a form (machine language) that the computer can understand and run.

Purposes of Compiler

Consider the earlier example of communicating with a friend who speaks a different language. A translator does more than simply convert words from one language to another. Before translating, the translator ensures that the sentence is correct, meaningful, and clearly expressed. If there are mistakes, they are identified and corrected to avoid confusion.

In a similar way, a compiler performs several important tasks beyond simple translation. Its purpose is not only to convert code but also to ensure that the program is correct and efficient. Such as:

Translation: The primary function of a compiler is to translate a program written in a high-level programming language into machine language. High-level languages such as C++, Java, and Python are designed for humans. The compiler converts this human-readable code into a form that the computer can execute.

Error Detection: A compiler checks the program for errors before translation is completed. These errors include violations of language rules (like missing semicolons), logical or meaning-related mistakes etc. By detecting errors early, the compiler helps programmers correct their code before execution.

Code Optimization: After ensuring the program is correct, the compiler improves the code through optimization. This involves making the program faster in execution, more efficient in memory usage and cleaner structure. Optimization ensures better performance without changing the actual output of the program.

Interpreter

An interpreter is another type of language translator used in programming. Like a compiler, it converts high-level programming code into a form that a computer can understand. However, the way it works is different.

Instead of translating the entire program at once, an interpreter reads and executes the program line by line. For example: JavaScript, Python, Ruby.

Key Characteristics of Interpreter

1. Line-by-line execution
2. No separate executable file is generated
3. Execution stops immediately if an error occurs
4. Slower than a compiler (due to continuous translation)

Advantages of an Interpreter

1. Easier to debug (errors are shown immediately)
2. No need for a separate compilation step
3. Platform-independent in many cases

Disadvantages of an Interpreter

1. Slower execution compared to compiled programs
2. Requires the interpreter every time the program runs
3. Less optimized compared to compiled code

Compiler vs Interpreter

Compiler	Interpreter
Translates the entire program at once	Translates and executes line by line
Faster execution	Slower execution

Decoding Compilers

Generates a separate executable file as output	Does not generate an executable file
Reports all errors after full compilation	Stops at the first error and reports it immediately
Compiled code can run without compiler	Requires interpreter every time to run
Performs code optimization	Limited or no optimization
Example: C, C++	Example: Python, JavaScript

Language Processing System

To understand a language processing system, think about how a simple idea becomes a final usable product in real life. Imagine you want to publish a book. You start by writing your content, but that alone is not enough. Your writing goes through several stages before it becomes a printed book in someone's hands.

In the same way, when you write a program, it must go through multiple steps before the computer can execute it. This complete journey is called a language processing system.

A Language Processing System is a collection of programs used to translate a program written in a high-level language into a form that a computer can understand and execute.

Source Program and Target Program

Source Program: The program written by the programmer in a high-level language (like C++, Java, Python). It is human-readable but not understandable by the computers.

Target Program: The final output after processing that computer can execute (usually binary form).

Components of a Language Processing System

Preprocessor: The preprocessor works on the source code before actual compilation begins. It performs tasks such as expanding macros (`#define`), including header files (`#include`), removing comments. It prepares the code for the next stage.

Decoding Compilers

Compiler: The compiler takes the preprocessed code and translates it into assembly code or intermediate code. It also checks for errors and ensures the code follows proper rules.

Assembler: The assembler converts the assembly code into machine code (object code). This is a lower-level translation which is closer to the computer understandable form.

Linker: The linker combines multiple object files and libraries into a single executable program. For example, we use built in library function such as Math functions, String functions. The linker connects them properly.

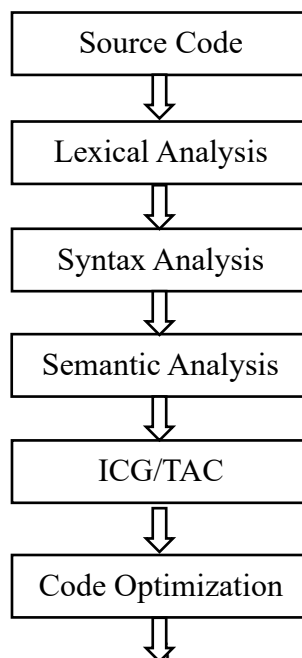
Loader: The loader loads the executable program into memory so that the CPU can execute it. This is the final step before the program runs.

Phases of a Compiler

To understand the phases of a compiler, think about how a raw idea is transformed step by step into a final product. Just like in a production process, where each step improves the product, a compiler also processes a program through multiple well-defined stages.

Each stage is called a phase, and every phase performs a specific task. The output of one phase becomes the input for the next phase.

Phases are:



Overview of Compiler Phases

Lexical Analysis: This is the first phase of the compiler. It reads the source code character by character and converts it into meaningful units called tokens.

Syntax Analysis (Parser): This phase checks whether the tokens follow the correct grammar rules of the language. It builds a syntax tree (also known as parse tree) and detects syntax errors.

Semantic Analysis: This phase checks the meaning of the program by type checking, variable declaration verification and scope resolution (where a variable or function can be used).

Intermediate Code Generation (ICG): The compiler converts the program into an intermediate representation which is easier to optimize and independent of machine.

Code Optimization: This phase improves the intermediate code to make it more efficient. It removes unnecessary operations, reduces execution time, and improves memory usage.

Code Generation: This is the final phase. It converts optimized code into machine code and produces assembly or binary instructions.

Note that we will study the phases of a compiler in more detail in the upcoming chapters.

Bootstrapping

Bootstrapping in compiler design can be understood through a simple analogy. Imagine a small bakery that wants to make a brand-new type of cake but does not yet have the tools or recipes. To start, the bakery uses a basic version of the recipe to make a simple cake. Once this small cake is successful, the bakery gradually improves its recipe and tools to make the full, final version of the cake.

Similarly, in programming, when a new programming language is created, a basic compiler for that language is first written using an existing language, like assembly

Decoding Compilers

or Python or Java. This basic compiler can compile simple programs written in the new language.

Then, programmers write a more advanced compiler using the new language itself and use the basic compiler to compile it.

Over time, the compiler becomes self-sufficient, capable of compiling any program written in that language, including its own source code.

This process of a compiler gradually building itself up to become fully functional is called bootstrapping. Just like the bakery gradually perfects its cake recipe, bootstrapping allows a programming language to become self-hosted and independent.

Compiler Tools

Creating a compiler from scratch is a complex task. But just like a factory uses special machines and tools to produce a finished product efficiently, compiler designers use compiler tools to simplify and automate parts of the compilation process.

For example, a lexical analyzer generator like **Lex** can automatically create a program that breaks the source code into tokens. It saves the designer from writing repetitive code manually.

Similarly, a parser generator like **Yacc** can take grammar rules of a language and automatically produce a parser. It checks the structure of the code for syntax errors.

Other tools help with syntax-directed translation, code optimization, and intermediate code generation. These tools acting like machines in a factory that refine raw materials into a final product.

By using these tools, compiler developers can focus on the logic and optimization of the compiler instead of spending time on small, repetitive tasks. Essentially, compiler tools act as helper systems that make the process of building a fully functional compiler faster, more reliable, and easier to manage.

Chapter 1 Exercises

Part A: Define / Explain

1. Define a compiler in your own words.
2. Define an interpreter. How is it different from a compiler?
3. Explain what a source program and a target program are.
4. What is bootstrapping in compiler design? Give an example.
5. Explain the role of a preprocessor in a language processing system.

Part B: True/False:

1. A compiler translates code line by line while executing it.
2. Bootstrapping allows a compiler to compile itself.
3. A preprocessor works after the compilation phase is complete.
4. An interpreter does not generate an executable file.
5. Optimization is performed to make the compiled code slower but more readable.

Part C: Short Answer Questions

1. List the phases of a compiler in order and briefly explain the purpose of each phase.
2. What are the advantages and disadvantages of using a compiler over an interpreter?
3. Name some common interpreted languages and compiled languages.
4. What is the role of a linker and a loader in program execution?
5. Give two examples of compiler tools and explain their purpose.

Part D: Comparison / Analytical Questions

1. Compare a compiler and an interpreter using at least four points.

Decoding Compilers

2. Compare a source program with a target program using an analogy.
3. Explain why code optimization is important in the compilation process.
4. How does the language processing system resemble a factory production line?
5. Explain with an example how bootstrapping allows a compiler to improve itself.

Part E: Scenario / Story-Based Questions

1. Imagine you wrote a program in Python, but your computer only understands binary. Explain step by step how the program is executed using an interpreter.
2. You are designing a new programming language. Explain how you could use bootstrapping to create its compiler.
3. Consider a program that uses multiple libraries. Explain the role of the linker and loader in ensuring the program runs correctly.
4. Think of a real-life analogy (other than book or bakery) that explains the phases of a compiler.

Part F: Challenge Questions

1. Explain the complete flow of a language processing system using your own analogy.
2. Why is a compiler considered more efficient for production programs compared to an interpreter?
3. Explain how lexical analysis and syntax analysis work together to detect errors in a program.
4. Give an example of a situation where an interpreter is more suitable than a compiler.
5. Imagine a new programming language “XLang.” Write a short plan for creating its compiler using the concept of bootstrapping.

Chapter 2

STRUCTURE OF A COMPILER

Introduction

In Chapter 1, we learned what a compiler is and the basic phases of a compiler. Now in this chapter, we will learn the structure of a compiler. We will learn about how a compiler is organized and how different parts of a compiler work together to translate a program.

A compiler is a large and complex program, so it is divided into smaller parts. Each part of the compiler has a specific job. These parts are called phases of the compiler. Every phase takes input from the previous phase, processes it, and sends output to the next phase.

The compiler is mainly divided into two major parts:

1. Frontend
2. Backend

The Frontend works with the source program. It checks whether the program is written correctly or not. It includes:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis

The Backend works with the intermediate code and converts it into the final machine code. It includes:

1. Intermediate Code Generation
2. Code Optimization
3. Code Generation

There are also two important components that work with all phases of the compiler:

Symbol Table: Stores information about variables, functions, identifiers, etc.

Error Handling: Detects and reports errors in each phase of the compiler.

In this chapter, we will learn about all these parts in detail and understand how a compiler converts a source program into machine code step by step.

Compiler Architecture

Compiler architecture can be understood through a simple real-life analogy. Imagine a large postal office that processes thousands of letters every day. When you send a letter, it does not directly go to the destination in one step. Instead, the letter passes through several departments, and each department performs a specific task before sending it to the next department.

First, the postal office workers collect letters and check the addresses to make sure they are written properly and readable. If the address is not clear, they send the letter back. This step is similar to Lexical Analysis, where the compiler reads the source code and checks the words (tokens).

Next, the letters go to another department and in there, workers check if the address format is correct. For example, house number, road number, city name, postal code, etc. If the format is wrong, the letter cannot be delivered. This step is like Syntax Analysis. In the Syntax Analysis the compiler checks whether the program follows the grammar rules of the programming language.

After that, the letters go to another department where workers check if the address actually exists. For example, if the city exists but the road number does not exist, then the letter still cannot be delivered. This step is similar to Semantic Analysis, where the compiler checks whether the program makes logical sense, like whether a variable is declared before use, type checking, etc.

Once all the checking is done, the postal office converts the letter into a delivery format, such as sorting it by region and delivery route. This is similar to Intermediate Code Generation, where the compiler converts the program into an intermediate form.

Then, the postal office tries to find the fastest and cheapest delivery route. This step is similar to Code Optimization, where the compiler makes the program faster and more efficient.

Finally, the letter is sent to the postman for delivery, and the postman delivers it to the destination. This step is like Code Generation, where the compiler produces the final machine code that the computer can run.

Decoding Compilers

Throughout the whole process, the postal office uses a database that stores addresses, postal codes, and city names. In compiler design, this database is called the Symbol Table, which stores information about variables, functions, and identifiers.

Also, if any problem happens at any step (wrong address, wrong format, missing information), the postal office reports the problem. In compiler design, this is called Error Handling, which detects and reports errors in every phase of the compiler.

So, just like a postal office processes a letter step by step through different departments, a compiler processes a program step by step through different phases. This overall organization and flow of different phases is called the Compiler Architecture.

Frontend vs Backend

Imagine a restaurant. When you go to a restaurant, your experience is divided into two main parts:

1. What happens in front of you
2. What happens behind the scenes in the kitchen

The Frontend (Customer Area)

When you enter the restaurant, a waiter comes to you and takes your order. The waiter listens carefully to what you say, checks if your order makes sense, and writes it down properly. For example: if you say something unclear, the waiter asks again. If you order something not on the menu, the waiter tells you it's not possible.

This is exactly like the Frontend of a compiler. It reads your code, checks if it is written correctly, understands what you mean. Frontend includes:

1. Lexical Analysis: understanding words (tokens)
2. Syntax Analysis: checking grammar
3. Semantic Analysis: checking meaning

So, the frontend is like the waiter who understands your order.

The Backend (Kitchen Area)

Now, after the waiter takes your order, it goes to the kitchen. In the kitchen what a chef does? He prepares the food. He may improve the recipe so that it becomes more tastier or faster to cook. Finally, he serves the finished dish.

This is like the Backend of a compiler. The backend takes the processed input from frontend, converts it into machine code and optimizes it for better performance.

The backend includes:

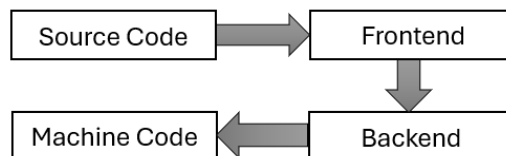
1. Intermediate Code Generation
2. Code Optimization
3. Machine code generation

So, the backend is like the kitchen that prepares the final food.

Data Flow in a Compiler

To understand Data Flow in a Compiler, imagine a water purification plant. Dirty water enters the plant from one side, and clean drinking water comes out from the other side. But the water does not become clean in one step. It passes through several filters, and each filter removes some impurities and improves the quality of the water.

Similarly, in a compiler, the source code enters from one side, and machine code comes out from the other side. Between them, the code passes through several phases, and each phase changes the form of the code step by step. This movement of code from one phase to another is called Data Flow in a Compiler.



Step-by-Step Flow

Source Code: This is the program written by the programmer in a high-level language like C, C++, or Java. Example: `int a = 5 + 3;`

This is the input to the compiler.

Decoding Compilers

Tokens (After Lexical Analysis): The lexical analyzer breaks the program into small pieces called tokens. For example: for the previous source code we can separate the tokens as: int, a, =, 5, +, 3, ;. These small parts are easier for the compiler to understand.

Parse Tree: The syntax analyzer checks grammar and creates a parse tree (tree structure of the program). This shows the structure of the statement like: assignment, expression, operator, values. So now the compiler understands the structure of the program.

Annotated Tree (After Semantic Analysis): Now the semantic analyzer checks the meaning of the program. It checks for is variable declared? Are types correct? Is the operation is valid? It adds extra information to the parse tree. This is called an annotated tree. Now the compiler understands the meaning of the program.

Intermediate Code: Now the compiler converts the program into Intermediate Code (a simple, machine-independent code). Example:

```
t1 = 5 + 3
```

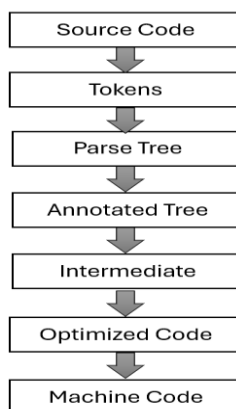
```
a = t1
```

This code is easier to optimize and convert into machine code.

Optimized Code: The compiler improves the code to make it faster or smaller. For example: a = 8. This is faster than doing addition every time.

Target Machine Code: Finally, the compiler converts the optimized code into machine code (0 and 1) which the computer can execute. This is the final output.

Full Data Flow



Phases Of Compiler (Expression)

In this lesson we will learn about how we can describe six phases of compiler for a given expression. Let us consider an expression:

Tokens and Lexemes

Before learning about the phases of a compiler using an example expression, we need to understand some important terms used in lexical analysis. These terms are Token, Lexeme, and Synchronization Token. These are basic concepts, and without understanding them, the phases of a compiler will be difficult to understand.

Think of programming code like a sentence in English. Before checking grammar or meaning, a teacher first reads the words carefully. Similarly, the compiler first identifies what each piece of code is, without worrying about correctness yet.

Lexeme: Lexeme is like a word in a sentence. Imagine a teacher reading a sentence: “Rakib eats an apple”. So, the teacher sees the words: “Rakib”, “eats”, “an”, “apple”. Each of these is a lexeme. It’s the actual text the student wrote.

In programming, for example: `int a = 10;`. The lexemes are: { `int`, `a`, `=`, `10`, `;` }. They are the actual characters in the source code.

Token: Token is like the category of the word. The teacher doesn’t just read the word; he/she also knows the type of the word. For example: “Rakib” is proper noun, “is” is a verb, “an” is an article, and “apple” is a noun.

Similarly in programming:

Lexeme	Tokens
<code>int</code>	KEYWORD
<code>a</code>	IDENTIFIER
<code>=</code>	OPERATOR
<code>10</code>	CONSTANT
<code>;</code>	SYMBOL

So, the token tells the compiler what kind of word it is, while the lexeme is the actual word. Now let’s start describing the expression using the six phases of compiler:

Lexical Analyzer

The Lexical Analyzer first breaks the program into small pieces and identifies what each piece is. It does not check grammar or meaning yet. It only identifies the pieces. This process is called Lexical Analysis.

The lexical analyzer reads the input character by character. Then it's forms the lexemes. Also note that, it ignores spaces and focuses only on meaningful characters.

Now for our given expression ($a = b * c + b / 12$) let's create the lexeme and token tables:

Identifier table:

Lexeme	Token
a	<id, 1>
b	<id, 2>
c	<id, 3>

Note here, we ignored the duplicate lexemes here.

Constant/Digit table:

Lexeme	Token
12	<d, 1>

Operator table:

Lexeme	Token
=	<op, 1>
*	<op, 2>
+	<op, 3>
/	<op, 4>

Tokenization: After creating the token table, the next step is tokenization of the expression. In this step, each lexeme in the expression is replaced by its corresponding token. So, the expression becomes:

<id, 1> <op, 1> <id, 2> <op, 2> <id, 3> <op, 3> <id, 2> <op, 4> <d, 1>

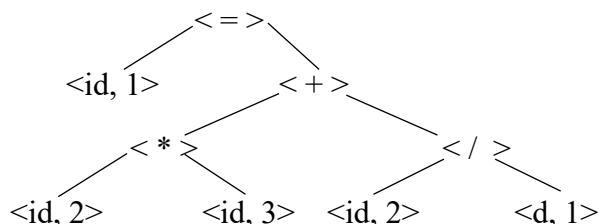
Here, we can also write in this way and this version is much easier for solving expression. Tokenized expression:

```
<id, 1> <=> <id, 2> <*> <id, 3> <+> <id, 2> </> <d, 1>
```

Note: If there is any parentheses we will ignore them.

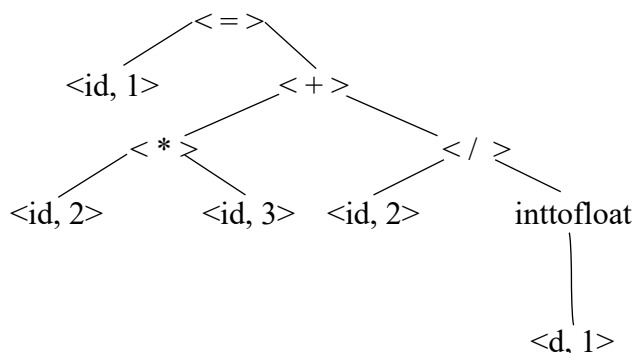
Syntax Analyzer

Now we move to the second phase of the compiler, which is Syntax Analysis. This phase is also called Parsing. We got the tokens in our last phase (lexical analysis). In the phase we will use those tokens and create a parse tree. Also note that, it's better to create a **balanced parse tree**. So, parse tree for our expression:



Semantic Analyzer

In this phase we will mainly convert the integers into float. So, the semantic tree becomes:



Intermediate Code Generation

In this phase, the compiler creates a middle language that both frontend and backend can understand. This middle language is called Intermediate Code (IC). So, Intermediate Code is a machine-independent code.

The most common intermediate code is Three Address Code (TAC).

For intermediate code generation we should follow these rules:

Decoding Compilers

1. Each instruction contains at most one operator.
2. Intermediate results are stored in temporary variables.
3. Each instruction uses at most three addresses (operands), including the result.

From the semantic tree we can generate our TAC:

```
t1 = inttofloat (<d, 1>)
t2 = <id, 2> / t1
t3 = <id, 2> * <id, 3>
t4 = t3 + t2
<id, 1> = t4
```

So, Intermediate Code Generation is the phase of the compiler that converts the annotated parse tree into an intermediate machine-independent code such as three-address code.

Code Optimization

In code optimization phase, it reorganizes IC to make it faster and smaller without changing its meaning.

We got the TAC from our last phase. If we analysis our TAC, we can see that, “**t1**” is returning us a constant. So, we can do constant folding here. It will eliminate “inttofloat(<d, 1>)” completely.

Also, we can see that, “t4” is used only once and immediately assigned to “<id, 1>”. We can eliminate “t4” and directly assign the expression to <id, 1>.

So, our optimized TAC:

```
t2 = <id, 2> / 12.0
t3 = <id, 2> * <id, 3>
<id, 1> = t3 + t2
```

In here, we can see that we have reduced our instruction by 2. Fewer instruction will helps us for faster execution.

Target Code Generation

Using the optimized TAC, we will produce machine-level instructions (or assembly) for the target processor. Notations used for TCG:

Decoding Compilers

Functions	Notation
Load	LDF
Store	STF
Addition	ADDF
Subtraction	SUBF
Multiplication	MULF
Division	DIVF

It is important to use the minimum number of registers. Using our optimized TAC, the corresponding target code can be written as:

```
LDF  R1, <id, 2>
LDF  R2, #12.0
DIVF R2, R1, R2
LDF  R3, <id, 3>
MULF R1, R1, R3
ADDF R1, R2, R1
STF  <id, 1>, R1
```

First of all, we load the value of <id,2> into register R1, and then we load the constant #12.0 into register R2. Next, we perform the division between R1 and R2, storing the result back in R2. We store it in R2 because we no longer need the constant value, so we can reuse the register efficiently. After that, we load <id,3> into R3 to prepare for multiplication. We then multiply the value in R1 with R3, storing the result back in R1. Next, we add the result of the division in R2 to the multiplication result in R1, keeping the final result in R1. Finally, we store the content of R1 into the memory location of <id,1>. This is our final result.

This step-by-step process ensures that we use the minimum number of registers, respect the correct order of operations, and generate optimized machine code directly from the TAC.

In summary, the compiler transforms a high-level expression into executable code through six systematic phases. It starts by breaking the program into tokens during lexical analysis, then organizes these tokens into a structured parse tree during syntax analysis. Semantic analysis ensures the program's meaning is correct. Intermediate code generation produces a machine-independent representation. Code optimization then improves this intermediate code for efficiency, and finally, target code generation converts it into target machine instructions. Together, these

Decoding Compilers

phases take a human-readable program and turn it into efficient, executable code, while maintaining correctness and optimizing performance.

Example-2

Describe the six phases of compiler for the expression:

ans = a * x + b * x + c

Solution:

Lexical Analyzer:

Creating the token table for the given expression:

Identifier table:

Lexeme	Token
ans	<id, 1>
a	<id, 2>
x	<id, 3>
b	<id, 4>
c	<id, 5>

Operator Table:

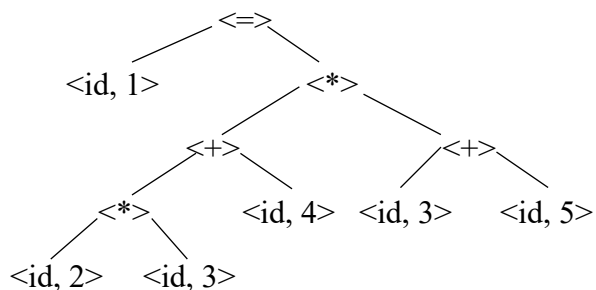
Lexeme	Token
=	<op, 1>
*	<op, 2>
+	<op, 3>

Tokenization:

<id, 1> <= > <id, 2> <*> <id, 3> <+> <id, 4> <*> <id, 3> <+> <id, 5>

Syntax Analyzer:

Creating the syntax tree from the tokens:



Decoding Compilers

Notice here, we have 5 operands in the expression. So, we can take 3 in one part and 2 in another part. This way we can balance the tree for the odd number of operands.

Semantic Analyzer:

In here, we can see that there is no constant value in the expression. So, our syntax tree and semantic tree will be same for the given expression.

ICG/TAC:

```
t1 = <id, 3> + <id, 5>
t2 = <id, 2> * <id, 3>
t3 = t2 + <id, 4>
t4 = t3 * t1
<id, 1> = t4
```

Code Optimization:

```
t1 = <id, 3> + <id, 5>
t2 = <id, 2> * <id, 3>
t3 = t2 + <id, 4>
<id, 1> = t3 * t1
```

TCG:

```
LDF    R1, <id, 3>
LDF    R2, <id, 5>
ADDF   R2, R1, R2
LDF    R3, <id, 2>
MULF   R3, R3, R1
LDF    R4, <id, 4>
ADDF   R4, R3, R4
MULF   R4, R4, R2
STF    <id, 1>, R1
```

Chapter 2 Exercises

Part A: Define / Explain

1. Define compiler architecture in your own words.
2. What is the difference between the frontend and the backend of a compiler?
3. Explain the role of the lexical analysis phase in a compiler.
4. What is intermediate code and why is it used in a compiler?
5. Explain the purpose of the symbol table in the compilation process.
6. What is code optimization? Why is it important?
7. Explain the role of code generation in a compiler.

Part B: True / False

1. The lexical analyzer checks the grammar of the program.
2. The syntax analyzer generates the parse tree.
3. Semantic analysis checks the meaning of the program.
4. Intermediate code generation produces machine code directly.
5. Code optimization improves the code without changing its meaning.
6. Code generation is the final phase of the compiler.
7. The symbol table is used only in lexical analysis.

Part C: Short Answer Questions

1. List the phases of the compiler frontend.
2. List the phases of the compiler backend.
3. What is the difference between a token and a lexeme?
4. Why are temporary variables used in three-address code?

Decoding Compilers

5. What is the difference between machine-dependent and machine-independent code?
6. Write the general form of a three-address code instruction.

Part D: Comparison / Analytical Questions

1. Compare Lexical Analysis and Syntax Analysis.
2. Compare Intermediate Code and Machine Code.
3. Compare Code Optimization and Code Generation.
4. Explain why the frontend of a compiler is machine-independent but the backend is machine-dependent.
5. Explain how the symbol table is used in different phases of the compiler.

Part E: Scenario / Story-Based Questions

1. Imagine the compiler as a factory. Explain the roles of the frontend and backend in this factory.
2. Consider the expression $a = b * c + b / 2$. Explain what happens to this expression in the lexical analysis phase.
3. For the same expression, explain what the syntax analyzer does.
4. Explain with a real-life analogy why code optimization is necessary.
5. If a compiler has very few registers available, how should the code generation phase handle this situation?

Part F: Challenge Questions

1. Describe the phases of compiler for the given statements:
 - a. $a = b * (c + d) - e / (f + g)$
 - b. $nSal = B_Sal + b1 * Exp + b2 * Edu + b3 * Mnx + b4 * Kpi + e.Er_T * 0.3$
 - c. $result = 821 + 1221 / 23 - 2.12 + b * c - d + b / 2.2$
2. Given the following TAC:

Decoding Compilers

```
t1 = a * b
t2 = c * d
t3 = t1 + t2
t4 = a * b
t5 = t3 + t4
x = t5
```

- a. Identify redundant computations
 - b. Optimize the code
 - c. Explain which optimization techniques you applied
3. A compiler has only two registers (R1 and R2) available. Generate target code for:
 - a. $a = b * c + d * e$
 - b. Explain your strategy.
4. Convert the following expression into Three-Address Code, then optimize it:
 - a. $x = (a + b) * (a + b) + (a + b)$
 - b. Explain how common subexpression elimination improves the code.

Chapter 3

INTRODUCTION TO LEXICAL ANALYSIS

Introduction

Imagine you are reading a storybook. You do not try to understand the whole page at once. First, you look at each word, then each sentence, and slowly you understand the full story.

A compiler works in a similar way. When it sees a program, it cannot understand everything at once. So, it starts from the smallest part. It breaks the program into small meaningful pieces called tokens.

This job is done by the lexical analyzer. It is like a smart reader or scanner. It reads the program character by character and separates it into useful parts. It also removes spaces and comments, because they are not needed for understanding the program.

In this chapter, we will learn how this “reader” works. We will understand tokens, patterns, regular expressions, machines like NFA and DFA, and tools like Flex. Everything will be explained in a simple and story-like way

Role of Lexical Analyzer

Think about a librarian who receives a new book. Before placing it on the shelf, the librarian checks every word carefully. The librarian separates important information and ignores unnecessary things like extra spacing or decoration.

The lexical analyzer works in the same way. It reads the program one character at a time and groups characters into meaningful words called tokens.

What does it do?

1. Reads input program
2. Removes spaces and comments
3. Breaks code into tokens
4. Sends tokens to the next phase

Decoding Compilers

Example:

Input:

```
int a = 5;
```

Output tokens:

1. `int` → keyword
2. `a` → identifier
3. `=` → operator
4. `5` → number
5. `;` → symbol

So, the lexical analyzer acts like the first reader that prepares the program for deeper understanding.

Tokens and Patterns

Tokens

A token is a small part of a program that has a meaning. It is made from one or more characters.

Imagine you are reading a sentence. You break the sentence into words. Each word has a meaning. In the same way, a program is broken into tokens.

Examples:

1. Keywords → `int`, `if`, `else`
2. Identifiers → `a`, `total`
3. Numbers → `10`, `25`

Patterns

A pattern is a rule that describes how a token looks. It tells the compiler how to recognize a group of characters.

Imagine you are checking student IDs. You may have a rule like all IDs must start with a letter and then have numbers. This rule helps you identify valid IDs.

Decoding Compilers

In the same way, patterns help the lexical analyzer identify tokens.

Example:

1. An identifier must start with a letter and can have letters or digits
2. A number must contain only digits

Lexemes, Tokens, Attributes

Imagine a small school where a teacher is checking student information. Each student comes with three things. First is the student's name. Second is the class the student belongs to. Third is extra details like roll number or marks.

Now think step by step. When a student says his name is Rahim, that name is like a lexeme. It is the actual word or text. When the teacher says Rahim is a student of Class 5, that is like a token. It tells the type or category.

When the teacher writes Rahim's roll number or marks, that is like an attribute. It gives extra information.

In the same way, a program also has words. The lexical analyzer reads each word. It understands what type it is and also stores extra information about it.

Lexeme

A lexeme is the actual word in the program.

Example: sum, 25, +

Token

A token is the type of the word.

Example:

1. sum → Identifier
2. 25 → Number

Attribute

Extra information → value or reference

Example:

1. 25 → value is 25

2. sum $\rightarrow$ memory location

Example Table:

Lexeme	Token	Attribute
count	identifier	pointer
=	operator	-
10	number	10

Regular Expressions (RE)

A regular expression is a simple way to describe patterns. Think of regular expressions as rules of a game. These rules tell what kind of words are allowed.

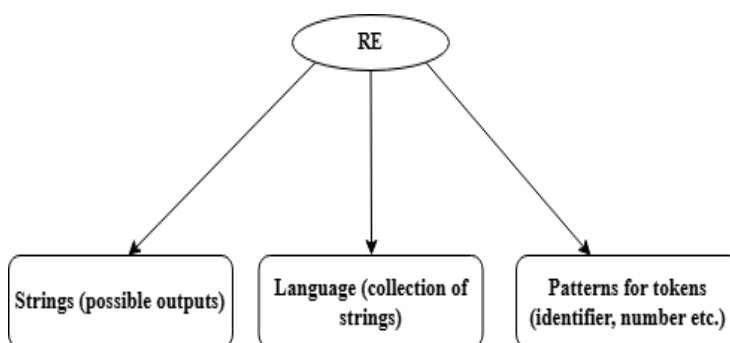
For example, if a game rule says a word must start with a letter and can have numbers after it, then any word that follows this rule is accepted.

Regular expressions are used to define these kinds of rules in programming. They describe patterns in a simple way.

For instance, a pattern for identifiers can say that it starts with a letter and continues with letters or digits. This helps the lexical analyzer recognize identifiers easily.

So, regular expressions are like clear instructions that guide the lexical analyzer.

From a Regular Expression, we can generate:



RE To Strings

RE can produce many strings.

Example 1:

Decoding Compilers

RE:

a^*

Possible strings:

1. ϵ (empty)
2. a
3. aa
4. aaa

Example 2:

RE:

ab

Possible strings:

1. ab

RE To Language

Language means a set of strings.

Example 1:

RE: zero or many 'a'

a^*

Language:

$L = \{\epsilon, a, aa, aaa, \dots\}$

Example 2:

RE:

$a \mid b$

Language:

$L = \{a, b\}$

RE To Token Pattern

Compiler uses RE to recognize tokens.

Example 1: Identifier

letter (letter | digit) *

Examples:

1. a
2. x1

Example 2: Number

digit+

Examples:

1. 5
2. 25

Finite Automata (NFA and DFA)

Imagine a small robot that reads a word letter by letter. As it reads each letter, it moves from one step to another. At the end, it decides whether the word is correct or not.

There are two types:

1. NFA (Non-deterministic Finite Automata)
2. DFA (Deterministic Finite Automata)

NFA (Non-deterministic Finite Automata)

NFA is a type of machine used to recognize patterns. It is simple and flexible.

In NFA, one input can lead to many possible states. This means from one state, the machine can move to more than one next state for the same input.

Decoding Compilers

It can also follow multiple paths at the same time. The machine does not need to choose only one path.

Sometimes, NFA can move from one state to another without any input. This is called an **epsilon (ϵ) move**.

So, NFA does not have a fixed path. It can move in different ways, which makes it easy to design.

Example:

If input = a then, It can go to state q1 or q2

So, we can see it has choices

Problem 1:

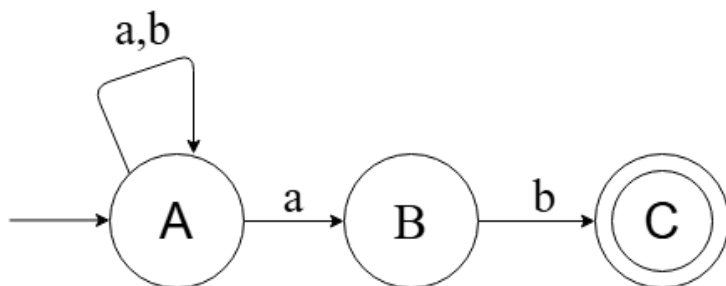
Let us take a simple NFA that accepts strings ending with "ab".

Solution:

If the rule is: strings end with "ab"

Then the language is:

$L = \{ ab, aab, bab, aaab, \dots \}$



DFA (Deterministic Finite Automata)

DFA is a type of machine used to recognize patterns. It is clear and efficient.

In DFA, one input leads to only one state. This means from one state, there is only one possible next state for each input.

It follows only one path at a time. There is no confusion or multiple choices.

DFA cannot move without input. It always needs an input to change its state.

Decoding Compilers

So, DFA has a fixed path, which makes it fast and easy to use in real systems.

Example:

If input = a

- It goes to only one fixed state

So, we can see it has no choices (Fixed)

Problem 1:

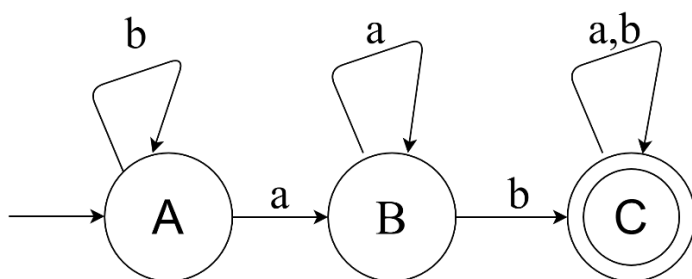
Construct a DFA that accepts all strings over $\{a,b\}$ which contain the substring "ab".

Solution:

If the rule is: all strings contain the substring "ab"

Then the language is:

$L = \{ ab, aab, bab, aaab, abab, bbab, \dots \}$



Difference between NFA and DFA

Point	DFA	NFA
Full form	Deterministic Finite Automaton	Non-deterministic Finite Automaton
Path choice	Only one path for each input	Can have multiple paths
Transition rule	Must have a transition for every input	Transition may be missing

Empty move (ϵ)	Not allowed	Allowed
Design	Harder to design	Easier to design

RE to NFA to DFA Conversion

First, you start with rules, which are regular expressions. Then you create a simple machine based on these rules. This machine is NFA. After that, you improve it into a better and faster machine, which is DFA.

This process helps in turning simple rules into a working system that can quickly recognize tokens.

Even though NFA is easier to create, DFA is better for actual use. That is why this conversion process is important.

Lex Tools (Flex)

Imagine you have to build the same machine again and again for different programs. It would take a lot of time and effort.

Flex is a tool that solves this problem. It automatically creates a lexical analyzer for you. You just need to give the rules, and Flex will generate the code.

It works like a helper that follows your instructions and builds the scanner quickly. This saves time and reduces mistakes.

Input Buffering

Think about reading a book letter by letter. It would be very slow and tiring. Instead, you read full words or lines at once.

The lexical analyzer also uses this idea. It reads a block of characters instead of one character at a time. This is called input buffering. This method makes the process faster and more efficient. It reduces the number of times the system needs to access input.

Error Handling in Lexical Analysis

Mistakes can happen while writing a program, just like spelling mistakes in writing.

The lexical analyzer must be able to find these mistakes. For example, if it sees an invalid symbol or a wrong number format, it should detect it. Instead of stopping completely, it tries to handle the error. It may skip the wrong part, show a message, and continue reading the rest of the program.

This helps the compiler to keep working even when there are small errors.

Chapter 3 Exercises

Part A: Define / Explain

1. Define lexical analysis.
2. What is a lexical analyzer?
3. Explain token.
4. What is a pattern?
5. Define lexeme.
6. What is an attribute?
7. Explain regular expression.
8. What is NFA?
9. What is DFA?
10. Explain input buffering.

Part B: True / False

1. Lexical analysis is the first phase of a compiler.
2. A token is the actual word in the program.
3. A lexeme is the type of a word.
4. Regular expressions are used to define patterns.
5. DFA can move without input (ϵ move).
6. NFA can have multiple paths.
7. Input buffering makes processing faster.
8. Lexical analyzer removes comments and spaces.

Part C: Short Answer Questions

1. What is the role of a lexical analyzer?

Decoding Compilers

2. What is the difference between token and lexeme?
3. Give two examples of tokens.
4. What is a regular expression?
5. Why is DFA used instead of NFA?
6. What is input buffering?

Part D: Comparison / Analytical Questions

1. Compare NFA and DFA.
2. Difference between token and pattern.
3. Compare lexeme and token.
4. Difference between NFA and DFA in terms of path and speed.

Part E: Scenario / Story-Based Questions

1. A compiler reads this line:
 - a. `int a = 10;`
 - b. What tokens will be produced?
2. A word starts with a number like `1abc`. Is it a valid identifier? Why?
3. A lexical analyzer finds an unknown symbol `@`. What should it do?
4. A program contains many spaces and comments. What will the lexical analyzer do with them?

Part F Challenge Questions:

1. Identify tokens, lexemes, and attributes:
 - a. `Int sum = 25;`
2. Write a pattern for identifiers.
3. Give one example of a string accepted by NFA.
4. Write the language for strings that start with a.

Chapter 4

INTRODUCTION TO SYNTAX ANALYSIS

Introduction

Syntax Analysis is the second phase of a compiler, also known as parsing. In this phase, the compiler checks whether the sequence of tokens produced by the lexical analyzer follows the grammatical rules of the programming language.

In simple words, syntax analysis checks whether the program is written in a correct structure or not. If the structure is correct, the parser creates a parse tree or syntax tree. If there are mistakes, the parser reports syntax errors.

The syntax of a programming language is usually defined using a Context-Free Grammar (CFG). The parser uses this grammar to analyze the structure of the program.

There are two main types of parsing:

1. Top-Down Parsing: Starts from the start symbol and tries to derive the string.
2. Bottom-Up Parsing: Starts from the input string and tries to reduce it to the start symbol.

In this chapter, we will learn:

1. Context-Free Grammar (CFG)
2. Parse Tree and Derivations
3. Ambiguity
4. Left Recursion and Left Factoring
5. FIRST and FOLLOW
6. Top-Down Parsing (LL(1))
7. Bottom-Up Parsing (Shift-Reduce, LR, SLR, CLR, LALR)
8. Yacc/Bison
9. Error Recovery

Syntax Analysis

Imagine a school teacher who is checking students' sentences. The teacher is not checking whether the ideas are good or bad, and not checking spelling mistakes either. The teacher is only checking whether the sentence structure is correct. For example, teacher sees this sentence, "I am going to school". This sentence is correct because it follows English grammar rules. But if a student writes, "Going I school to am." All the words are correct, but the sentence structure is wrong. So, the teacher marks it as a grammar mistake.

In compiler design, the syntax analyzer (parser) works exactly like this teacher. It does not check the meaning of the program and it does not check spelling. It only checks whether the structure of the program is correct according to grammar rules.

Before syntax analysis, there is lexical analysis. The lexical analyzer breaks the program into small parts called tokens. These tokens are like words in a sentence. Then the syntax analyzer checks whether these tokens are arranged correctly.

So, a syntax analyzer (also called a parser) is a phase of the compiler that checks whether the source program follows the correct grammar (syntax) of the programming language. It takes tokens from the lexical analyzer and builds a parse tree or syntax tree.

There are two main ways a parser checks the structure:

1. Top-Down Parsing
2. Bottom-Up Parsing

Roler of Parser

Check for Syntactic Correctness: The parser verifies that the source program follows the rules of the language. For example: In C++, a statement like `int x = ;` is syntactically incorrect because it violates the grammar of variable declaration.

Build a Parse Tree or Abstract Syntax Tree (AST): The parser constructs a hierarchical representation of the program. This tree is used in later phases for semantic checking, optimization, and code generation.

Provide a Basis for Semantic Analysis: Syntax analysis ensures that the program's structure is correct, so semantic analysis (like type checking) can be done safely.

Decoding Compilers

Facilitate Error Detection and Recovery: The parser detects syntax errors early in compilation. Some parsers can recover from errors to continue checking the rest of the program, providing multiple error reports in one compilation pass.

Bridge Between Lexical Analysis and Intermediate Code Generation: Lexical analysis groups characters into tokens. Syntax analysis arranges these tokens according to grammar rules to prepare for semantic analysis and code generation.

Top-Down Parsing

Imagine you want to build a tree diagram of a company. You start from the CEO, then go to Managers, then Employees. So, you start from the top and go down. This is called Top-Down Parsing:

1. Starts from the start symbol
2. Tries to generate the input
3. Builds parse tree from root to leaves

Bottom-Up Parsing

Now imagine you found many puzzle pieces on the floor. You start joining small pieces to make bigger parts, and finally you complete the full picture. You start from small parts and go up to the full structure. This is called Bottom-Up Parsing:

1. Starts from the input string
2. Tries to reduce it to the start symbol
3. Builds parse tree from leaves to root

Top-Down vs Bottom-Up Parsing

Top-Down Parsing	Bottom-Up Parsing
Start from the start symbol	Start from the input
Root $\rightarrow$ Leaves	Leaves $\rightarrow$ Root
Easy to understand	More Complex
Used in LL(1)	Used in LR, SLR, LALR

Context-Free Grammar (CFG)

Think of a programming language as a kitchen, and a program as a dish. The chef wants to make dishes using ingredients. But the chef must follow recipes exactly, otherwise the dish will fail. Here, ingredients are the tokens, recipe rules are the grammar rules and dish is the correct program.

A Context-Free Grammar is like a set of recipes that tells the parser how to combine tokens (ingredients) to make a valid program (dish). If you follow the recipe, the dish is valid. If you mix ingredients randomly, the dish fails — this is a syntax error.

Formal definition of CFG

A Context-Free Grammar (CFG) is a 4-tuple of:

$$G = (V, T, P, S)$$

Here,

N = Set of non-terminal symbols (like placeholders in the recipe: $\langle \text{Expression} \rangle$, $\langle \text{Statement} \rangle$)

T = Set of terminal symbols (actual tokens: id , $+$, $*$)

P = Set of production rules (the recipes themselves, e.g., $\langle \text{Expr} \rangle \rightarrow \langle \text{Expr} \rangle + \langle \text{Term} \rangle$)

S = Start symbol (main dish, e.g., $\langle \text{Program} \rangle$)

For a CFG (Context-Free Grammar), we can remember that everything on the left-hand side is a non-terminal, and everything on the right-hand side that is not a non-terminal is a terminal.

Example-1:

Let us consider production rules:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

Here,

$$N = \{ E, T, F \}$$

$$T = \{ +, *, (,), \text{id} \}$$

Decoding Compilers

$S = \{ E \}$

P = are given on the question

LMD & RMD

In syntax analysis, derivation is the sequence of steps by which a string of terminals (a program or expression) is generated from the start symbol using a context-free grammar (CFG).

Leftmost Derivation (LMD): In a leftmost derivation, at each step, the leftmost non-terminal is replaced first. Note that, LMD is used in top-down parsing (e.g., LL parsers).

Example-2:

Using LMD, generate the input string: “id + id * id” for the given CFG:

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Solution for example-2:

$E \rightarrow E + T$
 $\rightarrow T + T$ (since, $E \rightarrow T$)
 $\rightarrow F + T$ (since, $T \rightarrow F$)
 $\rightarrow \text{id} + T$ (since, $F \rightarrow \text{id}$)
 $\rightarrow \text{id} + T * F$ (since, $T \rightarrow T * F$)
 $\rightarrow \text{id} + F * F$ (since, $T \rightarrow F$)
 $\rightarrow \text{id} + \text{id} * F$ (since, $F \rightarrow \text{id}$)
 $\rightarrow \text{id} + \text{id} * \text{id}$ (since, $F \rightarrow \text{id}$)

Rightmost Derivation (RMD): In a rightmost derivation, at each step, the rightmost non-terminal is replaced first. RMD is used in bottom-up parsing (e.g., LR parsers) in reverse order.

Example-3:

Decoding Compilers

Using RMD, generate the input string: “id + id * id” for the given CFG:

$$\begin{array}{l} E \rightarrow E + T \mid T \\ T \rightarrow T * F \mid F \\ F \rightarrow (E) \mid \text{id} \end{array}$$

Solution for example-2:

$E \rightarrow E + T$
 $\rightarrow E + T * F$ (since, $T \rightarrow T * F$)
 $\rightarrow E + T * \text{id}$ (since, $F \rightarrow \text{id}$)
 $\rightarrow E + F * \text{id}$ (since, $T \rightarrow F$)
 $\rightarrow E + \text{id} * \text{id}$ (since, $F \rightarrow \text{id}$)
 $\rightarrow T + \text{id} * \text{id}$ (since, $E \rightarrow T$)
 $\rightarrow F + \text{id} * \text{id}$ (since, $T \rightarrow F$)
 $\rightarrow \text{id} + \text{id} * \text{id}$ (since, $F \rightarrow \text{id}$)

Parse Tree

A parse tree is a tree representation of the derivation of a string from a grammar. It shows how the start symbol of a grammar generates the input string according to the production rules.

Components of a parse tree:

1. Root: The root of the tree is the start symbol of the grammar.
2. Internal Nodes: Internal nodes are non-terminal symbols.
3. Leaves: Leaves are terminal symbols (tokens) of the input string.
4. Edges: Edges represent applications of production rules.

Purpose of a Parse Tree:

1. Shows the syntactic structure of the input string.
2. Helps in syntax checking.
3. Forms the basis for semantic analysis and code generation.
4. Helps visualize leftmost and rightmost derivations.

Example-4:

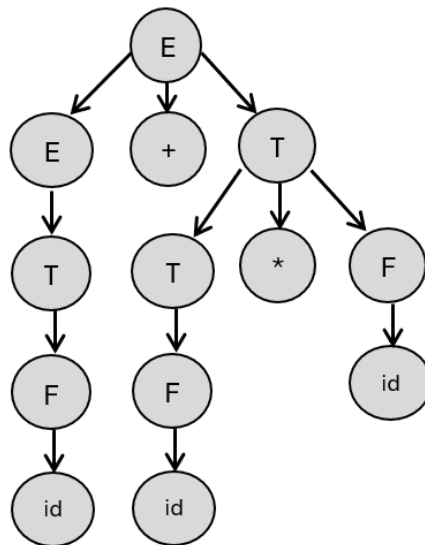
Decoding Compilers

Generate the parse tree for the input string: “id + id * id” for the given CFG:

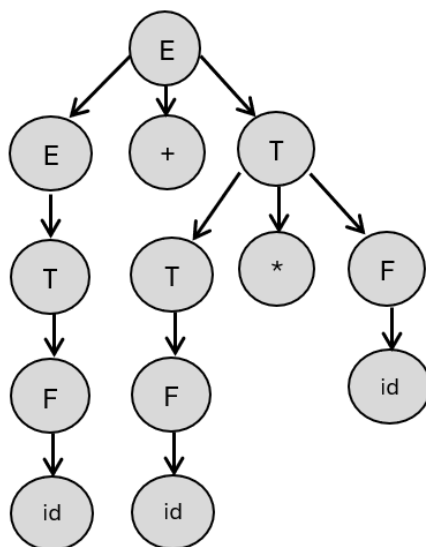
$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Solution for example-4:

From LMD (example-2):



From RMD (example-3):



Ambiguity in Grammar

An Ambiguous Grammar is a context-free grammar in which there exists two or more derivation tree for a single string.

Steps to determine grammar ambiguity

1. Find the left-most derivation for the given string and draw its parse tree.
2. Find the right-most derivation for the given string and draw its parse tree.
3. If both parse trees are same, then the grammar is not ambiguous.
4. If the parse trees are different, then the grammar is ambiguous.

For example, we can consider our example-4. In there we can see that, for string “id + id * id” it has same parse tree for both LMD and RMD. So, we can say that it is not ambiguous.

Example-5:

Find the ambiguity

Grammar	Expression
$E \rightarrow E + E$	id + id * id
$E \rightarrow E * E$	

$E \rightarrow id$	
--------------------	--

Solution for example-5:

Finding the left-most derivation for the given string:

$E \rightarrow E + E$

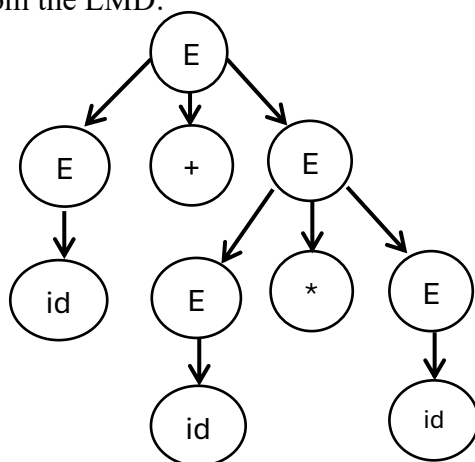
$\rightarrow id + E$ (since, $E \rightarrow id$)

$\rightarrow id + E * E$ (since, $E \rightarrow E * E$)

$\rightarrow id + id * E$ (since, $E \rightarrow id$)

$\rightarrow id + id * id$ (since, $E \rightarrow id$)

Parse tree from the LMD:



Now, finding the right-most derivation for the given string:

$E \rightarrow E * E$

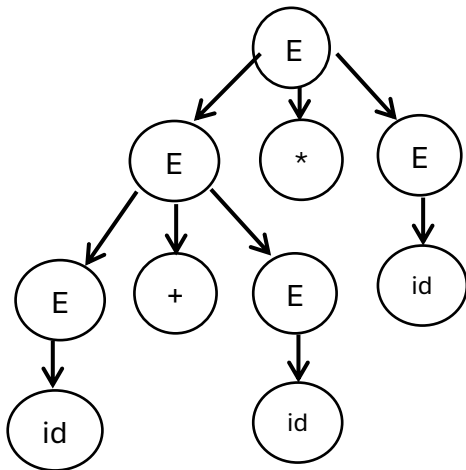
$\rightarrow E * id$ (since, $E \rightarrow id$)

$\rightarrow E + E * id$ (since, $E \rightarrow E + E$)

$\rightarrow E + id * id$ (since, $E \rightarrow id$)

$\rightarrow id + id * id$ (since, $E \rightarrow id$)

Parse tree from the RMD:



Here, we can see that, the parse tree for the LDM and the parse tree for the RMD are different for the string “id + id * id”. So, we can say that the grammar is ambiguous.

Problems of Ambiguity

Ambiguity in a grammar is problematic in compiler design because it prevents the parser from determining a unique structure for a program. This leads to uncertainty in parsing, evaluation, and code generation. Here’s why it matters:

Multiple Parse Trees: If a string has more than one parse tree, the compiler cannot decide which structure to follow.

For example: if a grammar is: $E \rightarrow E + E \mid E * E \mid id$ and input is “id + id * id” then from here get can get 2 different parse trees.

Parse Tree 1: (id + id) * id

Parse Tree 2: id + (id * id)

The same string produces different interpretations.

Confusion in Operator Precedence: Ambiguous grammar does not enforce precedence or associativity. Without precedence rules, “id + id * id” could be evaluated incorrectly. This can lead to wrong computation in programs.

Parsing Difficulty: Ambiguous grammars are hard to parse automatically. Many parsing algorithms (especially top-down parsers like LL(1)) cannot handle

Decoding Compilers

ambiguity. Bottom-up parsers may also produce shift/reduce conflicts when ambiguity exists.

Semantic Conflicts: The meaning of the program depends on the parse tree. Multiple parse trees may produce different semantics. This can lead to unexpected program behavior or compiler errors.

Compiler Complexity: Ambiguity increases the complexity of the parser. The compiler must include extra rules or manual disambiguation, which makes design harder.

Operator Precedence and Associativity

Operator precedence decides which operator in an expression is done first. Associativity decides how operators of the same precedence are grouped or calculated. These rules help avoid confusion and make sure the expression is calculated correctly. Some common operators, precedence, and associativity is given below:

Operator(s)	Precedence Level	Associativity
()	1 (highest)	Left-to-right
+, -, !	2	Right-to-left
*, /, %	3	Left-to-right
+, -	4	Left-to-right
<, >, <=, >=	5	Left-to-right
==, !=	6	Left-to-right
&&	7 (Lowest)	Left-to-right

Grammar Transformations

Imagine you are the manager of a busy restaurant, and your chefs follow a recipe book (grammar) to cook dishes (generate strings/programs). But sometimes, the recipes are confusing or inefficient, making it hard for the chefs to decide what to cook first or how to proceed. Grammar transformations are like fixing the recipe book so that the chefs can follow it easily and correctly.

So, grammar transformations are techniques used to modify context-free grammars (CFGs) to make them suitable for parsing, especially top-down parsers like LL(1). These transformations help eliminate ambiguity, left recursion, and common

prefixes, making parsing deterministic and easier to implement. The main transformations are:

Left Recursion Removal

Imagine you are teaching a new chef in your restaurant. You give them a recipe that says, “To make the sauce, first make the sauce, then add herbs.” The chef gets confused and keeps trying to make the sauce again and again, never finishing it. In compiler terms, this is like left recursion in a grammar.

A grammar is left-recursive if a non-terminal calls itself as the first symbol in one of its productions:

$$A \rightarrow A \alpha \mid \beta$$

Here,

A is the non-terminal (the recipe steps), α is some sequence that follows the recursive call, and β is a step that does not call itself first.

Because of left recursion, top-down parsers (LL parsers) will loop infinitely when trying to parse left-recursive rules. In order to remove left-recursion we can “rewrite the recipe” so that the chef starts with a non-recursive step first, and then adds the repeating part.

In order to remove left-recursion we can rewrite the recursive form ($A \rightarrow A \alpha \mid \beta$) into this form:

$$\begin{aligned} A &\rightarrow \beta A' \\ A' &\rightarrow \alpha A' \mid \epsilon \end{aligned}$$

Here,

A' is a new helper non-terminal, and ϵ represents the empty string (finish the step).

This ensures the chef (parser) can finish the recipe (parse the string) without infinite looping.

Example-6:

Remove the left-recursion from the grammar:

$$E \rightarrow E + T \mid T$$

Solution for example-6:

We can rewrite the grammar as:

Decoding Compilers

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

Example-7:

Remove the left-recursion from the grammar:

$$A \rightarrow A b$$

Solution for example-7:

$$A \rightarrow A'$$

$$A' \rightarrow b A' \mid \varepsilon$$

Example-8:

Remove the left-recursion from the grammar:

$$A \rightarrow A B d \mid A a \mid a$$

$$B \rightarrow B c \mid b$$

Solution for example-8:

$$A \rightarrow a A'$$

$$A' \rightarrow B d A' \mid a A' \mid \varepsilon$$

$$B \rightarrow b B'$$

$$B' \rightarrow c B' \mid \varepsilon$$

Example-9:

Remove the left-recursion from the grammar:

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L S \mid S$$

Solution for example-9:

$$S \rightarrow (L) \mid a$$

$$L \rightarrow S L'$$

$$L' \rightarrow S L' \mid \varepsilon$$

Example-10:

Remove the left-recursion from the grammar:

$$A \rightarrow B a \mid b$$

$$B \rightarrow A c \mid d$$

Solution for example-10:

Decoding Compilers

Here, A calls B. Then B calls A c. Eventually A can produce itself at the beginning of a production. This is called **indirect left recursion** because A indirectly calls itself through B.

Step 1: Substitute B in A:

$$\begin{aligned} A &\rightarrow B a \mid b \\ &\rightarrow (A c) a \mid d a \mid b \\ &\rightarrow A c a \mid d a \mid b \end{aligned}$$

Here, “ $A \rightarrow A c a$ ” shows direct left recursion.

Step 2: Remove left recursion for A:

$$\begin{aligned} A &\rightarrow d a A' \mid b A' \\ A' &\rightarrow c a A' \mid \varepsilon \end{aligned}$$

Step 3: Rewrite B using new A:

$$\begin{aligned} B &\rightarrow A c \mid d \\ B &\rightarrow d a A' c \mid b A' c \mid d \end{aligned}$$

Final grammar without left recursion:

$$\begin{aligned} A &\rightarrow d a A' \mid b A' \\ A' &\rightarrow c a A' \mid \varepsilon \\ B &\rightarrow d a A' c \mid b A' c \mid d \end{aligned}$$

Left factoring

Imagine you are the head chef in a busy restaurant. One day, two assistants come to you with similar recipes: “If the main ingredient is eggs, cook eggs then add milk”, “If the main ingredient is eggs, cook eggs then add chocolate.” Both recipes start the same way: “If the main ingredient is eggs, cook eggs...”. Your assistants are confused about which recipe should they follow first?

This is exactly like a grammar where two productions start with the same symbol. Top-down parsers (like your chefs) cannot decide which production to pick. In order to solve this issue of common prefix we use left factoring technique.

Left factoring is a technique that removes common prefixes in grammar rules so the parser can make a clear choice based on the next input symbol.

If a common prefix grammar is like this format:

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2 \mid \dots \mid \gamma_1 \mid \gamma_2 \mid \dots$$

Decoding Compilers

Then we can rewrite it as:

$$A \rightarrow \alpha A' \mid \gamma_1 \mid \gamma_2 \mid \dots$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots$$

Example-11:

Remove the common prefix from the grammar:

$$S \rightarrow iEtS \mid iEtSeS \mid a$$

$$E \rightarrow b$$

Solution for example-11:

$$S \rightarrow iEtS S' \mid a$$

$$S' \rightarrow eS \mid \varepsilon$$

$$E \rightarrow b$$

FIRST

FIRST is a set used in compiler design to determine which terminals can appear at the beginning of a string derived from a grammar symbol. It is essential for constructing predictive parsers (LL(1) parsers).

For a grammar symbol X (terminal or non-terminal), $\text{FIRST}(X)$ is the set of terminals that can appear first in some string derived from X . Follow 3 rules:

1. Remove left-recursion (if exists)
2. Perform left-factoring
3. If X is a terminal, $\text{FIRST}(X)$ is $\{X\}$
4. If X is a non-terminal, $\text{FIRST}(X)$ is computed from its productions.
5. If X can derive the empty string ε , then $\varepsilon \in \text{FIRST}(X)$

Example-12:

Find the FIRST for the given grammar:

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$F \rightarrow (E) \mid \text{id}$$

Tips: Always start with the last production rule when dealing with FIRST.

Solution for example-12:

Production	FIRST()
$E \rightarrow T E'$	$\{ (, \text{id} \}$
$E' \rightarrow + T E' \mid \varepsilon$	$\{ +, \varepsilon \}$

Decoding Compilers

$T \rightarrow F T'$	$\{ (, id \}$
$F \rightarrow (E) \mid id$	$\{ (, id \}$

Explanation:

FIRST (F) = (, id (as both of them are terminal)

FIRST (T) = FIRST (F) (as F is a non-terminal)

= (, id

Same goes for FIRST (E) = FIRST (T)

= FIRST (F)

= (, id

FOLLOW

FOLLOW is a fundamental concept in compiler design, used primarily in predictive (LL(1)) parsing to determine what symbols can appear immediately after a non-terminal.

For a non-terminal A, FOLLOW(A) is the set of terminals that can come right after A in any part of a sentence or expression. Note that, it also includes the end-of-input marker '\$' if A can appear at the end of the input.

Rules for Computing FOLLOW:

1. Remove left-recursion (if exists)
2. Perform left-factoring
3. If A is the start symbol of the grammar, add \$ to FOLLOW(A).
4. If a terminal symbol appears immediately after A, add that terminal to FOLLOW(A)
5. If a non-terminal symbol B appears immediately after A, add FIRST(B) (except ϵ) to FOLLOW(A)
6. If a non-terminal is at the end of a production, or the symbols after it can produce ϵ , add FOLLOW of the left-hand side (LHS) non-terminal to FOLLOW of the non-terminal

Example-12:

Compute the FOLLOW for the given grammar:

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

$T \rightarrow F T'$

$T' \rightarrow * F T' \mid \epsilon$

Decoding Compilers

$F \rightarrow (E) \mid id$

Note that, in order to find FOLLOW, we have to find FIRST.

Solution for example-12:

Production Rules	FIRST	FOLLOW
$E \rightarrow T E'$	$\{ (, id \}$	$\{ \$,) \}$
$E' \rightarrow + T E' \mid \epsilon$	$\{ +, \epsilon \}$	$\{ \$,) \}$
$T \rightarrow F T'$	$\{ (, id \}$	$\{ +, \$,) \}$
$T' \rightarrow * F T' \mid \epsilon$	$\{ *, \epsilon \}$	$\{ +, \$,) \}$
$F \rightarrow (E) \mid id$	$\{ (, id \}$	$\{ *, +, \$,) \}$

Explanation:

$FOLLOW(E) = \{ \$ \}$ (since, E is starting symbol and there is no more E on the right side. Also, $FOLLOW(E) =)$ (from production 5)

$FOLLOW(E') = FOLLOW(E)$ (since, in production 1 there is nothing after E')

$FOLLOW(T) = FIRST(E')$ (From production 1). Now in here, we can replace E' with it's FIRST (i.e +, ϵ). So, production 1 becomes:

$E \rightarrow T + \mid T \epsilon$. (Temporary)

Now we can find the $FOLLOW(T)$ on new Production 1.

So $FOLLOW(T) = +$, $FOLLOW(E) = +, \$,)$

Again, $FOLLOW(T) = FIRST(E')$ (from production 2) = then repeat the process.

Note, we don't need to write duplicate values.

LL(1) Parsing

Imagine again your magical restaurant kitchen. Now the restaurant is very famous, and many orders are coming. The chefs must cook very fast and must not make mistakes. So, you create a smart rule system so that the chef can decide what to do by looking at only the next ingredient. This smart system is called LL(1) Parsing.

In simple we can say, LL(1) parsing is a top-down parsing method used in compiler design to check whether a string belongs to a grammar.

LL(1) actually has a meaning:

Symbol	Meaning
L	Scan input from Left to right
L	Produce Leftmost derivation

1	Use 1 lookahead symbol
---	------------------------

So, LL(1) parser read the input left to right, builds left-most derivation and looks at only 1 next symbol to decide the rule.

For example, if a customer give an order of “id + id * id”, then the chef looks at the first ingredient “id”. He uses a decision table (LL(1) parsing table) to check which recipe step (production) to use. The chef never guesses, never goes back. He just follows the table.

Advantages of LL(1) parser

1. No backtracking
2. Fast
3. Deterministic
4. Easy to implement
5. Without LL(1) the parser:
 - a. May get confused
 - b. May need backtracking
 - c. May go to infinite recursion
 - d. May be slow

Conditions for LL(1) Grammar

A grammar can be LL(1) only if:

1. No left recursion
2. Left-factoring done
3. For each production: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$
4. If ϵ is in $\text{FIRST}(\alpha)$, then: $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$

This ensures the parser can decide using only one lookahead symbol.

LL(1) Parsing Table

An LL(1) Parsing Table is a decision table used by an LL(1) parser to decide which production rule to use based on the non-terminal on the stack (row) and the next input symbol (column).

So, the parsing table helps the parser choose the correct production without backtracking.

Example-13:

Find the LL(1) parsing table for the given grammar:

$S \rightarrow AaB$

$A \rightarrow a$

$B \rightarrow \epsilon$

Solution for example-13:

In order to find LL(1) parsing table first we need to find the FIRST and FOLLOW for the given grammar.

Finding the FIRST and the FOLLOW for the given grammar:

Production	FIRST	FOLLOW
$S \rightarrow AaB$	{ a }	{ \$ }
$A \rightarrow a$	{ a }	{ a }
$B \rightarrow \epsilon$	{ ϵ }	{ \$ }

Now we need to create LL(1) parsing table. To construct the LL(1) parsing table, we place all the terminals of the grammar column-wise (including the end marker \$, which is added as an extra column). Then we place all the non-terminals row-wise.

Each cell of the table represents a production rule that will be used when a particular non-terminal is on the top of the stack and a particular terminal is the current input symbol. The entries in the table are filled using the FIRST and FOLLOW sets of the grammar.

In order to find cell value, we have to use these 3 rules:

For each production ($A \rightarrow \alpha$):

1. For each terminal $a \in \text{FIRST}(\alpha)$, add: $M[A, a] = A \rightarrow \alpha$
2. If $\epsilon \in \text{FIRST}(\alpha)$, then for each $b \in \text{FOLLOW}(A)$, add: $M[A, b] = A \rightarrow \epsilon$
3. If $\epsilon \in \text{FIRST}(\alpha)$ and $\$ \in \text{FOLLOW}(A)$, add: $M[A, \$] = A \rightarrow \epsilon$

Finding the LL(1) parsing table:

Non-Terminal	a	\$
S	$S \rightarrow AaB$	
A	$A \rightarrow a$	
B		$B \rightarrow \epsilon$

Explanation:

For Production 1: $S \rightarrow A a B$

Decoding Compilers

We find $\text{FIRST}(S)$. Since the production starts with A, we look at $\text{FIRST}(A)$. We know that $\text{FIRST}(A) = \{ a \}$, so $\text{FIRST}(S) = \{ a \}$. Therefore, we place the production $S \rightarrow AaB$ in the parsing table cell $M[S, a]$.

For Production 2: $A \rightarrow a$

Here, $\text{FIRST}(A) = \{ a \}$. So, we place the production $A \rightarrow a$ in the parsing table cell $M[A, a]$.

For Production 3: $B \rightarrow \epsilon$

Since this production derives ϵ (empty string), we use the FOLLOW set. We find $\text{FOLLOW}(B)$. Since B is at the end of the production $S \rightarrow A a B$, we have: $\text{FOLLOW}(B) = \text{FOLLOW}(S) = \{ \$ \}$. Therefore, we place the production $B \rightarrow \epsilon$ in the parsing table cell $M[B, \$]$.

Example-14:

Find the LL(1) parsing table for the given grammar:

$S \rightarrow S a \mid S b \mid c d \mid c e$

Solution for example-14:

Removing the left-recursion for the given grammar:

$S \rightarrow c d S' \mid c e S'$

$S' \rightarrow a S' \mid b S' \mid \epsilon$

Applying left factoring:

$S \rightarrow c X$

$X \rightarrow d S' \mid e S'$

$S' \rightarrow a S' \mid b S' \mid \epsilon$

Finding the FIRST and the FOLLOW:

Production Rule	FIRST	FOLLOW
$S \rightarrow c X$	$\{ c \}$	$\{ \$ \}$
$X \rightarrow d S' \mid e S'$	$\{ d, e \}$	$\{ \$ \}$
$S' \rightarrow a S' \mid b S' \mid \epsilon$	$\{ a, b, \epsilon \}$	$\{ \$ \}$

LL(1) parsing table:

	a	b	c	d	e	\$
S			$S \rightarrow cX$			
X				$X \rightarrow dS'$	$X \rightarrow eS'$	
S'	$S' \rightarrow aS'$	$S' \rightarrow bS'$				$S' \rightarrow \epsilon$

Recursive Descent Parser

A Recursive Descent Parser (RDP) is a top-down parsing technique used to process and analyze the syntax of a program based on a grammar. It is one of the simplest and most intuitive methods to implement an LL(1) parser.

Key features of RDP

1. Top-down parsing: The parser starts from the start symbol and tries to expand it to match the input string.
2. Function per non-terminal: Each non-terminal in the grammar is implemented as a separate function or procedure.
3. Recursive calls: When a non-terminal appears on the right-hand side of a production, the corresponding function is called recursively.
4. Lookahead symbol: The parser keeps track of the current input symbol (called lookahead) to decide which production to use.
5. Epsilon handling: If a non-terminal can produce ϵ (empty string), the parser simply does nothing and returns control to the previous call.

Structure of a Recursive Descent Parser

Input string: The program or string to be parsed, usually terminated with a special symbol \$ to indicate the end.

Lookahead: A variable that points to the current symbol being read from the input.

Match function: Compares the lookahead with the expected terminal. If it matches, the parser moves to the next symbol; otherwise, it reports a syntax error.

Non-terminal functions: Implement the grammar rules using if-else statements to choose the correct production based on the lookahead symbol.

Example-15:

Create a complete Recursive Descent Parser (RDP) program for the following grammar:

$S \rightarrow c X$

$X \rightarrow d S' \mid e S'$

$S' \rightarrow a S' \mid b S' \mid \epsilon$

Solution for example-15:

Note that you can use any programming language you want. We will solve it using JavaScript. But the core logic will be same for every language.

Core idea to solve:

1. Start from the start symbol of the grammar (S) and try to match the input string.
2. Each non-terminal in the grammar is implemented as a function.
3. Each function checks the current input symbol (lookahead) to decide which production rule to apply.
4. If a terminal symbol is expected, the parser matches it and moves to the next symbol.
5. If a non-terminal symbol is encountered, the parser calls the corresponding function recursively.
6. ϵ -productions are handled implicitly by doing nothing when the lookahead does not match any terminal in the production.
7. If a mismatch occurs, the parser reports a syntax error and stops.
8. The input is accepted if all symbols are consumed and no syntax errors occur.

Using JavaScript:

```
// Input string with '$' at the end
let input = "cdaab$";
let index = 0;
let lookahead = input[index];

// Move to next input symbol
function next() {
    index++;
    lookahead = input[index] || '$'; // End of input
}

// Error handling
function error(msg) {
    console.error(`${msg} at position ${index}`);
    throw new Error("Parsing stopped");
}

// Match terminal
function match(token) {
    if (lookahead === token) {
        console.log(`Matched: ${token}`);
    }
}
```

Decoding Compilers

```
        next();
    } else {
        error(`Syntax error: expected '${token}', found
'${lookahead}'`);
    }
}

// Non-terminal functions
function S() {
    console.log("Enter S()");
    if (lookahead === 'c') {
        match('c');
        X();
    } else {
        error("Syntax error in S");
    }
    console.log("Exit S()");
}

function X() {
    console.log("Enter X()");
    if (lookahead === 'd') {
        match('d');
        SPrime();
    } else if (lookahead === 'e') {
        match('e');
        SPrime();
    } else {
        error("Syntax error in X");
    }
    console.log("Exit X()");
}

function SPrime() {
    console.log("Enter S'()");
    if (lookahead === 'a') {
        match('a');
        SPrime();
    } else if (lookahead === 'b') {
        match('b');
        SPrime();
    }
    //  $\epsilon$ -production  $\rightarrow$  do nothing
    console.log("Exit S'()");
}
```



```
// Start parsing
try {
    S();
    if (lookahead === '$') {
        console.log("Input string is accepted by grammar.");
    } else {
        console.log("Syntax error: input not fully consumed.");
    }
} catch (e) {
    console.error(e.message);
}
```

Bottom-Up Parsing

Imagine you are a detective solving a mystery, but instead of starting with the crime scene and guessing what happened, you start with all the evidence collected from the scene and gradually work backwards to reconstruct the full story.

Each piece of evidence is like a terminal symbol (actual input symbol) in a program. The final conclusion or reconstructed story is like the start symbol of the grammar.

The detective tries to combine small pieces of evidence into bigger clues, eventually reconstructing the entire sequence of events.

In programming language terms, bottom-up parsing works by starting from the input string (the leaves of a parse tree). Gradually reducing sequences of terminals and non-terminals into higher-level non-terminals using the production rules. Continuing this process until the start symbol of the grammar is reached.

Essentially, bottom-up parsing is like building the parse tree from leaves to root, opposite of top-down parsing (which starts from the root and tries to match leaves).

Key Concepts of Bottom-Up Parsing

Shift and Reduce: Shift reads the next input symbol and push it onto the stack. Reduce match a pattern on the top of the stack that corresponds to the right-hand side (RHS) of a production, and replace it with the left-hand side (LHS) non-terminal.

Handles: A handle is a substring of the input that matches the RHS of a production and can be reduced to the LHS. Finding the handle is the core operation in bottom-up parsing.

Decoding Compilers

From the detective analogy: input string evidence collected from the scene (terminals). Stack is the notebook where you record clues (partial parse). Shift is collecting next piece of evidence and add it to the notebook. Reduce is realize a pattern among clues and write a summarized clue in the notebook (replace RHS with LHS). Goals eventually, your notebook contains only one ultimate clue → the start symbol → the case is solved!

Types of Bottom-Up Parsers:

1. Simple Shift-Reduce Parsing
2. SLR (Simple LR) Parsing
3. LR Parsing
4. LALR Parsing
5. CLR Parsing

Shift-Reduce Parsing

Shift-Reduce parsing is a bottom-up parsing technique that uses a stack to keep track of partially processed symbols. There are two main actions:

Shift: Take the next input symbol and push it onto the stack. Analogy: Picking up an ingredient from the table and placing it in your mixing bowl.

Reduce: Look at the top symbols of the stack and see if they match the right-hand side (RHS) of a production. If yes, replace them with the left-hand side (LHS) non-terminal. Analogy: Mixing ingredients together according to the recipe to form a layer of the cake.

Steps in Shift-Reduce Parsing:

1. Initialize the stack as empty.
2. Repeat until input is consumed:
 - a. Shift the next input symbol onto the stack.
 - b. Check the stack from top to see if a sequence matches a production RHS (handle).
 - c. If a handle is found, reduce it to the corresponding LHS.
3. Accept if the stack contains only the start symbol and the input is fully read.
4. Error if no shift or reduce is possible but the stack/input is not yet accepted.

Decoding Compilers

Example-16:

Let us consider a grammar:

$S \rightarrow c X$

$X \rightarrow d S' \mid e S'$

$S' \rightarrow a S' \mid b S' \mid \epsilon$

And input string: $c d a b \$$

We'll show stack, remaining input, and action at each step:

Step	Stack	Input	Action
1	ϵ	$c d a b \$$	Shift c
2	c	$d a b \$$	Shift d
3	$c d$	$a b \$$	Reduce $d S' \rightarrow X$? Wait, check handle: d matches RHS of $X \rightarrow d S'$ but S' can be ϵ . So reduce $d \rightarrow X$? Actually handle = $d \rightarrow X$ after S' processed
4	$c d$	$a b \$$	Reduce $d S' \rightarrow X$ after shifting S'
5	$c d a$	$b \$$	Shift a
6	$c d a$	$b \$$	Reduce $b \rightarrow S'$
7	$c d S'$	$b \$$	Reduce $d S' \rightarrow X$
8	$c d S' b$	$\$$	Reduce $c X \rightarrow S$
9	$c d S'$	$\$$	Accept
10	$c x$	$\$$	Accept
11	S	$\$$	Accept

Step-by-Step Explanation:

1. Shift c onto stack $\rightarrow$ stack = c
2. Shift $d \rightarrow$ stack = $c d$
3. Shift $a \rightarrow$ stack = $c d a$
4. Reduce $a \rightarrow S' \rightarrow$ stack = $c d S'$
5. Shift $b \rightarrow$ stack = $c d S' b$
6. Reduce $b \rightarrow S' \rightarrow$ stack = $c d S'$
7. Reduce $d S' \rightarrow X \rightarrow$ stack = $c X$
8. Reduce $c X \rightarrow S \rightarrow$ stack = S
9. Input consumed & stack = start symbol $\rightarrow$ Accept

Key Observations:

1. The parser shifts symbols first until a handle is found.
2. Reduction happens only when the top of the stack matches a production RHS.
3. ϵ -productions are handled implicitly (no symbol is pushed for ϵ).
4. The process is fully bottom-up, building the parse tree from leaves to root

Operator Precedence Parsing

Operator precedence parsing is a bottom-up parsing technique used for expressions in programming languages. It is mainly applied when expressions contain operators like +, -, *, /, and parentheses, where the order of evaluation matters. Think of it like solving a math problem. You know that * comes before + and () always come first. The parser uses operator precedence rules to decide which operations to perform first without building a full parse tree.

How It Works:

Imagine you're in a restaurant kitchen. You have a stack of ingredients (numbers) and orders for cooking steps (operators). Each operator has a priority: some steps must be done before others. Multiplication (*) has higher priority than addition (+). You push ingredients and operators onto the stack, but whenever a higher precedence operator comes, you cook it first. Once all ingredients and steps are processed, the dish (expression) is ready.

This is exactly what an operator precedence parser does. It decides when to reduce based on the precedence and associativity of operators.

Steps in Operator Precedence Parsing

1. Define Precedence Table: Compare every pair of terminals (operators + \$) and determine whether one yields precedence (<), has precedence (>), or is equal (=).
2. Use a Stack for Parsing: Start with a stack containing \$. Read input symbols one by one.
3. Shift or Reduce:
 - a. Shift if the current input has higher precedence than the top of the stack.
 - b. Reduce when the stack top has higher precedence than the input.
 - c. Continue until stack = \$ S \$ (start symbol in stack).

Example-17:

We'll consider a simple expression grammar: $E \rightarrow E + E \mid E * E \mid id$

And input string: `id + id * id`

Now, Stack/Input Table:

Step	Stack	Input	Action
1	\$	id + id * id \$	Shift id

Decoding Compilers

2	\$ id	+ id * id \$	Reduce id $\rightarrow$ E
3	\$ E	+ id * id \$	Shift +
4	\$ E +	id * id \$	Shift id
5	\$ E + id	* id \$	Reduce id $\rightarrow$ E
6	\$ E + E	* id \$	Shift *
7	\$ E + E *	id \$	Shift id
8	\$ E + E * id	\$	Reduce id $\rightarrow$ E
9	\$ E + E * E	\$	Reduce E * E $\rightarrow$ E
10	\$ E + E	\$	Reduce E + E $\rightarrow$ E
11	\$ E	\$	Accept

LR(0) Parsing

LR(0) parsing is a bottom-up parsing technique that reads the input from left to right (L) and produces a rightmost derivation in reverse (R). It is called LR(0) because:

1. L $\rightarrow$ Left-to-right scanning of the input
2. R $\rightarrow$ Rightmost derivation in reverse
3. 0 $\rightarrow$ Uses 0 lookahead symbols to make parsing decisions

CFG to LR(0) Parsing Table

Step 1: Augment the Grammar

Add a new start symbol S' and a production $S' \rightarrow S$, where S is the original start symbol. This helps the parser know when the input is fully parsed.

Original Grammar: $S \rightarrow A A$	Augmented Grammar: $S' \rightarrow S$ $S \rightarrow A A$ $A \rightarrow a A$
---	---

Step 2: Create LR(0) Items

For each production $A \rightarrow \alpha$, create items by placing a dot ($\bullet$) at every position in the right-hand side. The dot shows how much of the production has been seen. Example items for $A \rightarrow a A$:

$A \rightarrow \bullet a A$ (nothing seen yet)

$A \rightarrow a \bullet A$ (a seen)

Decoding Compilers

$A \rightarrow a A \bullet$ (entire RHS seen)

Step 3: Compute Closure of Item Sets

Closure(I): For each item in I of the form $B \rightarrow \bullet X \beta$, if X is a non-terminal, add all productions of X with dot at start. Repeat until no new items can be added.

Step 4: Construct DFA of Item Sets

Treat each item set as a state. For each symbol X (terminal or non-terminal), move the dot past X in all items $\rightarrow$ this forms a new item set. Draw transitions from current state to new state on symbol X. Repeat until no new states are formed.

Step 5: Define ACTION and GOTO Tables

1. Terminals $\rightarrow$ columns in ACTION table.
2. Non-terminals $\rightarrow$ columns in GOTO table.
3. Rows $\rightarrow$ item sets (states)

Rules:

1. Shift: If moving dot past terminal a leads to state j, set $\text{ACTION}[i, a] = s_j$
2. Reduce: If item is $A \rightarrow \alpha \bullet$, set $\text{ACTION}[i, a] = r$ for all a in $\text{FOLLOW}(A)$
3. Accept: If item is $S' \rightarrow S \bullet$ and input is \$, set $\text{ACTION}[i, \$] = \text{acc}$
4. Goto: For non-terminal A and state i, if moving dot past A leads to state j, set $\text{GOTO}[i, A] = j$

Example-18:

Make a complete LR(0) parsing table for the given grammar:

$S \rightarrow A A$

$A \rightarrow a A \mid b$

Solution for example-18:

Augmented grammar for the given CFG:

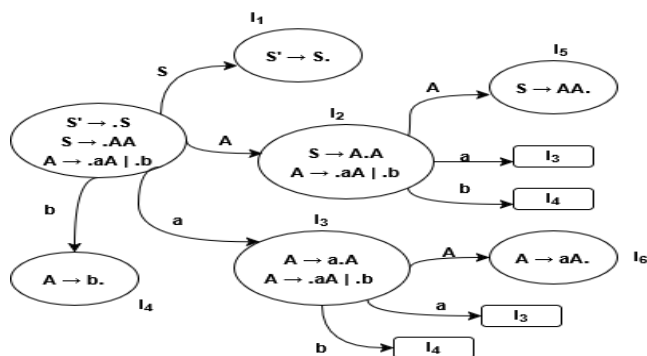
$S' \rightarrow S$ (also known as augmented production rule)

$S \rightarrow A A$

$A \rightarrow a A \mid b$

Now, canonical view:

Decoding Compilers



Numbering the production rules:

1. $S' \rightarrow S$
2. $S \rightarrow AA$
3. $A \rightarrow aA$
4. $A \rightarrow b$

LR(0) Parsing Table:

	Action			Goto	
	a	b	\$	S	A
I ₀	S ₃	S ₄		1	2
I ₁			Accept		
I ₂	S ₃	S ₄			5
I ₃	S ₃	S ₄			6
I ₄	R ₃	R ₃	R ₃		
I ₅	R ₁	R ₁	R ₁		
I ₆	R ₂	R ₂	R ₂		

Example-19:

For the given grammar find out if it is LR(0) or not:

$E \rightarrow T + E \mid T$

$T \rightarrow id$

Solution for example-19:

Augmented grammar for the given CFG:

$E' \rightarrow E$

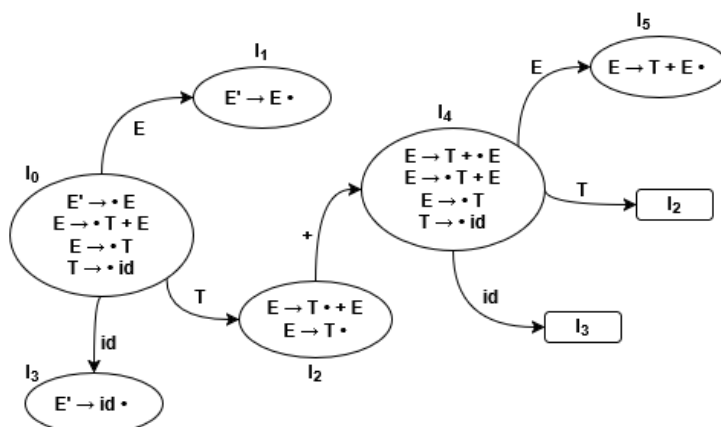
$E \rightarrow T + E$

$E \rightarrow T$

$T \rightarrow id$

Now, canonical view:

Decoding Compilers



Numbering the production rules:

1. $E' \rightarrow E$
2. $E \rightarrow T + E$
3. $E \rightarrow T$
4. $T \rightarrow id$

Action and Goto table:

	Action			Goto	
	id	+	\$	E	T
I ₀	S ₃			1	2
I ₁			Accept		
I ₂	R ₂	S ₄ / R ₂	R ₂	5	2
I ₃	R ₃	R ₃	R ₃		
I ₄	S ₃			5	2
I ₅	R ₁	R ₁	R ₁		

In the cell, $M[I_2, +]$ we can see that there is both a shift and a reduce action appear. This indicates that the given CFG is not LR(0).

Parser Generators

Imagine you are a chef trying to cook a very complex dish. You could follow every step manually, carefully checking each ingredient and timing yourself perfectly. But what if there was a kitchen assistant who already knew the recipe and could execute the steps correctly for you? That would save time and reduce mistakes.

In compiler design, parser generators act like that kitchen assistant. Instead of manually writing hundreds of lines of code for a parser, you provide a high-level description of your grammar, and the parser generator automatically produces the parser code for you.

Decoding Compilers

Two of the most popular parser generators are Yacc and Bison.

Yacc (Yet Another Compiler-Compiler)

Yacc is a tool that generates a parser from a grammar written in BNF (Backus-Naur Form). It is mainly used for C programming language.

Workflow of Yacc:

1. You write the grammar rules in a .y file.
2. You define actions (like building syntax trees or evaluating expressions) in C code.
3. Yacc generates a .c file containing the parser.

Features of Yacc:

1. Supports LALR(1) parsing, which can handle a wide range of grammars.
2. Automatically resolves shift/reduce and reduce/reduce conflicts when possible.

Bison

Bison is an enhanced version of Yacc, compatible with GNU systems. It generates C or C++ parsers.

Bison has additional features compared to Yacc:

1. Better error reporting
2. Supports pure parsers (reentrant code, safe for multi-threading)
3. Compatible with modern development tools

Working principle of Yacc/Bison

Grammar Specification: You write rules for your language. Example:

```
expr : expr '+' term { /*  
    action */ }  
      | term           { /*  
    action */ };  
term : 'id';
```

Parser Generation: Yacc/Bison converts this into an LALR(1) parser.

Integration with Lexer: Usually paired with Lex/Flex, which tokenizes the input.

Execution: The generated parser reads tokens and applies the grammar rules to analyze syntax.

Chapter 4 Exercises

Part A: Short Questions

1. What is syntax analysis?
2. What is a parser?
3. What is the role of syntax analysis in a compiler?
4. What is a context-free grammar (CFG)?
5. What is a parse tree?
6. What is an abstract syntax tree (AST)?
7. What is ambiguity in grammar?
8. What is left recursion?
9. What is left factoring?
10. What is recursive descent parsing?
11. What is backtracking in parsing?
12. What is predictive parsing?
13. What is bottom-up parsing?
14. What is shift-reduce parsing?
15. What is handle in shift-reduce parsing?
16. What is operator precedence parsing?
17. What is LR parsing?
18. What does LR(0) mean?

Decoding Compilers

19. What is an LR(0) item?
20. What is closure in LR parsing?
21. What is GOTO function in LR parsing?
22. What is the difference between ACTION and GOTO table?
23. What is shift-reduce conflict?
24. What is reduce-reduce conflict?
25. What is a parser generator?
26. What are Yacc and Bison?

Part B: Explanation Questions

1. Explain the role of syntax analysis.
2. Explain recursive descent parsing with an example.
3. Write the algorithm for predictive parsing.
4. Explain shift-reduce parsing with a stack example.
5. Explain operator precedence parsing.
6. Construct an operator precedence table for a given grammar.
7. Explain LR(0) parsing.
8. Write the steps to construct an LR(0) parsing table.
9. Explain closure and GOTO with an example.
10. Explain parser generators.
11. Explain the working process of Yacc/Bison.
12. Write the difference between top-down and bottom-up parsing.

Part C: Challenge Questions

1. Why is left recursion a problem for recursive descent parsing but not for LR parsing?

Decoding Compilers

2. Why is LR parsing more powerful than LL parsing?
3. Difference between LR(0) and LL(1).
4. Why can't all grammars be parsed using LR(0)?
5. What type of grammars can be parsed using operator precedence parsing?
6. Design a simple grammar and show all LR(0) items and parsing table.
7. Write a small Yacc/Bison grammar for arithmetic expressions.
8. Explain why left factoring is necessary for predictive parsing.

Part D: Practice Questions

1. Find the LMD, RMD, and Parser tree for the given grammars:

SI	Grammar	Input String
a	$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{while } E \text{ do } S \mid \text{begin } S \text{ end} \mid \text{id} := E \mid \epsilon$ $E \rightarrow \text{id} \mid \text{num} \mid E + E \mid E < E$	if id < num then id := id + num else while id < num do begin id := num end
b	$E \rightarrow E + T \mid T$ $T \rightarrow \text{id}$	id + id + id
c	$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$ $\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$ $\text{Factor} \rightarrow (\text{Expr}) \mid \text{id}$	id * id + id
d	$S \rightarrow A B$ $A \rightarrow a A \mid a$ $B \rightarrow b B \mid b$	aaabbb
e	$E \rightarrow E + E \mid E * E \mid \text{id}$	id + id * id
f	$S \rightarrow A 1 B$ $A \rightarrow 0 A \mid \epsilon$ $B \rightarrow 0 B \mid 1 B \mid \epsilon$	00101
g	$S \rightarrow b B \mid a A$ $A \rightarrow b \mid b S \mid a A A$ $B \rightarrow a \mid a S \mid b B B$	bbaababa

2. Determine whether the grammar is ambiguous:

SI	Grammar	Input String
a	$E \rightarrow E + E \mid E * E \mid \text{id}$	id + id * id

Decoding Compilers

b	$S \rightarrow (S) \mid SS \mid a$	$(a)(a)(a)$
c	$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{other}$	if E1 then if E2 then S1 else S2
d	$S \rightarrow SS \mid a \mid b$	a b
e	$E \rightarrow -E \mid E + E \mid \text{id}$	- id + id
f	$S \rightarrow SS \mid (S) \mid \varepsilon$	$()()$
g	$S \rightarrow A \mid B$ $A \rightarrow aA \mid \varepsilon$ $B \rightarrow bB \mid \varepsilon$	ε
h	$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid \text{while } E \text{ do } S \mid \text{id}$	if E then if E then id else id
i	$S \rightarrow SS \mid a \mid b$	a a b
j	$S \rightarrow bB \mid aA$ $A \rightarrow b \mid bS \mid aAA$ $B \rightarrow a \mid aS \mid bBB$	bbaababa

3. Perform left factoring on the following grammars:

SI	Grammar
a	$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \text{ else if } E \text{ then } S$
b	$\text{Stmt} \rightarrow \text{while } E \text{ do Stmt} \mid \text{while } E \text{ do Stmt else Stmt}$
c	$S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S$
c	$X \rightarrow abc d \mid abc e \mid ab f$
d	$S \rightarrow \text{while } E \text{ do } S \mid \text{while } E \text{ do } S \text{ else } S$

4. Remove left recursion from the following grammars:

SI	Grammar
a	$A \rightarrow Ax \mid Ay \mid z$
b	$S \rightarrow SS \mid Sa \mid b$
c	$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$ $\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$ $\text{Factor} \rightarrow (\text{Expr}) \mid \text{id}$
d	$\text{Stmt} \rightarrow \text{Stmt} ; \text{Stmt} \mid \text{id} := \text{Expr}$
e	$X \rightarrow XY \mid Y$ $Y \rightarrow YZ \mid Z$ $Z \rightarrow \text{id}$
f	$\text{Expr} \rightarrow \text{Expr} + \text{Expr} \mid \text{Expr} - \text{Expr} \mid \text{Expr} * \text{Expr} \mid (\text{Expr}) \mid \text{id}$

Decoding Compilers

g	$\text{Expr} \rightarrow \text{Expr} + \text{Expr} \mid \text{Expr} - \text{Expr} \mid \text{Expr} * \text{Expr}$ $\mid (\text{Expr}) \mid \text{id}$ $\text{Stmt} \rightarrow \text{if Expr then Stmt else Stmt} \mid \text{if Expr then Stmt} \mid \text{while Expr do Stmt} \mid \text{id} := \text{Expr}$ $\text{Block} \rightarrow \text{begin StmtList end}$ $\text{StmtList} \rightarrow \text{StmtList} ; \text{Stmt} \mid \text{Stmt}$
---	---

5. Find FIRST and FOLLOW sets for the following grammars:

SI	Grammar
a	$S \rightarrow A B$ $A \rightarrow a A \mid \varepsilon$ $B \rightarrow b B \mid \varepsilon$
b	$E \rightarrow T E'$ $E' \rightarrow + T E' \mid \varepsilon$ $T \rightarrow F T'$ $T' \rightarrow * F T' \mid \varepsilon$ $F \rightarrow (E) \mid \text{id}$
c	$\text{Expr} \rightarrow \text{Term Expr}'$ $\text{Expr}' \rightarrow + \text{Term Expr}' \mid - \text{Term Expr}' \mid \varepsilon$ $\text{Term} \rightarrow \text{Factor Term}'$ $\text{Term}' \rightarrow * \text{Factor Term}' \mid \varepsilon$ $\text{Factor} \rightarrow (\text{Expr}) \mid \text{id}$
d	$X \rightarrow Y Z$ $Y \rightarrow y Y \mid \varepsilon$ $Z \rightarrow z Z \mid \varepsilon$

6. Find if the grammar is LL(1) or not:

SI	Grammar
a	$S \rightarrow i E t S S'$ $S' \rightarrow e S \mid \varepsilon$ $E \rightarrow b$
b	$S \rightarrow A a \mid b$ $A \rightarrow A c \mid \varepsilon$
c	$X \rightarrow X Y \mid Y$ $Y \rightarrow Z \mid \varepsilon$ $Z \rightarrow \text{id}$
d	$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \text{other}$ $E \rightarrow \text{id}$
e	$\text{StmtList} \rightarrow \text{StmtList} ; \text{Stmt} \mid \text{Stmt}$

	$\text{Stmt} \rightarrow \text{id} := \text{Expr} \mid \text{while Expr do Stmt} \mid \text{if Expr then Stmt else Stmt}$ $\text{Expr} \rightarrow \text{id} \mid \text{num}$
--	--

7. Find if the grammar is LR(0) or not:

SI	Grammar
a	$S \rightarrow (S) \mid \varepsilon$
b	$S \rightarrow \text{if } E \text{ then } S \text{ else } S \mid \text{if } E \text{ then } S \mid \text{other}$ $E \rightarrow \text{id}$
c	$\text{StmtList} \rightarrow \text{StmtList} ; \text{Stmt} \mid \text{Stmt}$ $\text{Stmt} \rightarrow \text{id} := \text{Expr}$ $\text{Expr} \rightarrow \text{id}$
d	$S \rightarrow S S \mid a$
e	$S \rightarrow \text{begin StmtList end}$ $\text{StmtList} \rightarrow \text{StmtList} ; \text{Stmt} \mid \text{Stmt}$ $\text{Stmt} \rightarrow \text{id} := \text{Expr} \mid \text{if Expr then Stmt else Stmt} \mid \text{while Expr do Stmt}$ $\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$ $\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$ $\text{Factor} \rightarrow \text{id} \mid (\text{Expr})$

Chapter 5

INTRODUCTION TO SEMANTIC ANALYSIS

Introduction

Imagine you write a sentence:

"The sky is eating food."

All the words are correct. The grammar is also correct. But the meaning is wrong. The sky cannot eat food.

In the same way, a program can be correct in structure but wrong in meaning.

Semantic analysis is the stage of the compiler that checks the meaning of the program. It comes after syntax analysis. It makes sure that the program makes sense.

It checks things like:

1. Are variables used correctly?
2. Are types correct?
3. Are names defined before use?

So, semantic analysis makes sure the program is meaningful and correct.

Semantic Checking

Semantic checking is the process of checking the meaning of the program. The compiler looks at the program and checks if everything is logically correct.

Example:

```
int x = "hello";
```

This is wrong because we are putting a string into an integer variable.

Decoding Compilers

So, semantic checking finds this kind of mistake. It ensures that all parts of the program are used properly. It helps the compiler understand whether the program makes sense or not.

Symbol Table

Think of a notebook where you write important information.

The symbol table is like that notebook for the compiler. It stores details about variables, functions, and names used in the program.

Example, in:

```
int age = 20;
```

The symbol table stores:

Name	age
Type	int
Value	20

Whenever the compiler needs information, it checks the symbol table.

Type Checking

Type checking ensures that the correct type of data is used.

Example:

int	x = 10;	correct
int	x = "hello";	wrong

In programming, each value has a type, like integer, float, or string. Type checking makes sure that operations are done between correct types.

Example:

Adding two numbers is correct, but adding a number and a string is not correct. So, type checking helps avoid wrong operations.

Scope and Binding

Imagine a classroom where each student has a seat. A student can only sit in their assigned seat.

Decoding Compilers

In programming, scope defines where a variable can be used.

If a variable is declared inside a block, it can only be used inside that block. Binding means connecting a variable name with its value or memory location.

Example:

```
int x = 5;
```

Here,

1. x is the name
2. 5 is the value
3. Binding connects x with 5

So,

1. Scope = where we can use the variable
2. Binding = what the variable is connected to

Semantic Errors

Semantic errors are mistakes in meaning. The program may look correct, but it does not make sense.

Examples:

Using an undeclared variable	-	x = 10; (x not defined)
Type mismatch	int	x = "hello";
Wrong operation	-	adding string with number

These errors are found during semantic analysis. The compiler shows error messages so the programmer can fix them.

Chapter 5 Exercises

Part A: Define / Explain

1. Define semantic analysis.
2. What is semantic checking?
3. Explain symbol table.
4. What is type checking?
5. Define scope in programming.
6. What is binding?
7. What are semantic errors?

Part B: True / False

1. Semantic analysis checks the structure of a program.
2. Symbol table stores information about variables.
3. Type checking ensures correct data types are used.
4. A variable can be used outside its scope.
5. Semantic errors are related to meaning.
6. Binding connects a variable with its value.

Part C: Short Answer Questions

1. What is the role of semantic analysis in a compiler?
2. Give one example of a semantic error.
3. What information is stored in a symbol table?
4. Why is type checking important?
5. What is the difference between scope and binding?
6. What happens if a variable is used without declaration?

Part D: Comparison / Analytical Questions

1. Compare syntax analysis and semantic analysis.
2. Compare scope and binding.
3. Difference between type checking and semantic checking.
4. Explain how symbol table helps in semantic analysis.
5. Why are semantic errors important to detect?

Part E: Scenario / Story-Based Questions:

1. A student writes:
 - a. `int x = "hello";`
 - b. What is the problem? Explain.
2. A programmer uses a variable without declaring it. What error will occur? Why?
3. A variable is declared inside a block but used outside. What will happen? Explain.
4. A program adds a number with a string. Why is this wrong?

Part F: Challenge Questions

1. Find all semantic errors:
 - a. `int x;`
`x = y + 10;`
2. Identify the mistakes:
 - a. `int a = 5;`
`float a = 3.2;`
3. Correct the errors:
 - a. `int x = 5.7;`

Decoding Compilers

4. Write a small example showing scope and binding.

Chapter 6

INTRODUCTION TO ICG

Introduction

In the previous chapters, we learned how a compiler checks a program:

1. Lexical Analysis → checks words
2. Syntax Analysis → checks grammar
3. Semantic Analysis → checks meaning

So, after Semantic Analysis, the compiler now understands the program correctly. But there is a big problem. The source program is written in a high-level language like C, Java, or Python.

Computers do not understand high-level languages. Computers only understand machine language (0 and 1). So, the compiler must convert the program into machine language. But converting high-level language directly into machine code is very difficult, complex, and machine dependent.

So, instead of converting directly, the compiler first converts the program into a middle-level code, called, Intermediate Code (IR) and this phase is called Intermediate Code Generation (ICG).

So, in this chapter, we will learn:

1. What is Intermediate Representation (IR)
2. Types of Intermediate Code
3. Three Address Code (TAC)
4. Quadruples, Triples
5. Syntax Tree and DAG
6. Boolean Expressions
7. Control Flow Statements

Intermediate Representation (IR)

Imagine you wrote this simple program: $x = a + b * c;$. Now the compiler has already checked the spelling (Lexical Analysis), grammar (Syntax Analysis), and

Decoding Compilers

the meaning (Semantic Analysis). So, the compiler understands the program correctly.

But now comes the hardest part. How to convert this into machine code? The compiler faces 2 problems:

1. High-level code is too complex
2. Machine code is too low-level

So, jumping directly from high-level → machine code is very difficult. So, the compiler uses a smart trick. Instead of direct translation, it does this:

High-Level Code → Intermediate Code → Machine

This middle code is called Intermediate Representation (IR).

Intermediate Representation is a machine-independent representation of a program generated during compilation, used as an intermediate step between source and target code.

Why IR is Needed

There are many programming languages (C, Java, Python) and many machines (Intel, ARM, MIPS) exists in the world. If we do NOT use Intermediate Code, then we must build a separate compiler for each language–machine pair. For example, if we now create a new language Knight, we need to make Knight Intel Compiler, Knight ARM Compiler, Knight MIPS Compiler. This is very difficult, costly, and time consuming.

So, we need a solution for this problem. Now we introduce Intermediate Representation (IR). Instead of making many compilers, we divide the compiler into two parts:

Source Code → Front End → IR → Back End →
Machine Code

Here, front end is language dependent, backend is machine dependent, IR is machine dependent. As a result, with a single version of compiler that we could run with different machine version.

Advantages of IR

Machine Independence: Intermediate code is machine independent, which means it does not depend on any specific machine (Intel, ARM, MIPS). The same IR can be used to generate code for different machines. So, we can say IR makes the compiler machine independent. This is one of the biggest advantages.

Reduces Compiler Complexity: Without IR we need a separate compiler for each language and each machine. With IR we divide compiler into Front End and Back End. As a result the compiler becomes easier to design, easier to test and easier to maintain.

Makes Compiler Portable: Portable means the compiler can run on different machines. Since IR is machine independent, only back end needs to change for new machine and front end stays same. So, compiler becomes portable.

Makes Retargeting Easy: Retargeting means modify compiler to generate code for a new machine. If IR is used, we just write a new back end, no need to rewrite full compiler. So, IR makes retargeting easy.

Supports Multiple Source Languages: Using IR, we can support C, C++, Java, Python etc. All of them can be converted into same IR, then IR $\rightarrow$ machine code. So, IR helps build multi-language compilers.

Improves Program Analysis: Many program analysis techniques work on IR such as data flow analysis, control flow analysis, and dependency analysis. These are used for optimization, error detection, and performance improvement. So, IR helps in program analysis.

High-level vs Low-level IR

High-Level IR: Intermediate representation that is close to the source program and machine-independent is called high-level IR. Characteristics of high-level IR is given by:

1. Resembles high-level language constructs (variables, expressions)
2. Independent of machine architecture
3. Easy to generate from syntax tree
4. Suitable for high-level optimizations (constant folding, algebraic simplification, loop transformations)

Example of high-level IR:

Decoding Compilers

```
t1 = c * d
t2 = b + t1
a = t2
```

Low-Level IR: Intermediate representation that is closer to machine code but still abstract is called low-level IR. Characteristics of low-level IR is given by:

1. Uses registers, memory locations, low-level instructions
2. Easier to map to target machine code
3. Suitable for low-level optimizations (register allocation, instruction scheduling)
4. Abstracted enough to remain portable between machines

Example:

```
LOAD R1, c
MUL R1, d
LOAD R2, b
ADD R2, R1
STORE a, R2
```

Types of IR

IR can be classified into three main types:

1. Linear IR
2. Tree IR
3. Graph IR

Linear IR (TAC)

Linear IR is a sequence of instructions, each representing a simple operation. Also called Three Address Code (TAC). Characteristics:

1. Instructions executed sequentially.
2. Each instruction typically has at most three addresses/operands.
3. Easy to generate and manipulate.
4. Commonly used for optimization and code generation.

Example of TAC:

<pre>t1 = b * c t2 = a + t1 d = t2</pre>
--

Decoding Compilers

Liner IR most widely used in modern compilers. It also supports both analysis and optimizations.

There are 3 types of TAC representation:

1. Simple TAC (direct)
2. Quadruples
3. Triples

Simple TAC

TAC written in a linear textual form where each instruction has at most three addresses. It uses temporary variables (t_1 , t_2) for intermediate results. It is easy to read and generate. It is also good for small compilers or basic examples. For example ($a = b + c * d$):

$t_1 = c * d$

$t_2 = b + t_1$

$a = t_2$

Here,

1. t_1 stores $c * d$.
2. t_2 stores $b + t_1$.
3. a stores final result

Also each instruction has ≤ 3 addresses $\rightarrow$ simple linear IR.

Quadruples

Each TAC instruction is stored as a quadruple (4 fields):

Field	Meaning
Operator	Operation (+, -, =)
Argument 1	First operand
Argument 2	Second operand
Result	Variable to store result

For example, we can convert the above TAC into quadruples:

Operator	Arg1	Arg2	Result
*	c	d	t_1
+	b	t_1	t_2
=	t_2		a

Triples

Each TAC instruction is stored as a triple (3 fields):

Field	Meaning
Operator	Operation (+, -, =)
Argument 1	First operand
Argument 2	Second operand

Key difference from quadruples is result field is implicit (the position (index) of instruction is the reference). It also reduces storage space.

Converting the above TAC into triples:

Index	Operator	Arg1	Arg2
(0)	*	c	d
(1)	+	b	(0)
(2)	=	(1)	a

Here,

1. (0) refers to result of instruction 0 (t_1)
2. (1) refers to result of instruction 1 (t_2)

Indirect Triples

Indirect Triples are an enhancement of Triples, used to make code modification and optimization easier. In Triples, the result of an instruction is implicitly referenced by its index. But the problem is, if we insert new instructions, all subsequent indexes must be updated. In order to solve this problem Indirect Triples use a pointer table (array of pointers to triples) so instruction numbers don't change when new instructions are inserted. For example, we can convert the above TAC into quadruples:

PTR	Index	Operator	Arg1	Arg2
35	(0)	*	c	d
36	(1)	+	b	(0)
37	(2)	=	(1)	a

Example-1:

Find the TAC, Quadruples, Triples and Indirect Triples for the expression: $a + (b * c + b * c + d)$

Decoding Compilers

Solution for example-1:

Finding the TAC for the expression:

$t1 = b * c$

$t2 = t1 + t1$

$t3 = t2 + d$

$t4 = a + t3$

Finding the Quadruples from TAC:

Operator	Arg1	Arg2	Result
*	b	c	t1
+	t1	t1	t2
+	t2	d	t3
+	a	t3	t4

Finding the Triples from the TAC:

Index	Operator	Arg1	Arg2
(0)	*	b	c
(1)	+	(0)	(0)
(2)	+	(1)	d
(3)	+	a	(2)

Finding the Indirect Triples form the TAC:

PTR	Index	Operator	Arg1	Arg2
56	(0)	*	b	c
57	(1)	+	(0)	(0)
58	(2)	+	(1)	d
59	(3)	+	a	(2)

Example-2:

Find the TAC, Quadruples, Triples and Indirect Triples for the expression: $a = b * -c + b * -c$

Solution for example-2:

Finding the TAC for the expression:

$t1 = \text{uminus } c$

$t2 = b * t1$

$t3 = \text{uminus } c$

$t4 = b * t3$

$t5 = t2 + t4$

$t6 = t5$

Decoding Compilers

Finding Quadruples from the TAC:

Operator	Arg1	Arg2	Result
uminus	c		t1
*	b	t1	t2
uminus	c		t3
*	b	t3	t4
+	t2	t4	t5
=	t5		t6

Finding Triples from the TAC

Index	Operator	Arg1	Arg2
(0)	uminus	c	
(1)	*	b	(0)
(2)	uminus	c	
(3)	*	b	(2)
(4)	+	(1)	(3)
(5)	=	a	(4)

Finding the Indirect Triples from the TAC:

PTR	Index	Operator	Arg1	Arg2
34	(0)	uminus	c	
35	(1)	*	b	(0)
36	(2)	uminus	c	
37	(3)	*	b	(2)
38	(4)	+	(1)	(3)
39	(5)	=	a	(4)

Example-3:

Find the TAC the expression: $a + b * c / e \uparrow f + b * a$

Solution for example-3:

t1 = $e \uparrow f$
t2 = $b * c$
t3 = $t2 / t1$
t4 = $b * a$
t5 = $a + t3$
t6 = $t5 + t4$

Boolean Expressions in TAC

Boolean expressions are expressions that produce true or false. For example:

1. $a < b$
2. $a > b$
3. $a == b$
4. $a < b \text{ AND } c < d$
5. $a < b \text{ OR } c < d$

Two Ways to Represent Boolean Expressions in TAC:

1. Boolean Value Code: Expression returns 0 or 1
2. Jumping Code: Expression uses jumps to labels

Example of Boolean Value Code for expression $t = (a < b)$:

TAC can be written as:

1. if $a < b$ goto L1
2. $t = 0$
3. goto L2
4. L1: $t = 1$
5. L2:

Example of Jumping Code for expression (if $a < b$ then $x = 1$):

1. if $a < b$ goto L1
2. goto L2
3. L1: $x = 1$
4. L2:

Short-Circuit Evaluation (AND, OR)

Short-circuit means do not evaluate the second expression if the result is already known.

Rule for TAC pattern for AND (expression: $A \text{ AND } B$):

```
if A goto L1
goto FALSE
L1: if B goto TRUE
goto FALSE
TRUE: ...
FALSE: ...
```

TAC Pattern for OR (expression: $A \text{ OR } B$):

Decoding Compilers

```
if A goto TRUE
if B goto TRUE
goto FALSE
TRUE: ...
FALSE: ...
```

Note that, Boolean expressions are represented using control flow (jumps) instead of computing true/false values. This is called Flow-of-Control method.

Backpatching

Backpatching is a technique used to fill in the missing jump addresses in intermediate code after the target label is known.

Simple meaning is we generate jump instructions first, and fill the label addresses later. Because when the compiler is generating TAC, sometimes it does not yet know where to jump. So, it leaves the target blank, and later patches (fills) it.

Backpatching Uses 3 Lists:

List	Meaning
truelist	List of jumps when condition is TRUE
falselist	List of jumps when condition is FALSE
nextlist	List of jumps to next statement

Backpatching Functions:

Function	Meaning
makelist(i)	Create a new list with instruction i
merge(p1, p2)	Combine two lists
backpatch(p, i)	Fill all jumps in list p with target i

Jumping Code vs Boolean Value Code

Feature	Jumping Code	Boolean Value Code
Output	Jumps to label	Produces 0 or 1
Used for	if, while, control flow	Assignments
Efficient	More efficient	Less efficient
Example	if a<b goto L1	t = (a<b)

Example-3:

Decoding Compilers

Find the TAC, Quadruples, Triples, Indirect Triples for the statements:

if $A < B$

 then 1

 else 0

Solution for example-3:

Finding the TAC:

(1) if $A < B$ goto L1

(2) $t1 = 0$

(3) goto L2

(4) L1: $t1 = 1$

(5) L2:

Creating quadruples from TAC:

Operator	Arg1	Arg2	Result
if <	A	B	L1
=	0	–	t1
goto	–	–	L2
=	1	–	t1
label	–	–	L2

Triples:

Index	Operator	Arg1	Arg2
(0)	if <	A	B
(1)	=	0	–
(2)	goto	–	–
(3)	=	1	–
(4)	label	–	–

Indirect Triples:

PTR	Index	Operator	Arg1	Arg2
45	(0)	if <	A	B
46	(1)	=	0	–
47	(2)	goto	–	–
48	(3)	=	1	–
49	(4)	label	–	–

Example-3:

Find the TAC for the statements:

“if $a < b$ and $c < d$ then $t = 1$ else $t = 0$ ”

Solution for the example-3:

- (1) if (a<5) goto (3)
- (2) goto (4)
- (3) if (c<d) goto (6)
- (4) t=0
- (5) goto (7)
- (6) t = 1

Example-4:

Find the TAC for the statements:

“if a < b OR c < d then t = 1 else t = 0”

Solution for example-4:

TAC:

- (1) if (a < b) goto (5)
- (2) if (c < d) goto (5)
- (3) t = 0
- (4) goto (6)
- (5) t = 1
- (6)

Example-5:

Find the TAC for the statements:

“while (a < b) a = a + 1;”

Solution for example-5:

- (1) TAC:
- (2) if (a < b) goto (3)
- (3) goto (5)
- (4) a = a + 1
- (5) goto (1)
- (6)

Tree IR (Syntax Tree)

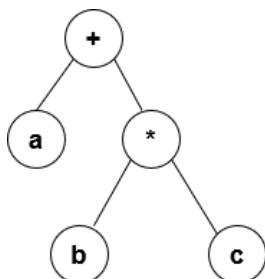
Tree IR is an intermediate representation where the program is represented in the form of a tree structure. It also called as Syntax Tree / Abstract Syntax Tree (AST) in many cases. Characteristics:

1. Nodes represent operators
2. Leaves represent operands (variables, constants)

Decoding Compilers

3. Shows hierarchy of expressions
4. Easy to analyze and manipulate for optimizations

Example: AST for expression “a + b * c” given by:



It is useful for expression evaluation and algebraic optimizations. It is also Intermediate step before generating linear IR.

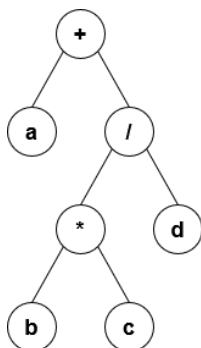
Example-6:

Draw the AST for the expression:

“a + b * c / d”

Solution for example-6:

AST for the given expression (considering operator precedence) :



Key Properties of Tree IR

Feature	Description
Structure	Hierarchical
Root	Last operation
Leaves	Operands
Internal nodes	Operators
Machine independent	Yes

Importance of Tree IR

Tree IR helps compiler to:

1. Detect common subexpressions
2. Perform constant folding
3. Perform algebraic simplification
4. Convert to DAG
5. Generate Three Address Code

Tree IR vs Linear IR

Feature	Tree IR	Linear IR (TAC)
Structure	Tree	Sequence
Representation	Hierarchical	Instructions
Easy Optimization	Yes	Medium
Easy Code Generation	Medium	Yes
Example	Syntax Tree	Three Address Code

Prase Tree vs Syntax Tree

Parse Tree	Syntax Tree
Shows complete derivation of a string from the grammar.	Shows only the essential structure of the program.
Also called Concrete Syntax Tree.	Also called Abstract Syntax Tree (AST).
Includes all terminals and non-terminals.	Omits unnecessary terminals; only important nodes.
Used to verify grammar correctness.	Used for semantic analysis, optimization, and code generation.
Usually large and detailed.	Compact and smaller.
Internal nodes represent all non-terminals.	Internal nodes represent operators or constructs only.

Leaf nodes represent terminals (identifiers, keywords, literals).	Leaf nodes represent operands or variables.
---	---

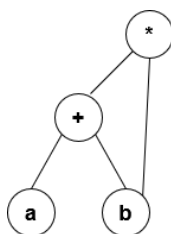
Graph IR (DAG)

Graph IR is an Intermediate Representation where the program is represented as a graph instead of a linear code. The most common Graph IR is Directed Acyclic Graph (DAG). It is mainly used to represent expressions and helps in optimization.

Characteristics:

1. Node represents an operation or value
2. Edges represent dependencies
3. Combines identical expressions to avoid repeated calculation

Example of DAG $(a + b * b)$:



Used for optimization, especially common subexpression elimination. It also minimizes duplicate computation.

Properties of DAG

Directed Graph: All edges have a direction, pointing from operands/subexpressions to the operator. For example: $a \rightarrow * \rightarrow +$.

Acyclic: There are no cycles in the graph. We cannot go back from a parent node to a child node.

Nodes Represent Operators or Operands: Leaf nodes $\rightarrow$ operands (variables, constants). Internal nodes $\rightarrow$ operators (+, *, /, -, etc.)

Common Subexpression Elimination: If the same operation with the same operands occurs more than once, it is represented by a single node. It helps to optimize code by avoiding repeated calculations.

Edges Represent Computation Dependency: An edge from node X to node Y means Y depends on X for its value.

Single Root Node: The root node represents the final result of the expression.

Reusability of Nodes: Nodes of repeated subexpressions are reused to reduce computation and memory usage.

Post-order Traversal Generates TAC: By traversing the DAG by post-order, we can generate optimized Three Address Code (TAC).

Machine-Independent Representation: A DAG shows calculations in a general way, without depending on a specific computer.

Optimizable Structure: DAG makes algebraic simplifications, constant folding, and common subexpression elimination easy.

Importance of DAG

Graph IR (DAG) is used for:

1. Code optimization
2. Common subexpression elimination
3. Dead code elimination
4. Efficient code generation

Common Subexpression Elimination using DAG

A common subexpression is an expression that appears more than once and has the same operands and operator. For example:

$a = b * c + d$ $e = b * c + f$

Here, $b * c$ appears twice → this is a common subexpression. Instead of calculating $b * c$ two times, the compiler calculates it once and reuses the result. This optimization is called Common Subexpression Elimination (CSE).

Steps for Common Subexpression Elimination Using DAG:

1. Create leaf nodes for operands
2. Create node for operator
3. Before creating new node, check if same node already exists

Decoding Compilers

4. If exists $\rightarrow$ reuse node
5. If not $\rightarrow$ create new node
6. Generate TAC from DAG

Syntax Tree vs DAG

Syntax Tree	DAG
Duplicate expressions allowed	Duplicate removed
More nodes	Less nodes
No optimization	Optimization possible
Tree structure	Graph structure

Construction of DAG

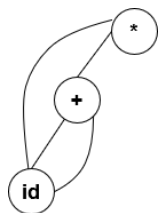
Rule 1: In a DAG, Leaf nodes represent identities, names or constants. Identifier nodes represent operators.

Rule 2: While Constructing a DAG, there is a check made to find if there is an existing node with the same children. A new node is created only when such a node does not exist. This action allows to detect common sub-expression and eliminate the re-computation of same.

Rule 3: The assignment of the form $x:=y$ must not be performed until and unless it is a must.

Example-7:

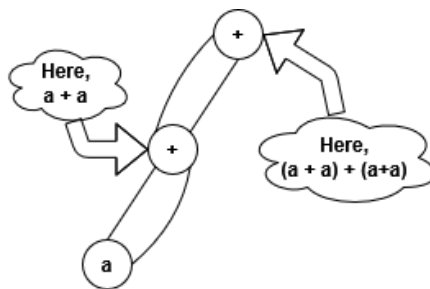
DAG for the expression “ $\text{id} + \text{id} * \text{id}$ ”



Example-8:

DAG for the expression: “ $a + a + a + a + a + a$ ”

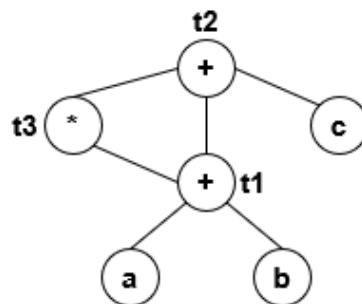
Decoding Compilers



Example-9:

Draw the DAG for the expression: “ $(a + b) * (a + b + c)$ ”

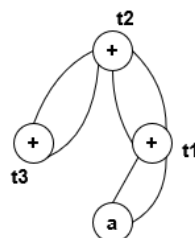
TAC:
 $t1 = a + b$
 $t2 = t1 + c$
 $t3 = t1 * t2$



Example-10:

Draw the DAG for the expression: “ $(((a + a) + (a + a)) + ((a + a) + (a + a)))$ ”

TAC
 $t1 = a + a$
 $t2 = t1 + t1$
 $t3 = t2 + t2$

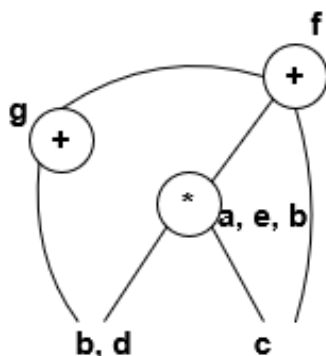


Example-11:

Draw the DAG for the block:

Decoding Compilers

$a = b * c$
 $d = b$
 $e = d * c$
 $b = e$
 $f = b + c$
 $g = f + d$



Example-12:

Draw the DAG for the block:

$S1 = 4 * I$

$S2 = \text{addr}(A) - 4$

$S3 = S3[S1]$

$S4 = 4 * I$

$S5 = \text{addr}(B) - 4$

$S6 = S5[S4]$

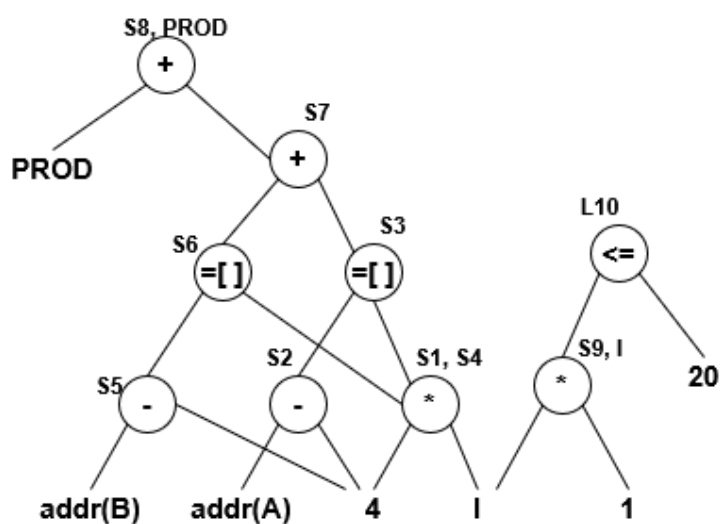
$S7 = S3 * S6$

$S8 = \text{PROD} + S7$

$\text{PROD} = S8$

$S9 = I + 1$

If $I \leq 20$ goto L10



Chapter 6 Exercises

Part A: Theory Questions

1. What is Intermediate Representation (IR)? Why is it used in a compiler?
2. What is Three Address Code (TAC)? Write its general form.
3. Explain the difference between:
4. Quadruples
5. Triples
6. Indirect Triples
7. What are the advantages of TAC?
8. What is a Syntax Tree?
9. What is a DAG? Why is DAG better than Syntax Tree?
10. What is a Basic Block?
11. What is Flow Graph?
12. What is Boolean Expression in TAC?
13. What is Short-Circuit Evaluation?
14. What is Backpatching? Why is it needed?
15. What is the difference between Jumping Code and Boolean Code

Part B: Practice Questions

1. Generate TAC, Quadruples, Triples, and Indirect Triples for the given expressions:

SI	Expression
a	$a = b + c * d$
b	$a = (b + c) * (d + e)$
c	$a = b * -c + b * -c$
d	$a + b * c / e \uparrow f + b * a$
e	$x = (a + b) * (c - d) / (e + f)$

Decoding Compilers

f	$E = (a + b) * (c - d) + (e * f) / (g + h)$
---	---

2. Generate TAC for the given expressions:

SI	Expression / Blocks
a	$X = (a + b) * (c - d) / (e + f)$
b	<pre> if (a > b) then x = a - b; else x = b - a; </pre>
c	<pre> while (i < n) { sum = sum + i; i = i + 1; } </pre>
d	<pre> if (x == y) { z = x + y; } else if (x > y) { z = x - y; } else { z = y - x; } </pre>
e	<pre> if ((x > 0) && (y < 10)) { z = x + y; } </pre>
f	<pre> for (i = 0; i < n; i = i + 1) { if (i % 2 == 0) evenSum = evenSum + i; } </pre>
g	<pre> if ((a + b) > (c * d)) { max = a + b; } else { max = c * d; } </pre>
h	$P = -(a + b) * (c - d)$
i	$Y = (a < b) \wedge (c \neq d) \vee (e \geq f)$

3. Draw Syntax Tree and DAG:

SI	Expression / Block
a	$(x * y) + (x * y) + (z - w)$
b	$(p \wedge q) \vee (p \wedge q) \vee r$
c	$(a + b) * (a + b) - (c / d)$

Decoding Compilers

d	$(a + b) + (c + d) * (a + b)$
e	<pre>if ((x + y) > (z + w)) { result = x + y; } else { result = z + w; }</pre>
f	$(a * b) + (c * d) + (a * b) + (c * d)$
g	<pre>while ((i < n) && (i < n)) { sum = sum + i; i = i + 1; }</pre>
h	$((x - y) + (y - x)) * (x - y)$
i	<pre>if ((a + b) == (c + d)) { x = a + b; } else { x = c + d; }</pre>

Chapter 7

RUN-TIME ENVIRONMENT

Introduction

When a program runs, it needs memory to store many things. It needs space for variables, values, instructions, and function data. All these things must be stored in the correct place.

Memory is limited, so it must be used carefully. If memory is not managed well, the program may become slow or stop working. It may also give wrong output.

The run-time environment manages all this memory. It decides where data will be stored and how it will be used. It also decides when the data should be removed.

When a function is called, the run-time environment gives memory for it. When the function finishes, it removes that memory. This helps reuse memory again and again.

It also controls how data moves between functions. It makes sure that correct values are passed and received.

The run-time environment also keeps track of the program step by step. It knows which part of the program is running and what will run next.

So, it acts like a manager that controls everything during program execution.

Without it, the program cannot run properly. So, it is a very important part of compiler design.

Storage Organization

Storage organization means how memory is arranged and used when a program runs.

When a program starts, it needs memory for many things. It needs space for variables, functions, and data. This memory cannot be used randomly. It must be divided in a proper way.

Decoding Compilers

So, the memory is divided into different sections. Each section has a specific job. This helps the program use memory in a clear and organized way.

Because of this organization, the compiler and program know where to store data and how to find it quickly. It becomes easy to manage memory.

If memory is not organized properly, many problems can happen. The program may use wrong data, give incorrect results, or even crash.

So, storage organization is very important for correct program execution.

It helps in: Storage organization helps the program in many ways. It stores data in the correct place so that the program can find it easily. It also helps with accessing data quickly, which makes the program faster. By organizing memory properly, it avoids confusion and errors. It also helps in managing memory in a clear and efficient way.

The main parts of memory are: The main parts of memory are stack, heap, and static memory. The stack is used for function calls and local variables. The heap is used for dynamic memory, where memory is given when needed. Static memory is used for fixed data like global variables that stay for the whole program.

Each part has its own role. Together, they help the program run smoothly and efficiently.

Stack, Heap, Static Memory

When a program runs, it uses different types of memory to store data. This memory is divided into three main parts: stack, heap, and static memory. Each part has a different purpose and works in a different way.

The stack is used for function calls and local variables. It works automatically and is very fast. Memory is added when a function starts and removed when the function ends.

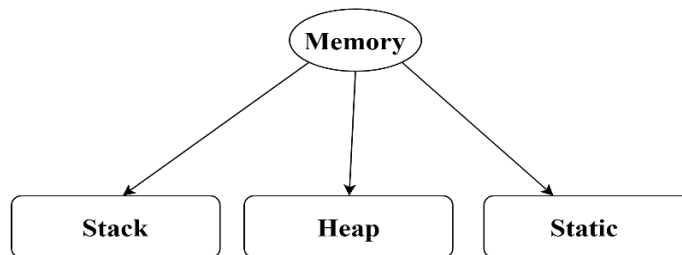
The heap is used for dynamic memory. Memory is given when the program needs it. It is more flexible but slower than the stack. The programmer must manage this memory.

Static memory is used for fixed data like global variables. This memory stays for the whole program and does not change during execution.

Decoding Compilers

All these memory parts work together to help the program run smoothly. They make sure data is stored correctly and used efficiently.

There are three types of memory:



Stack Memory

Stack memory is used to store function calls and local variables. It is a very important part of memory during program execution.

When a function starts, the run-time environment gives memory from the stack. This memory block is called an activation record. It stores things like local variables, parameters, and return address.

When the function finishes, the memory is removed automatically. The programmer does not need to manage it. This makes stack memory easy to use.

Stack works in LIFO order (Last In First Out). This means the last function that starts will finish first

Example:

```
main() {  
    fun1();  
}  
  
fun1() {  
    fun2();  
}  
  
fun2() {  
}
```

Decoding Compilers

Here,

1. `main()` calls `fun1()`
2. `fun1()` calls `fun2()`

So, stack works like this:

1. `main()` starts
2. `fun1()` starts
3. `fun2()` starts
4. `fun2()` ends
5. `fun1()` ends
6. `main()` ends

So, the last function (`fun2`) ends first.

Stack memory is very fast because it is managed automatically. But it has limited size.

If too many function calls happen (like deep recursion), the stack becomes full. This is called stack overflow.

Heap Memory

Heap memory is used when memory is needed during program execution. It is used for dynamic data, which means data that is not known before the program starts.

Sometimes, the program does not know how much memory it will need. So, it asks for memory while running. This memory is taken from the heap.

The program must manage this memory manually. This means the programmer must free the memory after using it.

Example:

```
int *p;  
p = malloc(sizeof(int));  
*p = 10;
```

Here, memory is taken from the heap and stored in pointer `p`.

After using it, we must free it:

```
free(p);
```

Decoding Compilers

Heap memory is flexible. It can store large data and can grow when needed.

But heap is slower than stack because it needs extra work to manage memory.

If memory is not freed properly, it stays in memory even when it is not needed. This is called a memory leak. It can make the program slow and use too much memory.

So, heap memory is powerful but must be used carefully.

Static Memory

Static memory is used for fixed data in a program. This means the data is known before the program starts.

When a program begins, static variables are created and stored in memory. These variables stay in memory until the program ends. They are not removed during execution.

Because of this, static memory is easy to use. The program can access these variables anytime without creating or deleting them again and again.

Example:

```
int x = 10;    // global variable
void fun(){
    static int y = 5;    // static variable
}
```

Here, x and y are stored in static memory. They stay in memory for the whole program.

Static memory is fast because it does not need to be created or removed repeatedly.

But it is not flexible like heap memory. The size and data are fixed.

So, static memory is simple, fast, and useful for data that does not change.

Activation Records

Activation record is a memory block created for each function call.

It stores all information needed to run the function.

This includes:

1. Input parameters

Decoding Compilers

2. Local variables
3. Return address
4. Temporary values

Each function call gets its own activation record.

When functions call other functions, many activation records are created in the stack.

When a function finishes, its record is removed.

This helps manage nested and recursive function calls properly.

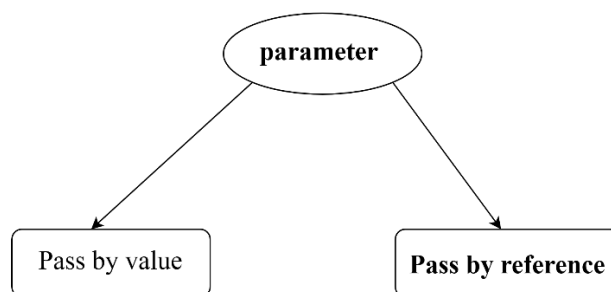
Parameter Passing

Parameter passing means sending data from one part of a program to a function.

Functions often need input values to do their work. These values are called parameters. Parameter passing helps functions use data from other parts of the program.

It also helps in sharing data between functions. This makes the program organized and reusable.

There are two main types of parameters passing:



Pass by Value

In pass by value, a copy of the value is sent to the function.

The function works on this copy. So, if the function changes the value, the original value does not change.

Example:

Decoding Compilers

```
void fun(int x){
    x = 20;
}

int main(){
    int a = 10;
    fun(a);
}
```

Here, a is 10. The function gets a copy of a. When x becomes 20 inside the function, the original a is still 10.

This method is safe because the original data is not changed. But it uses more memory because a copy is created.

Pass by Reference

In pass by reference, the actual variable is sent to the function.

The function works on the original variable. So, if the function changes the value, the original value also changes.

Example:

```
void fun(int *x){
    *x = 20;
}

int main(){
    int a = 10;
    fun(&a);
}
```

Here, the function gets the address of a. When the function changes the value, a becomes 20.

This method is faster because no copy is made. It also uses less memory. But it is not fully safe because the original value can be changed.

Recursion Handling

Recursion happens when a function calls itself again and again. It is used to solve problems step by step.

When a recursive function runs, each call creates a new activation record in the stack. This record stores the function's data like variables and return address.

Decoding Compilers

The stack keeps all these function calls in order. The last function call is stored at the top of the stack.

Example:

```
void fun(int n){  
    if(n == 0) return;  
    fun(n-1);  
}
```

If we call fun(3), the calls will be:

1. fun(3)
2. fun(2)
3. fun(1)
4. fun(0)

Each call is stored in the stack.

When the function reaches the stopping condition (base case), it stops calling itself. Then the functions return one by one. The stack removes each activation record step by step.

If there is no stopping condition, the function will call itself forever. This will fill the stack. When the stack becomes full, it causes a stack overflow error. So, every recursive function must have a proper stopping condition.

Recursion is useful for problems like factorial, tree traversal, and searching. But it must be used carefully to avoid too much memory use.

Chapter 7 Exercises

Part A: Define / Explain

1. Define run-time environment.
2. What is storage organization?
3. Explain stack memory.
4. What is heap memory?
5. Define static memory.
6. What is an activation record?
7. Explain parameter passing.
8. What is pass by value?
9. What is pass by reference?
10. Define recursion handling.

Part B: True / False

1. Stack memory is used for function calls.
2. Heap memory is managed automatically.
3. Static memory is used for global variables.
4. Activation record stores function information.
5. Pass by value changes original data.
6. Pass by reference uses the original variable.
7. Recursion does not use stack.
8. Stack overflow happens when stack is full.

Part C: Short Answer Questions

1. Why is run-time environment important?

Decoding Compilers

2. What are the parts of memory?
3. What is LIFO in stack?
4. Give one example of heap memory use.
5. What is the difference between stack and heap?
6. What is a memory leak?
7. Why is recursion risky?

4. Comparison / Analytical Questions

1. Compare stack and heap memory.
2. Difference between static and dynamic memory.
3. Compare pass by value and pass by reference.
4. Difference between recursion and loop.

5. Scenario / Story-Based Questions

1. A function is called and then another function is called inside it. Which memory is used to store this?
2. A programmer forgets to free memory from heap. What problem can happen?
3. A variable is used for the whole program. Which memory is used?
4. A function changes the original value of a variable. Which parameter passing method is used?
5. A function calls itself many times and program crashes. What is the reason?

6. Challenge Questions

1. Draw stack working for function calls.
2. Explain how activation records are used in recursion.

Chapter 8

CODE OPTIMIZATION

Introduction

Suppose you write a program that works perfectly. It gives the correct output. But when you run it, it takes too much time. Or maybe it uses too much memory.

Now think like this. You have a messy room. Everything is there, but it is not arranged properly. You clean and arrange it. After that, you can move easily and find things quickly.

In the same way, the compiler improves the program. It does not change what the program does. It only changes how the program runs. This process is called code optimization. So, code optimization means making the program better without changing its result.

Goals of Optimization (Performance, Space, Energy)

The main goal of optimization is to improve how the program works. One goal is performance, which means making the program run faster. Faster programs save time and give results quickly.

Another goal is to reduce space, which means using less memory. Memory is limited, so using less memory helps the program run smoothly even on small devices.

Another important goal is saving energy. Programs that do less work use less power. This is very important for mobile phones and laptops because it helps the battery last longer.

So, optimization makes the program faster, smaller, and more energy efficient.

Easy to Remember:

Goal	Meaning	Benefit
Performance	Make the program run faster	Saves time

Space	Use less memory	Works better on small devices
Energy	Use less power	Saves battery

Levels of Optimization

Optimization can happen at different stages of the program. Machine-independent optimization is done before the program is converted into machine code. It improves the program in a general way and does not depend on any specific computer.

Machine-dependent optimization is done after that. It depends on the hardware or processor. It uses special features of the machine to make the program run faster.

So, machine-independent optimization works for all systems, while machine-dependent optimization is specific to one system.

Easy to Remember:

Type	Meaning	When Used
Machine-independent	Does not depend on hardware	Before machine code
Machine-dependent	Depends on processor or system	After machine code

Scope of Optimization (Local vs Global)

Optimization can also be done on different parts of a program. Local optimization works on a small part of the program, usually inside a basic block. It is simple and quick but gives limited improvement.

Global optimization works on the whole program. It looks at many parts together and finds better ways to improve the code. It gives better results but is more complex and takes more time.

So, local optimization is easy and fast, while global optimization is more powerful but harder to apply.

Easy to Remember:

Type	Meaning	Advantage
Local	Works on small part	Simple and fast

Global	Works on whole program	Better result
--------	------------------------	---------------

Basic Blocks and Flow Graphs

Some parts of the program run in a straight line, without any jump or stop. These parts are easy to understand and analyze.

The compiler also divides a program into small parts to understand it better. These parts are called basic blocks.

To understand how the program moves from one part to another, the compiler uses a diagram called a control flow graph.

In this section, we will learn about basic blocks, flow graphs, leader statements, and how they help in optimization.

Definition of Basic Blocks

A basic block is a group of statements that are executed one after another.

There is no jump or branch inside a basic block. Once execution starts, it continues until the end of the block.

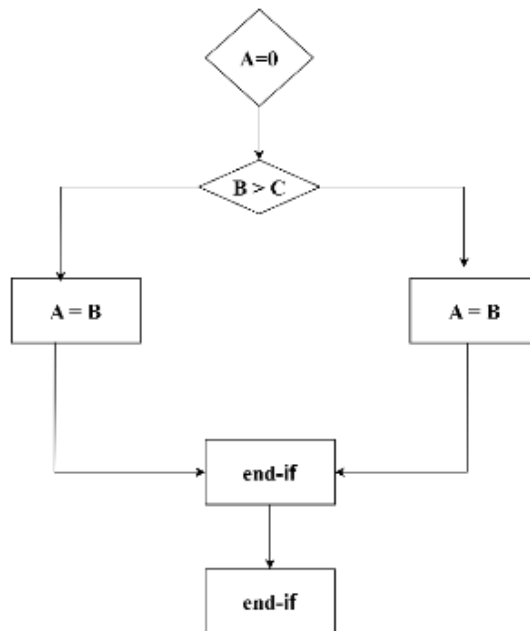
This means all statements inside the block will always run together. Because of this, basic blocks are easy for the compiler to analyze and optimize.

Control Flow Graph (CFG)

A control flow graph shows how the program moves from one basic block to another.

Each basic block is shown as a node. Arrows show the direction of execution. The graph helps the compiler understand all possible paths in the program.

By using CFG, the compiler can easily find which parts of the program can be improved.



Leader Statements and Block Boundaries

To create basic blocks, the compiler first needs to divide the program into smaller parts. For this, it finds special statements called leaders.

A leader is the first statement of a basic block. It shows where a new block begins. Without leaders, the compiler cannot correctly divide the program.

There are three important places where leaders are found. The first statement of the program is always a leader because execution starts from there. The target of a jump statement is also a leader because the program may directly go to that line. Also, any statement that comes after a jump becomes a leader, because execution continues from there.

After identifying all leaders, the compiler groups the statements into basic blocks. Each block starts with a leader and includes all statements until the next leader appears. This way, the program is divided into clear and manageable parts.

This process helps the compiler understand the structure of the program step by step. It also makes further analysis easier.

Use in Optimization

Basic blocks and control flow graphs are very important in optimization. They help the compiler understand how the program moves from one part to another.

With basic blocks, the compiler looks at small parts of the program. This makes it easy to find extra or repeated work. For example, if the same calculation is written many times, the compiler can do it once and use the result again.

The control flow graph shows all possible paths of the program. It helps the compiler find code that is never used. This unused code can be removed to make the program smaller and faster.

Basic blocks also help improve loops. Loops run many times, so improving them can make a big difference. The compiler can move extra work outside the loop or reduce repeated work.

So, basic blocks and CFG help the compiler understand the program clearly and make it faster, smaller, and better.

Local Optimization Techniques

Local optimization means making a small part of a program better. The compiler looks at a small piece of code and tries to improve it.

Sometimes a program has extra work. It may do the same calculation again and again. It may also use extra variables that are not needed. These things make the program slow.

The compiler finds these extra parts and removes them. It also makes the code shorter and cleaner. The main idea is to reduce unnecessary work. This helps the program run faster and use less memory.

The output of the program does not change. The result stays the same, but the program becomes better and more efficient.

Constant Folding

Constant folding means solving simple calculations before the program runs.

When the compiler sees a calculation that has only numbers, it can find the answer early. It does not wait for the program to run.

Decoding Compilers

Example:

$$x = 2 + 3 \rightarrow x = 5$$

Here, the compiler already knows that $2 + 3 = 5$. So, it replaces the calculation with the result.

This means the program does not need to calculate it again during execution. So, it saves time.

If there are many such calculations in the program, removing them early can make the program much faster. It also makes the code simpler and easier to read.

Algebraic Simplification

Algebraic simplification means removing extra math operations that are not needed.

Sometimes a program has calculations that do not change the result. These operations are useless. They only make the program longer and slower. The compiler finds these unnecessary operations and removes them.

Example:

$$x = y + 0 \rightarrow x = y$$

$$x = y * 1 \rightarrow x = y$$

Here, adding 0 or multiplying by 1 does not change the value. So, the compiler removes these operations.

This makes the code shorter and cleaner. It also makes the program run faster because less work is done. It also helps the code become easier to read and understand.

Strength Reduction

Strength reduction means replacing a slow operation with a faster one. Strength reduction means replacing a slow operation with a faster one. Some operations take more time to run, like multiplication and division. These operations are called heavy operations.

The compiler tries to replace these with simpler operations like addition or subtraction. These are faster and take less time.

Decoding Compilers

Example:

$x = y * 2 \rightarrow x = y + y$

Here, multiplication is replaced with addition. Since addition is faster, the program runs faster. This is very useful inside loops. Loops run many times, so even a small change can save a lot of time. So, by using faster operations, the compiler improves the speed of the program

Copy Propagation

Copy propagation means removing extra variables and extra steps.

Sometimes in a program, one variable copies another variable. Then that copied variable is used again. This creates extra work. The compiler finds this and removes the extra variable. It uses the original value directly.

Example:

$a = b$
 $c = a \rightarrow c = b$

Here, we do not need a. We can directly use b.

This makes the code shorter and cleaner. It also reduces unnecessary steps in the program. The program becomes easier to understand and may run faster.

Global Optimization Techniques

Global optimization means improving the whole program. It does not work on just one small part. It looks at many parts of the program together.

The compiler studies how different parts of the program are connected. It checks how variables are used and how calculations are repeated.

Sometimes, the same work is done in many places. The compiler finds this and removes extra work. It may also move code to a better place or remove code that is not needed.

Because it looks at the whole program, it can find better ways to improve performance. This makes the program faster and more efficient.

But this process is more complex. It takes more time and needs more analysis than local optimization.

Common Subexpression Elimination

Common subexpression elimination means removing repeated calculations from a program.

Sometimes the same calculation is written many times in different places. This creates extra work for the program.

The compiler finds these repeated expressions. Instead of calculating them again and again, it calculates them only once. Then it stores the result and uses it again where needed.

Example:

```
a = b + c  
d = b + c
```

Here, $b + c$ is calculated two times.

Optimized:

```
t = b + c  
a = t  
d = t
```

Now, the calculation is done only once.

This saves time and reduces extra work. It also makes the program faster and more efficient.

Dead Code Elimination

Dead code elimination means removing code that is not useful in a program.

Sometimes a program has lines of code that are never used. These lines do not affect the final result. They only make the program longer and slower.

The compiler checks the program and finds this unused code. It removes the code so that the program becomes shorter and cleaner.

Example:

```
x = 10  
x = 20
```

Decoding Compilers

Here, the first line is not needed. Its value is replaced by the second line. So, the compiler removes it.

Also, if a variable is declared but never used, it can be removed. By removing dead code, the program becomes faster and uses less memory.

Code Motion (Loop-Invariant Code Motion)

Code motion means moving code to a better place. Sometimes a calculation inside a loop does not change in every loop run. This is called loop-invariant code. The compiler moves this calculation outside the loop.

Example:

```
for(i=0; i<10; i++){  
    x = 2 + 3;  
}
```

Optimized:

```
x = 5;  
for(i=0; i<10; i++){  
}
```

Now the calculation is done only once. This saves time because loops run many times.

Instruction Scheduling

Instruction scheduling means arranging instructions in a better and smarter order.

Sometimes, the instructions in a program are not in the best order. Because of this, the processor may need to wait before doing the next work. This waiting time makes the program slow.

The compiler looks at the instructions and changes their order. It arranges them in a way so that the processor can work continuously without stopping.

For example, if one instruction needs a result from another instruction, the processor may need to wait. The compiler can move other independent instructions in between. This reduces waiting time.

So, instead of doing work step by step with delays, the processor can do useful work all the time.

Decoding Compilers

Instruction scheduling helps to use the processor efficiently. It improves the speed of execution and makes the program run smoothly.

It is especially useful in modern processors where many instructions can be handled at the same time.

Local Optimization Techniques

Loop optimization means improving loops so that they run faster and use less work.

Loops are very important in a program because they repeat many times. If a loop has even a small extra work, it will be repeated again and again. This can make the program very slow. So, the compiler focuses a lot on improving loops.

Importance of Loops in Performance

Loops run many times in a program. Because of this, they take a large part of execution time.

If we reduce even a small work inside a loop, it can save a lot of time.

For example, if a loop runs 1000 times, and we remove one unnecessary operation, we save that operation 1000 times.

So, loop optimization can greatly improve performance.

Loop Unrolling

Loop unrolling means increasing the work inside the loop and reducing the number of loops runs.

Normally, a loop runs many times with small work. In loop unrolling, we increase the work per loop and reduce loop control.

Example:

```
for(i=0; i<4; i++){  
    print(i);  
}
```

Optimized:

```
print(0);  
print(1);
```

```
print(2);  
print(3);
```

Now, there is no loop control. So, no checking of condition again and again. This reduces overhead and improves speed.

Loop Fusion and Fission

Loop Fusion

Loop fusion means combining two loops into one loop. If two loops do similar work, we can combine them.

Example:

```
for(i=0; i<5; i++) a[i]=i;  
for(i=0; i<5; i++) b[i]=i;
```

Optimized:

```
for(i=0; i<5; i++){  
    a[i]=i;  
    b[i]=i;  
}
```

This reduces loop control and improves performance.

Loop Fission

Loop fission means splitting one loop into multiple loops. Sometimes the splitting helps improve performance or memory use.

Induction Variable Analysis

Induction variable analysis means finding variables that change in a simple and regular way inside a loop.

In most loops, some variables increase or decrease step by step. These variables follow a clear pattern. These are called induction variables.

Example:

```
for(i=0; i<10; i++){  
    x = 2*i;  
}
```


Decoding Compilers

Here, i is an induction variable. It increases by 1 in each loop step.

The compiler studies this pattern. It understands how the value changes in each iteration.

Sometimes, calculations using induction variables can be simplified. The compiler can reduce repeated work inside the loop.

For example, instead of calculating $2*i$ every time, the compiler may update the value step by step.

This reduces the number of calculations and makes the loop faster.

Induction variable analysis is very useful in loop optimization because loops run many times. Even small improvements can save a lot of time.

Loop Interchange

Loop interchange means changing the order of nested loops.

In a program, sometimes we have loops inside other loops. These are called nested loops. The order of these loops can affect how the program runs.

The compiler checks the loops and may change their order to make the program faster.

Example:

```
for (i)
  for (j)
    can become
  for (j)
    for (i)
```

Both programs give the same result. But the speed can be different.

This happens because of memory access. When data is stored in memory, it is stored in a specific order. If we access data in a good order, it becomes faster. If we access data in a bad order, it becomes slow.

Loop interchange helps access data in a better way. It improves cache use and reduces memory delay.

For example, in arrays, accessing elements row by row is faster than column by column (in most systems). Changing loop order helps follow this pattern.

Decoding Compilers

So, loop interchange improves performance by making memory access faster and more efficient.

Sometimes changing order improves speed because data is accessed more efficiently.

Data Flow Analysis for Optimization

Data flow analysis means studying how data moves in a program.

The compiler checks where a variable gets its value. It also checks where that value is used. This helps the compiler understand how the program works.

In a program, a variable may get a value at one place and be used at another place. The compiler tracks this flow of data step by step.

Sometimes, a variable is given a value but never used. This is unnecessary. The compiler can remove it.

Sometimes, the same calculation is done again and again. Data flow analysis helps find this and avoid repeating it.

This process helps the compiler improve the program. It removes extra work and makes the program faster and better.

Concept of Data Flow Analysis

Data flow analysis means checking how data moves in a program.

The compiler looks at where a variable gets a value. It also checks where that value is used.

This helps the compiler understand the program better.

Sometimes a variable gets a value but is not used later. This is not needed. The compiler can remove it.

Sometimes the same calculation is done many times. The compiler can find this and do it only once.

So, data flow analysis helps remove extra work and make the program faster.

Reaching Definitions

Reaching definitions means finding where a variable gets its value in a program.

The compiler checks each variable and finds the line where the value was assigned. This value “reaches” the place where the variable is used.

Example:

```
x = 5;  
y = x + 2;
```

Here, $x = 5$ gives the value to x . When we use x in the next line, this value is used.

So, $x = 5$ is the reaching definition of x .

This helps the compiler understand which value is used at each place. It also helps remove unnecessary assignments and improve the code.

Live Variable Analysis

Live variable analysis means checking which variables are still needed in the future.

The compiler looks at each variable and checks if it will be used later in the program.

If a variable is not used again, it is not needed.

Example:

```
int x = 10;
```

If x is never used later, then this line is useless. The compiler can remove it.

This helps reduce memory use and makes the code smaller and faster. It also helps remove dead code from the program.

Available Expressions

Available expressions mean finding expressions that are already calculated before.

The compiler checks if the same expression appears again. If it is already calculated and nothing has changed, it can reuse the result.

Decoding Compilers

Example:

```
a = b + c;  
d = b + c;
```

Here, $b + c$ is calculated two times.

The compiler can calculate it once and use it again.

Optimized:

```
t = b + c;  
a = t;  
d = t;
```

This saves time and reduces repeated work. It also makes the program faster and more efficient.

Advanced Optimization Techniques

Advanced optimization techniques are used to make programs faster and more efficient in a smarter way. These techniques go beyond simple optimization and improve performance in a deeper level.

The compiler uses these techniques after basic and global optimization. It studies the program carefully and applies special methods to improve speed, reduce memory use, and remove extra work.

These techniques focus on different parts of the program. Some improve small instructions, some reduce function call time, and some improve how the program runs again and again.

For example, peephole optimization improves small parts of code. Inline expansion removes function calls. Tail recursion elimination reduces memory use by converting recursion into loops. Profile-guided optimization focuses on the most used parts of the program.

All these techniques help the program run faster and more smoothly. They also make better use of system resources like memory and processor.

However, advanced optimization can be complex. It needs more analysis and time. So, the compiler must balance between performance and complexity.

Peephole Optimization

Peephole optimization means looking at a small part of code and improving it.

The compiler checks a few instructions at a time. It finds simple patterns and replaces them with better ones.

Sometimes the code has extra or useless operations. The compiler removes or simplifies them.

Example:

$$x = x + 0 \rightarrow x = x$$

Here, adding 0 does nothing. So, it is removed. This makes the code shorter and faster.

Inline Expansion

Inline expansion means replacing a function call with the actual code of that function.

Normally, when a function is called, the program jumps to another place and comes back. This takes time.

The compiler removes this step and directly puts the function code in place.

Example:

call add()

becomes

$$x = a + b$$

Now, there is no function call. This saves time and makes the program faster.

Tail Recursion Elimination

Tail recursion elimination means converting a recursive function into a loop.

In recursion, a function calls itself again and again. This uses memory because each call uses stack space.

If the recursive call is the last step, it can be replaced with a loop.

Decoding Compilers

Loops use less memory and are faster. So, the compiler changes recursion into loop when possible. This improves performance and reduces memory use.

Trade-offs and Limitations

Optimization makes a program faster and better. But it is not always simple. Sometimes improving one thing can make another thing worse.

The compiler must make careful decisions. It cannot always make the program perfect in every way.

So, there are some trade-offs and limitations in optimization. These help us understand that optimization has both benefits and challenges.

Space vs Time Trade-off

Space vs time trade-off means balancing between memory use and speed.

Sometimes, to make a program faster, we need to use more memory. For example, the compiler may store extra values so that it does not need to calculate them again. This saves time but uses more memory.

On the other hand, if we want to use less memory, the program may need to do more calculations. This can make the program slower.

Example:

Instead of calculating a value again and again, we store it in a variable. This uses memory but saves time.

So, the compiler must choose the best balance between speed and memory.

Complexity of Analysis

Optimization is not always easy. Some techniques are simple, but advanced optimization is complex.

The compiler must analyze many parts of the program. It needs to understand data flow, loops, and dependencies. This takes time and effort.

More optimization means more checking and more processing. This increases the complexity of the compiler.

Also, very complex optimization may not always give a big improvement. So, the compiler must decide how much optimization is needed

Machine-Specific Considerations

Some optimizations depend on the machine or hardware.

Different computers have different processors, memory systems, and instruction sets. So, one optimization may work well on one machine but not on another.

The compiler must consider the machine features before applying some optimizations.

Example:

An optimization for a fast CPU may not work well on a slow CPU. So, machine-dependent optimization must match the system.

Chapter 8 Exercises

Part A: Define / Explain

1. Define code optimization.
2. What is the goal of optimization?
3. Explain local optimization.
4. What is global optimization?
5. Define basic block.
6. What is control flow graph (CFG)?
7. Explain constant folding.
8. What is dead code elimination?
9. Define loop optimization.
10. What is data flow analysis?
11. Explain peephole optimization.
12. What is inline expansion?

Part B: True / False

1. Code optimization changes the output of a program.
2. Constant folding reduces runtime calculation.
3. Dead code elimination removes useful code.
4. Loops are important for performance.
5. Local optimization works on small part of code.
6. Global optimization works on whole program.
7. CFG shows program flow.
8. Inline expansion increases function calls.

Part C: Short Answer Questions

1. Why is code optimization important?
2. What is the difference between local and global optimization?
3. Give one example of constant folding.
4. What is strength reduction?
5. Why are loops important in optimization?
6. What is live variable analysis?
7. What is instruction scheduling?

Part D: Comparison / Analytical Questions

1. Compare local and global optimization.
2. Difference between constant folding and algebraic simplification.
3. Compare loop unrolling and loop fusion.
4. Difference between peephole optimization and inline expansion.
5. Compare space vs time trade-off.

Chapter 9

INTRODUCTION TO TCG

Introduction

In the previous chapters, we studied how a source program is analyzed, translated into intermediate code, and optimized. After optimization, the compiler must generate the final program that can run on a machine. This phase is called Target Code Generation.

Target code generation is the final phase of the compiler. In this phase, the intermediate representation (IR) of the program is converted into assembly language or machine language for a specific target machine. The generated code must be correct, efficient, and make proper use of machine resources such as registers and memory.

Unlike machine-independent phases (such as lexical analysis, syntax analysis, and intermediate code generation), target code generation is machine-dependent. This means the code generator must consider the architecture of the target machine, including:

1. Instruction set
2. Number of registers
3. Addressing modes
4. Memory organization

The main challenge of code generation is to produce efficient code. The compiler must decide:

1. Which instructions to use (instruction selection)
2. How to use registers efficiently (register allocation)
3. How to access variables in memory (addressing modes)
4. How to improve small parts of code (peephole optimization)
5. How to generate code for control statements like if, while, and for

Decoding Compilers

The quality of the generated target code affects the speed and memory usage of the final program. A good code generator produces fast and compact code, while a poor code generator produces slow and inefficient code.

In this chapter, we will study the basic problems and techniques used in target code generation, including instruction selection, register allocation, addressing modes, peephole optimization, and code generation for control flow statements.

Target Code Generation

To understand target code generation, imagine a factory that produces a finished product. The raw material enters the factory, passes through several stages, and at the end, a final product is produced.

A compiler works in a similar way. The source code is like the raw material, and it passes through different phases such as lexical analysis, syntax analysis, semantic analysis, intermediate code generation, and code optimization. After all these steps, the program reaches the final stage, called target code generation, where the final machine code is produced.

In this final stage, the code generator acts like a skilled worker who converts the semi-finished product into a final product that the machine can use. The code generator does not work with the original source code. Instead, it takes the intermediate representation (IR) of the program, which was produced by earlier phases of the compiler.

This intermediate code is easier to translate into machine code because it is simple and machine-independent. Along with the intermediate code, the code generator also uses the symbol table, which contains important information about the variables used in the program, such as their names, data types, and memory locations. With the help of the symbol table, the code generator knows where each variable is stored and how to access it.

In simple words, target code generation is the phase where the compiler converts intermediate code into machine code for a specific machine, while trying to make the program run as fast and as efficiently as possible. This is the final step of the compiler, and the code produced in this step is the actual program that the computer executes.

Role of Target Code Generator

The target code generator is the final component of the compiler that is responsible for converting intermediate code into target machine code. Its main role is to produce correct and efficient assembly or machine instructions that can be executed directly by the computer. Since this phase directly affects the performance of the final program, the code generator must carefully use machine resources such as registers and memory.

The code generator performs several important tasks during this process. First, it translates each intermediate instruction into one or more machine instructions. At the same time, it must choose appropriate instructions based on the target machine's instruction set. It also manages the use of registers. This decides which variables should be stored in registers and when values should be moved to or from memory.

Another important role of the code generator is to ensure efficient memory usage. Since memory access is slower than register access, the code generator tries to minimize unnecessary memory operations. It also determines the proper order of evaluation of expressions so that the number of registers required is minimized.

In addition, the code generator aims to produce optimized code by avoiding redundant instructions and using efficient instruction patterns whenever possible. Although major optimizations are performed earlier, small improvements are still handled at this stage.

Main Responsibilities of Code Generator:

1. Convert intermediate code into assembly or machine code
2. Select appropriate machine instructions
3. Allocate and manage registers efficiently
4. Control memory access (load/store operations)
5. Determine the order of evaluation of expressions
6. Produce efficient and optimized target code

In summary, the code generator acts as the final translator that turns the compiler's internal representation into a real executable program while ensuring correctness and efficiency.

Input to Code Generator (IR, Symbol Table)

The code generator does not work with the original source code directly. Instead, it relies on two important inputs from earlier phases of the compiler: the Intermediate Representation (IR) (note that it's the Optimized IR from code optimization phase) and the Symbol Table. The IR acts like a simplified, half-processed version of the program, while the symbol table provides detailed information about all variables and identifiers used in the program.

The Intermediate Representation (IR) is a machine-independent code that represents the program in a simple form that is easier to translate into machine instructions. A common form of IR is three-address code (TAC), where each instruction has at most three addresses or operands. For example, consider the source code:

$$d = (a + b) * c + e;$$

The intermediate code (TAC) might look like this:

```
t1 = a + b
t2 = t1 * c
d = t2 + e
```

Here, t1 and t2 are temporary variables used to hold intermediate results. The code generator reads each of these instructions and translates them into target machine instructions.

The Symbol Table is a data structure that stores information about all the identifiers in the program, such as variable names, data types, scope, and memory locations. For example, the symbol table for the above program may look like this:

Variable	Type	Memory Location
a	int	1000
b	int	1004
c	int	1008
d	int	1012
e	int	1016

The code generator uses this table to know where each variable is stored in memory and how to access it. For example, when generating the instruction for $a + b$, it knows it must read a from memory location 1000 and b from 1004.

Output of Code Generator

Once the code generator has received the optimized intermediate representation (IR) and the symbol table, its main job is to produce the final target code that the computer can execute. This output is usually in the form of assembly language or machine code. Assembly code is a symbolic representation of machine instructions that is easier for humans to read, while machine code is the binary instructions directly understood by the processor. The quality of this output directly affects the performance of the final program.

Let's continue with the example we used earlier. Suppose the source program is:

$$d = (a + b) * c + e;$$

The intermediate code (TAC) is:

```
t1 = a + b
t2 = t1 * c
d = t2 + e
```

And the symbol table tells us the memory locations of the variables:

Variable	Memory Location
a	1000
b	1004
c	1008
d	1012
e	1016

The code generator reads this information and produces target instructions. An example of assembly-like output might be:

```
MOV R1, [1000] ; Load a into R1
ADD R1, [1004] ; Add b to R1 → R1 = a + b
MUL R1, [1008] ; Multiply by c → R1 = (a + b) * c
ADD R1, [1016] ; Add e → R1 = (a + b) * c + e
MOV [1012], R1 ; Store result in d
```

Or a similar version is:

```
LDF R1, a
LDF R2, b
ADDF R1, R1, R2
LDF R2, c
MULF R1, R1, R2
```

LDF	R2, e
ADDF	R1, R1, R2
STF	d, R1

Here, the code generator has translated the intermediate instructions into machine-level operations, chosen appropriate registers, and used the memory locations provided by the symbol table. This output is ready for execution on the target machine.

Issues in Code Generation

The code generator's job is to convert the optimized intermediate code into machine code that runs efficiently on the target machine. However, this is not a simple task. There are several important challenges, or issues, that the code generator must handle to produce correct and efficient code. These issues affect both the speed and memory usage of the final program.

One of the main issues is instruction selection. Each operation in the intermediate code may be implemented using different machine instructions. The compiler must choose the most appropriate instruction depending on the target machine's instruction set. For example, adding two numbers could be done in a single instruction if the machine supports it, or it might require multiple instructions if the operands are in memory. Choosing the right instruction is crucial for efficiency.

Another critical issue is registers allocation. Registers are very fast but limited in number, while memory is slower and abundant. The code generator must decide which variables should reside in registers and which should be stored in memory. Poor register allocation can lead to frequent memory accesses, slowing down the program. For example, if a variable is repeatedly used in calculations, keeping it in a register instead of memory can save time.

Memory usage and addressing modes are also important. Different machines provide different ways to access memory, such as direct, indirect, or indexed addressing. The compiler must choose the best mode for each variable access. Efficient use of memory reduces execution time and avoids unnecessary memory operations.

The order of evaluation of expressions is another challenge. Consider the expression:

$$a + b * c + d$$

The compiler must decide the sequence of operations and the temporary storage required so that it uses as few registers as possible. Incorrect ordering can lead to excessive register usage or extra memory loads and stores.

Finally, small local optimizations can be performed during code generation, sometimes called peephole optimizations. For instance, two consecutive instructions might cancel each other out, or a simpler instruction could replace a more complex one. These small improvements, though minor, help generate faster and smaller machine code.

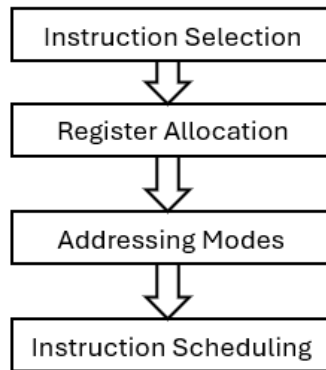
Summary of main issues:

1. **Instruction Selection:** Choosing the most appropriate machine instructions for each operation.
2. **Register Allocation:** Deciding which variables go into registers and which stay in memory.
3. **Memory Access & Addressing Modes:** Choosing efficient ways to access operands.
4. **Order of Evaluation:** Determining the sequence of operations to minimize register and memory usage.
5. **Local / Peephole Optimizations:** Making small improvements to improve speed and reduce code size.

Phases of Target Code Generation

Target code generation is not done in one single step. A compiler performs it in several smaller phases, each with a specific job. You can think of it like a factory line where each worker has a task, and together they produce the final product: the machine code.

There are three main phase and an optional phase for target code generation. Phase are:



Here, Instruction Scheduling is the optional phase.

Instruction Selection

The instruction selection phase of code generation decides which machine instructions should be used to implement the operations in the program. Think of it like choosing the right tool to do a specific job in a factory. For each task in the intermediate code, the compiler must pick a machine instruction that will perform the same task efficiently.

Translating TAC to Target Code

When the compiler translates intermediate code into machine instructions, it often works with three-address code (TAC). TAC breaks complex expressions into simple steps using temporary variables. The code generator then converts each step into machine instructions that the CPU can execute.

When generating code for simple arithmetic expressions, the compiler follows a straightforward process:

1. Load operands into registers.
2. Perform the arithmetic operation using a machine instruction.
3. Store the result back into memory or a register.

Different CPUs have different machine instructions, and the compiler must choose the appropriate instruction for each operation.

Some important points:

Arithmetic Instructions: Perform addition, subtraction, multiplication, division (e.g., ADDF, SUBF, MULF, DIVF).

Decoding Compilers

Load/Store Instructions: Move data between memory and registers (e.g., LDF, STF).

Immediate Instructions: Use constants directly in operations (e.g., ADDF R1, R1, #5).

Suppose we have the following source code:

$$d = (a + b) * c + e;$$

The TAC might like:

```
t1 = a + b
t2 = t1 * c
t3 = t2 + e
d = t3
```

Now, we translate this TAC into target instructions using a floating-point example (or any CPU instruction style). The translation might be:

```
LDF    R1, a      ; Load a into R1
LDF    R2, b      ; Load b into R2
ADDF   R1, R1, R2  ; Add R1 + R2 → R1
LDF    R2, c      ; Load c into R2
MULF   R1, R1, R2  ; Multiply R1 * R2 → R1
LDF    R2, e      ; Load e into R2
ADDF   R1, R1, R2  ; Add R1 + R2 → R1
STF    d, R1      ; Store result from R1 into d
```

The compiler chooses registers and instructions based on the target machine. Temporary variables t1, t2, t3 are replaced by register usage, which is faster than memory access.

Cost of Instructions

In target code generation, not all instructions are equal. Each machine instruction has a “cost”. It usually measured in terms of execution time, memory usage, or register pressure. Understanding instruction costs is important because it helps the compiler generate efficient code that runs faster and uses fewer resources.

Register vs Memory Access

Decoding Compilers

Registers are inside the CPU and are very fast. Accessing memory, even main memory, is much slower. For example:

```
ADDF R1, R2, R3 ; Add contents of R2 and R3 → R1 (fast)
ADDF R1, R2, [1000] ; Add memory value at address 1000 → R1 (slower)
```

The first instruction operates entirely in registers which is very fast. But the second instruction requires a memory read, which is slower because the CPU must fetch the value from RAM. So, Compilers try to keep frequently used variables in registers to reduce memory accesses.

Arithmetic Operation Cost

Different arithmetic operations have different costs:

Addition / Subtraction are usually fast, single-cycle instructions. Multiplication / Division are often slower, may take multiple cycles. Floating-point operations are typically slower than integer operations, especially multiplication and division. Also note that, the compiler may reorder operations or simplify expressions to minimize expensive operations.

Immediate vs Memory Operand

Using constants directly in instructions is cheaper than loading constants from memory. Compilers prefer immediate instructions whenever possible. For example:

```
ADDF R1, R1, #5 ; Add constant 5 directly
LDF R2, five ; Load 5 from memory → slower
ADDF R1, R1, R2 ; Add → slower overall
```

Instruction Sequence and Pipeline Effects

Some CPUs can execute instructions in pipelines, meaning multiple instructions overlap. If an instruction depends on the result of a previous one, the CPU may have to stall, increasing the effective cost. The compiler can sometimes reorder independent instructions to avoid stalls and reduce overall execution time. For example:

```
LDF R1, a
ADDF R1, R1, b ; Dependent on previous load
```

Register Allocation

Registers are the fastest storage units in a computer. A good register allocation strategy can make a program run much faster, while poor allocation can slow it down significantly. Register allocation is the process of assigning variables and temporary values to CPU registers instead of memory. This topic explains why registers are important, how the compiler assigns them, and what happens when there are not enough registers.

Need for Register Allocation

Every program uses variables, temporary values, and constants. If all of them were stored in main memory, every operation would require slow memory access. Registers solve this problem because they are inside the CPU and much faster. For example:

Without register:

```
LDF R1, a ; Load a from memory
ADD R1, R1, b ; Add b from memory
STF c, R1 ; Store result back to memory
```

Here, each load/store accesses memory which is slower execution.

With registers:

```
LDF R1, a ; Load a into register
LDF R2, b ; Load b into another register
ADD R1, R1, R2 ; Add in registers
STF c, R1 ; Store result once → faster
```

Here, registers allow fewer memory accesses, which improves speed and efficiency.

Register Assignment

The compiler decides which variables or temporary values go into which registers. There are two main approaches:

1. Simple Assignment
2. Lifetime Analysis

Simple Assignment

Decoding Compilers

Assign variables to registers as needed, keeping frequently used variables in registers. For example:

```
LDF R1, x
LDF R2, y
ADDF R1, R1, R2
STF z, R1
```

Here,

1. $x \rightarrow R1, y \rightarrow R2, \text{result} \rightarrow R1$.
2. Temporary values are also stored in registers.

Lifetime Analysis

The compiler tracks when a variable is needed (its lifetime) and tries to reuse registers for non-overlapping variables. For example:

1. a is used from line 1 to 3
2. b is used from line 2 to 4
3. If lifetimes don't overlap, the same register can be reused which will reduce register usage.

Register Spilling

Sometimes, the number of variables exceeds available registers. In this case, the compiler performs register spilling:

1. Some variables are temporarily stored in memory instead of registers.
2. Spilled variables (a variable that cannot fit in a CPU register) are loaded into registers only when needed.

For example, suppose only 2 registers are available, but 3 variables a, b, c are needed:

```
LDF R1, a    ; Load a
LDF R2, b    ; Load b
STF a_mem, R1 ; Spill a to memory
LDF R1, c    ; Load c into R1
ADDF R1, R1, R2 ; Compute c + b
```

Spilling slows execution because memory is slower than registers, so compilers try to minimize spills using careful allocation.

Local Register Allocation

Local register allocation is the process of assigning registers to variables within a single basic block. A basic block is a straight-line sequence of instructions with no jumps or branches, so all instructions are executed in order. By working locally, the compiler can assign registers more easily and efficiently, without worrying about complex interactions across the entire program.

Think of it like organizing tools on a workbench for a single task: you only need the tools for that task, and once you finish, the tools can be reused for the next task.

Process of Local Register Allocation:

1. Identify variable lifetimes
2. Assign registers to variables
3. Reuse registers efficiently

Benefits of Local Register Allocation:

1. Reduces memory accesses: Keeps temporary variables in registers.
2. Simpler than global allocation: Works only within one basic block.
3. Improves speed: Fewer loads and stores mean faster execution.
4. Registers are reused efficiently: Minimizes need for spilling to memory.

Addressing Modes

When a compiler generates target code, it must decide how to access the data, whether the operand is in a register, memory, or is a constant. These different ways of accessing operands are called addressing modes. Choosing the right addressing mode can make the program faster, simpler, and smaller.

Think of it like finding items in a kitchen. Sometimes you grab something from the counter (register), sometimes from a drawer (memory), and sometimes you already have it in your hand (immediate value).

Different types of addressing modes:

1. Immediate
2. Direct
3. Indirect
4. Register
5. Indexed / Base + offset

Immediate Addressing

In immediate addressing, the operand is a constant value included directly in the instruction. This is fast because no memory access is needed. For example:

```
ADDF R1, R2, #5.0 ; Add constant 5 to value in R2 that store in R1
```

Here, #5.0 is the immediate value. Use it when you want to add, subtract, or load a fixed constant. Think like this: you already have the sugar in your hand instead of taking it from a jar.

Direct Addressing

In direct addressing, the instruction specifies the memory location of the operand. The CPU fetches the value directly from that memory address. For example:

```
LDF R1, a ; Load value of variable a from memory into R1
```

Here, 'a' is stored at a fixed memory location. Note that direct addressing is simple but slower than registers. Think, you take a cup from a specific shelf in your kitchen.

Indirect Addressing

In indirect addressing, the instruction points to a register or memory location that contains the address of the operand. Example:

```
LDF R1, [R2] ; Load value from memory address stored in R2 to R1
```

Here, R2 contains the address of the variable, not the value itself. It is useful for pointers, arrays, and dynamic memory access.

Analogy: You have a note that says “The sugar is in drawer #3,” so you first look at the note, then go to the drawer.

Register Addressing

In register addressing, the operand is already in a CPU register. This is the fastest mode because no memory access is needed. For example:

```
ADDF R1, R2, R3 ; Add R2 and R3 → store result in R1
```

Decoding Compilers

Use register addressing for temporary values and frequently used variables. It reduces instruction cost and improves speed.

Analogy: You already have the butter in your hand, ready to use.

Indexed / Base + Offset Addressing

In indexed or base + offset addressing, the operand's address is computed as base address + offset. This is very useful for arrays and structures. Example:

```
LDF R1, a[R2] ; Load value at address (a + R2) → R1
```

Here, 'a' is the base address, and R2 is the offset. This allows easy access to array elements. Think like, you know sugar is in the 3rd drawer, and the base of the shelf is counted from 0, so you calculate the exact drawer to open.

Choosing Addressing Modes

The compiler chooses an addressing mode based on:

1. Speed: Register > Immediate > Direct > Indirect
2. Availability of registers: If registers are full, memory access may be needed.
3. Nature of the operand:
 - a. Constant → Immediate
 - b. Temporary variable → Register
 - c. Array element → Indexed / Base + Offset
4. Instruction type: Some instructions may only support certain addressing modes.

Instruction Scheduling

Imagine a construction site building a small house. There are several workers. Each assigned different tasks such as pouring the foundation, laying bricks for walls, installing windows, painting and so on.

Some tasks cannot start until others are finished. Such as you can't lay bricks before the foundation is ready. Other tasks, like painting doors or preparing windows, can happen at the same time, independently.

Decoding Compilers

If the site manager assigns tasks strictly in the order they appear on the blueprint, some workers will stand idle, waiting for others to finish. This wastes time and resources.

Instruction scheduling in a compiler is like the site manager reordering tasks intelligently so all workers stay busy and the house is built faster.

When a compiler generates target code, it must consider how the CPU executes instructions. Modern CPUs often have pipelines, where multiple instructions are processed simultaneously at different stages (fetch, decode, execute, memory access, write-back).

If two instructions depend on each other, the CPU may have to wait, causing a pipeline stall, which slows the program. Instruction scheduling is the compiler's technique of reordering instructions to minimize these stalls and make the program run faster.

The Problem: Pipeline Hazards

Modern CPUs execute instructions in a pipeline, similar to the construction workflow:

- Stage 1: Fetch instruction
- Stage 2: Decode instruction
- Stage 3: Execute operation
- Stage 4: Memory access
- Stage 5: Write back result

If an instruction depends on the result of a previous one, the CPU must wait, causing a pipeline stall. Example:

```
LDF  R1, a    ; Load a from memory
ADDF R2, R1, b ; Add b → R2
MULF R3, R2, c ; Multiply → R3
```

Here, ADDF depends on LDF. So, it must wait until LDF finishes. Similarly, MULF depends on ADDF. As a result, it also must wait.

Instruction Scheduling Solution

The compiler can reorder independent instructions to fill idle cycles:

```
LDF R1, a    ; Load a
LDF R4, d    ; Load d (independent)
ADDF R2, R1, b ; Add a + b
MULF R3, R2, c ; Multiply
```

Here, LDF R4, d does not depend on R1. So it can run while waiting. For this, CPU stays busy which means fewer stalls.

Scheduling in Loops

Loops are like building multiple houses in a row. Optimizing one iteration (one house) reduces idle time across all iterations, saving significant overall time. For example:

```
while (i <= n){
    t1 = a[i] + b;
    t2 = c * t1;
}
```

Here, loading `a[i]` and adding `b` are independent of multiplying `c * t1` for the previous iteration. Reordering instructions fills pipeline stages means faster execution.

Peephole Optimization

Imagine a proofreader reviewing a short paragraph of text. The proofreader looks closely at a few words at a time, checking for redundancy, mistakes, or ways to make the sentence clearer and shorter.

This is exactly what peephole optimization does in a compiler. Instead of analyzing the entire program, it examines small sequences of instructions (like a “peephole”) and tries to make them faster, smaller, and more efficient.

Think of it like looking at a few instructions through a tiny window, spotting patterns that can be improved, and rewriting them in a smarter way.

Importance of peephole optimization:

Eliminating Redundant Instructions: Sometimes, the compiler generates instructions that do nothing useful because of repeated or unnecessary operations. Peephole optimization can remove these redundant instructions. For example:

```
MOV R1, R2  
MOV R1, R2 ; Redundant → can remove
```

Here, the second MOV does the same thing as the first instruction, which is unnecessary. After peephole optimization:

```
MOV R1, R2 ; Only one instruction remains
```

Algebraic Simplification: Some instruction sequences perform operations that can be simplified using algebraic rules. For example:

```
ADD R1, R0 ; Adding 0 → can remove  
MUL R2, 1 ; Multiplying by 1 → can remove
```

It simplifies code and reduces execution time.

Strength Reduction: Some operations are expensive, like multiplication or division, and can be replaced by cheaper operations, like addition, subtraction, or bit shifting. For example:

```
MUL R1, 2 can be replaced as ADD R1, R1  
DIV R2, 4 → LSR R2, #2 ; Logical shift right
```

Here, multiplying by 2 can be replaced as ADD which is cheaper. Dividing by powers of 2 is cheaper as bit shift.

Use of Machine Idioms: Certain sequences of instructions can be replaced by a single instruction provided by the machine, called a machine idiom. For example:

```
CLR R1  
ADD R1, R2 ; Could be replaced by MOV R1, R2 depending on machine
```

By using machine-specific instructions reduces code size and speeds up execution.

Code Generation for Control Statements

Imagine a robot following instructions to move around a factory floor. The robot can make decisions, repeat tasks, or skip steps based on conditions. In a program,

Decoding Compilers

control statements like if, if–else, while, do–while, and for tell the CPU how to make decisions and repeat tasks.

The code generator’s job is to translate these decisions into assembly or machine instructions, often using jumps (branch instructions) to control the flow.

Think of it like giving the robot a map with arrows, so it knows where to go based on conditions.

Example-1:

Block:

```
if (a > b)
    max = a;
```

Target code for the statement can be written as:

```
if a > b goto L1
goto L2
L1: max = a
L2:
```

Target Code:

```
LDF  R1, a    ; Load a
LDF  R2, b    ; Load b
CMPF R1, R2   ; Compare a and b
JLE  L2       ; Jump to L2 if a ≤ b
STF  max, R1   ; max = a
L2:
```

Example-2:

Block:

```
if (a > b)
    max = a;
else
    max = b;
```

TAC:

Decoding Compilers

```
if a > b goto L1
goto L2
L1: max = a
goto L3
L2: max = b
L3:
```

Target Code:

```
LDF  R1, a    ; Load a
LDF  R2, b    ; Load b
CMPF R1, R2   ; Compare a and b
JLE  L2       ; Jump if a ≤ b
STF  max, R1  ; max = a
JMP  L3       ; Skip else block
L2: STF max, R2 ; max = b
L3:
```

Example-3:

Block:

```
while (i < 10)
    sum += i;
```

TAC:

```
i = 0
sum = 0
L1: if i >= 10 goto L2
sum = sum + i
i = i + 1
goto L1
L2:
```

Target Code:

```
MOVF R1, #0    ; i = 0
MOVF R2, #0    ; sum = 0
L1: CMPF R1, #10 ; Compare i and 10
JGE  L2       ; Exit loop if i ≥ 10
```

Decoding Compilers

```
ADDF R2, R2, R1 ; sum += i
ADDF R1, R1, #1 ; i++
JMP L1 ; Repeat loop
L2:
```

Example-4:

Block:

```
do
    sum += i;
while (i < 10);
```

TAC:

```
i = 0
sum = 0
L1: sum = sum + i
i = i + 1
if i < 10 goto L1
```

Target Code:

```
MOVF R1, #0 ; i = 0
MOVF R2, #0 ; sum = 0
L1: ADDF R2, R2, R1 ; sum += i
ADDF R1, R1, #1 ; i++
CMPF R1, #10
JL L1 ; Repeat if i < 10
```

Example-5:

Block:

```
for (i = 0; i < 10; i++)
    sum += i;
```

TAC:

```
i = 0
sum = 0
```

Decoding Compilers

```
L1: if i >= 10 goto L2
sum = sum + i
i = i + 1
goto L1
L2:
```

Target Code:

```
MOVF R1, #0    ; i = 0
MOVF R2, #0    ; sum = 0
L1: CMPF R1, #10 ; Compare i and 10
JGE L2        ; Exit if i ≥ 10
ADDF R2, R2, R1 ; sum += i
ADDF R1, R1, #1 ; i++
JMP L1        ; Repeat loop
L2:
```

Chapter 9 Exercises

Part A: Conceptual Questions

1. Explain the role of a code generator in a compiler. Why is it important to generate efficient target code?
2. What are the inputs and outputs of a code generator? Give examples.
3. What is peephole optimization? Explain how it improves code with a short example.
4. Discuss the different addressing modes and give one example of when each would be used.
5. What are the main challenges in register allocation? Explain register spilling with an example.
6. Compare the target code generated for while and do-while loops. How do their jump instructions differ?

Part B: Practice Problems

1. Convert the following three-address code (TAC) into assembly-style target code:

SI	TAC
a	t1 = a + b t2 = t1 * c d = t2
b	if x < y goto L1 z = x - y goto L2 L1: z = y - x L2:
c	i = 0 sum = 0 L1: if i >= 5 goto L2 sum = sum + i i = i + 1 goto L1 L2:

Decoding Compilers

2. Identify redundant instructions and optimize the following code:

```
MOV R1, R2
MOV R1, R2
ADD R3, R1, #0
MUL R4, R5, 1
```

3. Apply strength reduction:

```
MUL R1, 2
DIV R2, 4
```

4. Replace sequences with machine idioms if possible:

```
CLR R1
ADD R1, R2
```

5. Assume you have 3 registers (R1, R2, R3) and the following TAC:

```
t1 = a + b
t2 = c * d
t3 = t1 + t2
```

- Perform local register allocation and show register assignment.
 - Explain if register spilling is required.
6. Explain why register allocation is important for loops, giving a small example with a loop sum calculation.

Part C: Control Statement Exercises

7. Write TAC and Target Code for the following:

SI	Blocks
a	if (a + b > c) d = a + b; else d = c;
b	while (i < n) sum = sum + arr[i];
c	for (i = 1; i <= 5; i++) fact = fact * i;
d	do x = x + 2; while (x < 10);

Chapter 10

INTRODUCTION TO ERROR HANDLING

Introduction

Suppose you are writing a paragraph. Sometimes you may make small mistakes. You may write a wrong word, forget a comma, or write a sentence that does not make proper sense. These mistakes can make your writing hard to understand.

When a teacher checks your writing, the teacher reads carefully and finds these mistakes. The teacher may correct the spelling, fix the punctuation, or explain what is wrong. This helps you improve your writing and make it correct.

In the same way, when we write a program, mistakes can also happen. A programmer may type a wrong symbol, forget a semicolon, or use a wrong data type. These mistakes are called errors.

The compiler works like a teacher. It reads the program and checks for mistakes. If it finds any error, it shows a message so the programmer can fix it.

Error handling is the process of finding and managing these errors. The main goal is not only to detect errors but also to handle them in a smart way. The compiler should try to continue its work even if there are some errors in the program.

There are different types of errors, such as lexical errors, syntax errors, and semantic errors. Each type of error happens at a different stage of the compiler.

In this chapter, we will learn about these different types of errors and understand how the compiler handles them using different methods.

Error Types

Errors are problems in a program. These problems make the program incorrect or difficult to run. Errors can happen at different stages of compilation.

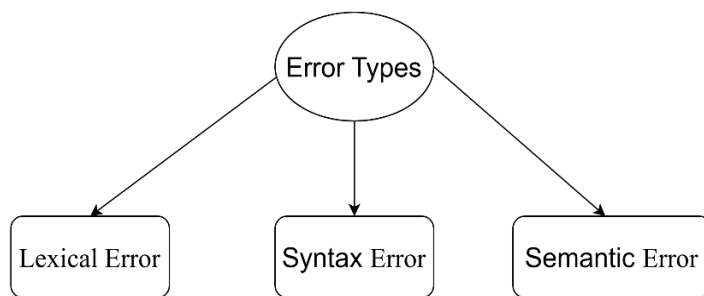
Sometimes errors are very small. For example, a programmer may forget a semicolon or use a wrong symbol. These are simple mistakes but still need to be

Decoding Compilers

fixed. Sometimes errors are more serious. For example, the logic of the program may be wrong, or a variable may be used incorrectly. These errors can make the program give wrong results.

The compiler checks the program step by step. When it finds an error, it shows a message. This message helps the programmer understand what went wrong and where the mistake is.

After seeing the error message, the programmer can fix the code and make it correct. So, error types help us understand what kind of mistake has happened and how we can solve it easily.



Lexical, Syntax, Semantic Errors

Lexical Error

A lexical error happens when the compiler finds an unknown or invalid symbol.

Example 1:

```
int a = 10$;
```

Here, \$ is not valid, so it is a lexical error. This error happens during lexical analysis.

Example 2:

```
int 1a = 5;
```

Identifier cannot start with a number (1a)

Example 3:

```
float x = 5.2.3;
```

Invalid number format (x = 5.2.3)

Decoding Compilers

Syntax Error

A syntax error happens when the structure of the program is wrong.

Example 1:

```
int a = ;
```

Here, the statement is incomplete. This error happens during syntax analysis.

Example 2:

```
int a = 10
```

Missing semicolon (;)

Example 3:

```
if (a > 5 {  
    printf("Hi");  
}
```

Missing closing bracket ‘)’

Semantic Error

A semantic error happens when the program looks correct but does not make sense.

Example 1:

```
int a = "hello";
```

Here, the type is wrong. This error happens during semantic analysis.

Example 2:

```
int x;  
x = y + 5;
```

y is not declared

Example 3:

```
int x = 5;  
float x = 3.2;
```

Same variable declared again with different type

Types of Errors Table

Error Type	Description	Example Code	Problem
Lexical Error	Wrong or unknown symbol in program	<code>int a = 10\$;</code>	\$ is invalid
Syntax Error	Wrong structure or missing part	<code>int a = ;</code>	Value is missing
Semantic Error	Meaning is wrong but structure is correct	<code>int a = "hello";</code>	Type mismatch

Panic Mode Recovery

Sometimes the compiler finds an error in the program and cannot understand what to do next. It may get confused because the structure of the program is broken.

But the compiler does not stop completely. Instead, it tries to recover from the error and continue its work. This is called panic mode recovery.

In panic mode, the compiler skips some part of the input. It ignores the wrong or confusing part until it finds a safe point. A safe point is a place where the compiler can start again properly.

Usually, symbols like semicolon `;`, closing bracket `}`, or keywords are used as safe points.

After finding a safe point, the compiler continues checking the rest of the program.

This method is simple and fast, but sometimes it may skip a large part of the program.

Example 1:

```
int a = 10
```

```
int b = 20;
```

Missing semicolon after 10

Compiler skips until it finds ;

Then continues from `int b = 20;`

Example 2:

Decoding Compilers

```
if (a > 5
    printf("Hi");
}
```

Missing)

Compiler skips input until it finds) or }

Then continues checking

Phrase-Level Recovery

Sometimes the compiler finds a small error in the program. Instead of skipping a large part like panic mode, it tries to fix the error in a small area. This method is called phrase-level recovery.

In this method, the compiler looks at the error and tries to correct it immediately. It may insert a missing symbol, remove an extra symbol, or replace a wrong part.

The goal is to fix the mistake without affecting the rest of the program. So, the compiler keeps most of the code unchanged and continues its work smoothly.

This method is more careful and smarter than panic mode. It helps in handling small errors easily.

Example 1:

```
int a = 10
```

Missing semicolon

Compiler adds ; automatically

Example 2:

```
int a == 10;
```

Wrong operator ==

Compiler may replace it with =

Chapter 10 Exercises

Part A: Define / Explain

1. Define error handling.
2. What is an error in a program?
3. Explain lexical error.
4. What is a syntax error?
5. Define semantic error.
6. What is panic mode recovery?
7. Explain phrase-level recovery.

Part B: True / False

1. Error handling helps the compiler detect mistakes.
2. Lexical errors happen due to wrong symbols.
3. Syntax errors are related to program structure.
4. Semantic errors are related to meaning.
5. Panic mode stops the compiler completely.
6. Phrase-level recovery tries to fix small errors.

Part C: Short Answer Questions

1. Why is error handling important?
2. Give one example of a lexical error.
3. Give one example of a syntax error.
4. Give one example of a semantic error.
5. What is the difference between panic mode and phrase-level recovery?

Part D: Comparison / Analytical Questions

1. Compare lexical error and syntax error.
2. Difference between syntax error and semantic error.
3. Compare panic mode and phrase-level recovery.
4. Difference between lexical, syntax, and semantic errors.

Chapter 11

SYMBOL TABLE MANAGEMENT

Introduction

Imagine you have a notebook where you write important information about your friends. You write their names, phone numbers, and other details so that you can find them later easily. If your notebook is well organized, you can quickly find any information. But if it is messy, it becomes difficult to search.

In the same way, a compiler also needs a place to store information about variables, functions, and identifiers used in a program. This place is called a symbol table.

Whenever a programmer writes a program, many names are used, like variable names, function names, and so on. The compiler must remember all these names and their details. For example, it needs to know the type of a variable, its value, and where it is stored in memory.

Symbol table management means how the compiler stores this information, how it finds it when needed, and how it removes it when it is no longer required.

In this chapter, we will learn how symbol tables are designed, how data is stored using hash tables and trees, how scope is handled, and how operations like insert, lookup, and delete work.

Symbol Table Design

Designing a symbol table means deciding how the information will be stored and managed. A good design makes the compiler fast and efficient.

If the design is not good, the compiler will take more time to search for information. This will slow down the whole process.

A good symbol table design should be simple and organized. It should allow fast insertion of new variables and quick searching of existing variables. It should also be able to handle many variables without confusion.

Decoding Compilers

For example, when a new variable is declared, it should be stored quickly. When the variable is used later, the compiler should find it easily.

So, symbol table design is very important for the performance of the compiler.

Hash Table and Tree

Hash Table

A hash table is one of the most common ways to store data in a symbol table.

It works by using a special function called a hash function. This function takes a name and converts it into a number. This number tells where the data should be stored.

Because of this, searching is very fast. The compiler does not need to check all entries. It directly goes to the correct place.

For example, if we want to find variable x, the hash function tells exactly where x is stored.

Sometimes two names may go to the same place. This is called a collision. In that case, the compiler uses special methods to handle it. So, hash tables are fast and widely used.

Hash Function



X → **2**
Y → **4**
Z → **2** (collision)

Index	Data
0	-
1	-
2	x → z
3	-
4	y
5	-

Tree (Binary Tree)

Decoding Compilers

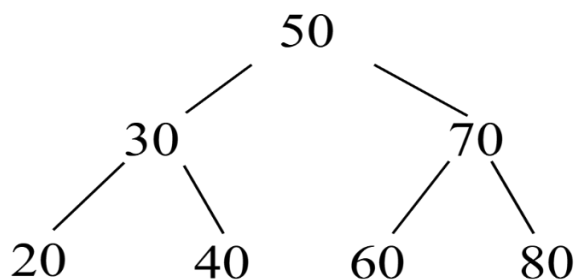
A tree stores data in a hierarchical way. It looks like a structure with branches.

Each element is connected in a way that smaller values go to one side and larger values go to another side.

When searching, the compiler starts from the top and moves step by step down the tree.

Trees are useful when we want data to be sorted. They help in keeping the information in order.

However, searching in a tree may take more time than a hash table. So, trees are useful for ordered data, while hash tables are faster for searching.



Scope Handling

Scope defines where a variable can be used in a program. It tells the compiler the area in which a variable is valid.

When a variable is declared inside a block, it can only be used inside that block. Outside that block, the variable cannot be accessed. This helps keep the program clear and organized.

The symbol table keeps track of all variables and their scopes. It knows which variable belongs to which part of the program. This is very important because the same variable name can be used in different places.

When the program enters a new block, a new scope is created. Any variable declared inside this block belongs only to that scope. When the program exits the block, that scope is removed, and its variables are deleted from the symbol table.

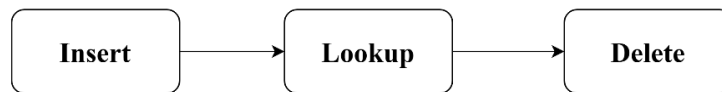
Decoding Compilers

Sometimes, two variables may have the same name but are declared in different scopes. In this case, the compiler treats them as different variables. There is no confusion because each variable belongs to its own scope.

So, scope handling helps the compiler manage variables properly. It avoids confusion and ensures that each variable is used in the correct place.

Insert, Lookup, Delete

These are the basic operations used in a symbol table.



Insert

Insert means adding a new entry into the symbol table.

When a variable is declared, it is inserted into the table with its details.

Example:

```
int a = 5;
```

Here, a is inserted with type int and value 5.

Lookup

Lookup means searching for an entry in the symbol table.

When a variable is used in the program, the compiler looks it up in the table to find its details.

Example:

If we use a later, the compiler searches for a in the table.

Delete

Delete means removing an entry from the symbol table.

When a scope ends, the variables of that scope are no longer needed. So, they are removed from the table.

Example:

When a block ends, its variables are deleted

Chapter 11 Exercises

Part A: Define / Explain

1. Define symbol table.
2. What is symbol table management?
3. Explain symbol table design.
4. What is a hash table?
5. What is a tree structure?
6. Define scope handling.
7. Explain insert operation.
8. What is lookup operation?
9. What is delete operation?

Part B: True / False

1. A symbol table stores information about variables.
2. Hash table search is slow.
3. Insert operation removes data from the table.
4. Lookup is used to search data.
5. Delete removes variables from the symbol table.

Part C: Short Answer Questions

1. Why is a symbol table important?
2. What information is stored in a symbol table?
3. What is the use of a tree in symbol table?
4. What is scope in programming?
5. What happens when a scope ends?

Part D: Comparison / Analytical Questions

1. Compare hash table and tree.
2. Difference between insert and lookup.
3. Compare lookup and delete operation.
4. Difference between hash table and tree in terms of speed.

Part E: Scenario / Story-Based Questions

1. A programmer writes:
 - a. `int a = 5;`
 - b. What operation is used to store a?
2. The compiler needs to find the value of x. Which operation will be used?
3. A block ends and variables inside it are removed. Which operation is used?
4. Two variables have the same name but in different blocks. How does scope handle this?

Part F: Challenge Questions

1. Identify operations (insert, lookup, delete):
 - i. `int a = 10;`
`int b;`
`b = a + 5;`
2. Draw a simple hash table example.
3. Draw a simple tree structure.

Chapter 12

CODE TO COMPILE

Introduction

In the previous chapters, we explored the theoretical foundations of compiler design, learning about lexical analysis, parsing, semantic checks, intermediate code, optimization, and code generation. But theory alone is not enough to fully understand the magic of compilation. This chapter brings that theory to life by guiding you through the practical implementation of a compiler, phase by phase.

You will learn not only how each phase works conceptually but also how to write code that performs the tasks of a real compiler from recognizing tokens in source code, building parse trees, checking semantics, generating intermediate representations, optimizing instructions, to finally producing target machine code.

Implementing a compiler may seem daunting at first, but breaking it down into its individual phases makes the process manageable and exciting. Each phase serves a specific purpose, and by coding them yourself, you will gain a hands-on understanding of how data flows through the compiler, how errors are detected and handled, and how optimizations improve efficiency. You will also learn how essential data structures, like symbol tables and three-address code, enable smooth communication between different stages of the compiler.

Understanding compiler implementation is not just an academic exercise. Writing the code for each phase develops your problem-solving skills, programming expertise, and system-level thinking. It equips you with the ability to debug complex systems, optimize code, and even design your own programming languages or interpreters in the future. Moreover, this practical experience builds a strong foundation for advanced topics in computer science, including virtual machines, language design, and cybersecurity.

By the end of this chapter, you will have transformed from a student of compiler theory to a practitioner capable of constructing a working compiler. You will experience the satisfaction of watching a source program pass through multiple stages of processing, evolving from plain text into executable instructions.

Decoding Compilers

This chapter is designed to bridge the gap between abstract concepts and real-world implementation, ensuring you to understand, test, and modify every phase of a compiler with confidence.

Get ready to turn theory into working code and explore the inner workings of one of the most fascinating and foundational tools in computer science.

One important note: we will use JavaScript as our programming language for the examples in this chapter. However, you are free to use any other language of your choice, as the core concepts remain the same across all programming languages.

Basic Code

Before start main part of compiler, first we need to learn some basic concepts on string. Given by:

Count String Length

The length of a string is the number of characters it contains. Knowing the length is essential when iterating through the string to analyze characters, identify tokens, or skip comments.

Code:

```
function lengthCount(str) {
  let count = 0;
  let i = 0;

  while (str[i] !== undefined) {
    // keep going until no character

    count++;
    i++;
  }
  return i;
}

let str = "This is a book";
console.log(`Length of the string: ${lengthCount(str)}`);
```

Remove white space

Whitespace includes spaces (), tabs (\t), and sometimes newlines (\n). A lexical analyzer usually ignores these characters while scanning tokens. Removing

Decoding Compilers

unnecessary whitespace makes it easier to identify identifiers, keywords, operators, and literals. So, it is necessary to skip whitespace when reading characters and keep whitespace only if it is necessary for token separation.

Steps:

1. Loop through each character of the string.
2. Check if it white space.
3. If not character is not whitespace them add the character to result string.
4. If whitespace then ignore it.

Code:

```
function removeWhiteSpace(str) {
  let result = "";
  let i = 0;

  while (str[i] !== undefined) {
    if (str[i] !== " ") {
      // skip spaces

      result += str[i]; // build new string
    }
    i++;
  }

  return result;
}

let str = "    int    a = 5;    ";
console.log(`New string: "${removeWhiteSpace(str)}"`);
```

Count Vowels

Vowels in English are a, e, i, o, u (both uppercase and lowercase). Counting vowels can be useful in lexical analysis for string processing, statistics, or specific token checks.

Steps:

1. Loop through each character of the string.
2. Check if it is a vowel.
3. Increment a counter whenever a vowel is found.

Code:

Decoding Compilers

```
function countVowels(str) {
  let count = 0;
  let i = 0;
  const vowels = ["a", "e", "i", "o", "u"];

  while (str[i] !== undefined) {
    if (vowels.includes(str[i].toLowerCase())) {
      // you can use use hard coded if statmetns here also
      // such as
      /*
          if(str[i]==='a' || (and so on...) )
        */

      count++;
    }
    i++;
  }

  return count;
}

let str = "Hena lives in Tokyo!";
console.log(`Number of vowels: ${countVowels(str)}`);
```

Multiple Line Input

In a compiler, the lexical analyzer must process the entire source program, which often spans multiple lines. To handle this, the source code is read line by line and stored in memory (e.g., an array or vector). This allows the analyzer to scan each character sequentially, identify tokens, and pass them to the parser. Using multi-line input ensures that complex programs with loops, functions, and nested structures can be analyzed correctly.

Key steps in multi-line input handling:

1. Read one line at a time from the input.
2. Store each line in a list or array for processing.
3. Repeat until a stopping condition is reached (e.g., end-of-file or a sentinel value).
4. Pass the stored lines to the tokenizing logic.

Code:

```
#include <iostream>
#include <string>
```

Decoding Compilers

```
#include <vector>
using namespace std;

int main()
{
    vector<string> sourceCode; // Store each line of code
    string line;
    cout << "Enter your code (type END to finish):\n";

    int count = 0; // optional: count number of lines
    while (true)
    {
        getline(cin, line); // Read one line from input
        if (line == "END")
            break; // Stop when user, if line only contain "END"
        count++;
        sourceCode.push_back(line); // Store line in vector
    }

    // Display the collected lines (optional)
    cout << "\nSource Code Entered:\n";
    for (auto l : sourceCode)
    {
        cout << l << endl;
    }
    cout << "Total Lines: " << count << endl;

    return 0;
}
```

Handling Comments

Comments are parts of the source code that are ignored by the compiler during execution. They are used to document code and improve readability. The lexical analyzer must detect and skip comments so that they don't interfere with token generation.

There are 2 types of comments:

1. Single-line comments: Usually start with `//` and continue to the end of the line.
2. Multi-line comments: Enclosed between `/*` and `*/` and can span multiple lines.

Key Points in Handling Comments:

Decoding Compilers

1. Ignore all characters inside the comment.
2. Properly detect the start and end of comments.
3. Ensure multi-line comments spanning multiple lines are handled correctly.
4. Continue normal tokenization after the comment ends.

Steps:

1. Iterate through each line of code.
2. Use a flag to track multi-line comments.
3. Skip characters inside single-line and multi-line comments.
4. Store only the actual code lines in the output

Code:

```
function removeComment(code) {
  let result = "";
  let i = 0;
  let inSingleLine = false;
  let inMultiLine = false;

  while (i < code.length) {
    // Check for start of single-line comment
    if (
      !inSingleLine &&
      !inMultiLine &&
      code[i] === "/" &&
      code[i + 1] === "/"
    ) {
      inSingleLine = true;
      i += 2;
      continue;
    }

    // Check for start of multi-line comment
    if (
      !inSingleLine &&
      !inMultiLine &&
      code[i] === "/" &&
      code[i + 1] === "*"
    ) {
      inMultiLine = true;
      i += 2;
      continue;
    }

    // End of single-line comment
    if (inSingleLine && code[i] === "\n") {
```

Decoding Compilers

```
        inSingleLine = false;
        result += code[i];
        i++;
        continue;
    }

    // End of multi-line comment
    if (inMultiLine && code[i] === "*" && code[i + 1] === "/") {
        inMultiLine = false;
        i += 2;
        continue;
    }

    // If not inside a comment, copy character
    if (!inSingleLine && !inMultiLine) {
        result += code[i];
    }

    i++;
}

return result;
}

// Example usage:
let code = `
int a = 5; // initialize a
/* a multi
line comment */
int b = 10;
`;

console.log(removeComment(code));
```

Regex

Regular Expressions (Regex) are patterns used to match and identify strings. In lexical analysis, regex is very useful for recognizing tokens such as identifiers, numbers, keywords, operators, and literals. Instead of checking characters one by one manually, regex allows us to define token patterns in a simple and powerful way.

For example:

1. An identifier usually starts with a letter and may contain letters and digits.

Decoding Compilers

2. A number contains only digits.
3. An operator can be +, -, *, /, =, etc.

Using regular expressions, the lexical analyzer can quickly match these patterns and classify tokens.

Regex Notation and Description:

N.	Description	N.	Description
.	Any char except newline	\$	End of a string
\d	Digit (0-9)	[]	Matches char in brackets (array)
\D	Not a digit	[^]	Matches char not in brackets
\w	Word char (A-Z, a-z, 0-9, _)		Either or
\W	Not a word char	()	Group
\s	Whitespace	*	0 or more
\S	Not whitespace	+	1 or more
\b	Word boundary	?	0 or 1
\B	Not a word boundary	{n}	Exact number
^	Beginning of a string	{m, n}	Range of numbers (min, max)

Example-1:

Consider a phone number format like 321-555-4231. This format follows a specific pattern. The first three characters are digits (0–9), followed by a dash (-), then again three digits, another dash (-), and finally four digits. Using regular expressions, we can write this pattern as:

```
\d\d\d-\d\d\d-\d\d\d\d
```

This regex means:

1. \d → any digit (0–9)
2. \d\d\d → exactly three digits
3. d- → dash character

So, the full expression matches a phone number in the format XXX-XXX-XXXX.

We can also write the same regex in a shorter form using repetition notation:

```
\d{3}-\d{3}-\d{4}
```

Decoding Compilers

This means:

1. `\d{3}` → exactly 3 digits
2. `\d{4}` → exactly 4 digits

This is a more compact and commonly used way to write regular expressions.

Example-2:

Suppose we want to create an email pattern that only supports the domains gmail.com, custom.com, and yahoo.com. An email address generally has two main parts: the username and the domain. The username may contain letters, digits, dots (.), and underscores (_), followed by the @ symbol and then the domain name.

So, the pattern structure is:

1. Username (letters, digits, ., _)
2. @ symbol
3. Domain (gmail.com or custom.com or yahoo.com)

Using regular expressions, we can write this pattern as:

```
[a-zA-Z0-9._]+@[gmail\.com|custom\.com|yahoo\.com)
```

Example-3:

Suppose name patterns:

1. Mr. Snatch
2. Mr Smith
3. Mrs. Robinson
4. Ms devis
5. Mr. T

A suitable regex pattern for these names is:

```
M(r|s|rs)\.?\s[A-Z]\w*
```

Let us break the pattern step by step. The pattern starts with the letter M, which means all names must begin with the letter M. After that, the expression (r|s|rs) is used. This means the next letters can be r, s, or rs, which forms the titles Mr, Ms, and Mrs.

Next, we have .?. The backslash is used because the dot (.) is a special character in regular expressions, so we use \. to represent an actual dot. The question mark (?) means the dot is optional. So the pattern will match both Mr and Mr.

Decoding Compilers

After that, `\s` represents a whitespace character, meaning there must be a space between the title and the name.

Then `[A-Z]` means the first letter of the name must be a capital letter.

Finally, `\w*` means zero or more word characters. Word characters include letters, digits, and underscore. This allows names like Smith, Robinson, or even T.

So overall, the pattern means: the text should start with Mr, Ms, or Mrs, followed by an optional dot, then a space, and then a name that starts with a capital letter and may contain more letters after that.

Regex Pattern Code

```
function checkRE(input, regex) {
  if (regex.test(input)) {
    return "Accepted";
  } else {
    return "Rejected";
  }
}

// Example usage:
let testStrings = ["ab", "abab", "ababab", "aba", "abba",
  "abababab"];
const regex = /^(ab){2,}$/; // Regex for (ab) repeated 2 or more
times

for (let str of testStrings) {
  console.log(`${str}: ${checkRE(str, regex)}`);
}
```

Max Frequency Word

Now we have find the max frequency word from a given sentence. Note that we have to ignore all the prepositions, articles, punctuations, and conjunctions.

Code:

```
const ignoreWords = [
  "a", "an",
  "the",
  "and",
```

Decoding Compilers

```
"or",
"but",
"so",
"yet",
"for",
"nor",
"on",
"in",
"at",
"by",
"with",
"about",
"against",
"between",
"into",
"through",
"during",
"before",
"after",
"above",
"below",
"to",
"from",
"up",
"down",
"over",
"under",
"again",
"further",
"then",
"once",
];

function maxFrequencyWord(text) {
  const cleanedText = text
    .toLowerCase()
    .replace(/[.,\/#!$%^&*;:{}=\-_`~()]/g, "");

  const words = cleanedText.split(/\s+/);

  const frequency = {};
  for (const word of words) {
    if (!ignoreWords.includes(word) && word !== "") {
      frequency[word] = (frequency[word] || 0) + 1;
    }
  }
}
```

Decoding Compilers

```
}

let maxWord = "";
let maxCount = 0;
for (const word in frequency) {
  if (frequency[word] > maxCount) {
    maxWord = word;
    maxCount = frequency[word];
  }
}

return { word: maxWord, count: maxCount };
}

const inputString =
  "The cat and the dog played in the garden. The dog was very
happy.";
const result = maxFrequencyWord(inputString);
console.log(
  `The most frequent word is "${result.word}" with
${result.count} occurrences.`
);
```

Creating a New Language

In this section, we will design and implement a small custom programming language using JavaScript called as **SimpleScript**. The main goal of this project is to understand how a compiler or interpreter works internally by building a simple language from scratch. Instead of only studying theory, we will actually process a new language and execute its programs. This will help us clearly understand how source code is transformed into executable actions.

Our new language will support basic programming features such as variable declaration, assignment, arithmetic expressions, strings, comments, and a print statement. The syntax of the language will be simple and easy to understand so that we can focus on the internal working process rather than complex syntax rules.

To implement this language, we will follow several important steps. The first step is tokenization, where the source code will be broken into tokens such as keywords, identifiers, numbers, operators, and strings. The second step is building the AST (Abstract Syntax Tree), where the tokens will be organized into a tree structure that represents the program logically. The third step is syntax analysis, where we check

Decoding Compilers

whether the program follows the grammar rules of our language. If the syntax is correct, the program will move to the next step.

In the fourth step, we have two possible options. The first option is to convert our new language into another programming language such as JavaScript, C, or Python. The second option is to directly execute the program using an interpreter. In this chapter, we will choose the second option, which means we will directly execute the program after syntax analysis by interpreting the AST.

So, the overall process of our language implementation will be:

Source Code → Tokenization → AST → Syntax Analysis → Execution

By completing this project, you will understand how a programming language is implemented, how a compiler works, and how an interpreter executes a program. This practical implementation will connect all the compiler design concepts you learned in previous chapters and show how they work together in a real system.

Demo Code for SimpleScript

```
input a = 5 // initiate
input b = 2
/* multiline comment */
input sum = a + b //precedence order followed

print sum
```

This SimpleScript program demonstrates the basic features of the language. The input keyword is used to declare and initialize variables. Variables can store numbers (integers, floats) or strings, and they should be given a value when declared.

Arithmetic operations supported and operator precedence maintained.

Comments can be added as single-line (//) or multi-line (/* ... */) and are ignored during execution. This helps document your code without affecting its behavior.

The print statement outputs the value of a variable or expression to the console. Unlike more advanced template strings, values are printed directly without needing {} syntax.

Overall, this demo shows how SimpleScript handles variable declarations, arithmetic operations, comments, and output in a straightforward, easy-to-read style.

Main Function

In our SimpleScript interpreter, the main function serves as the central entry point for executing any program. All the core components of the interpreter, such as tokenization, parsing, AST generation, and execution, are called from this function. It acts as the orchestrator, ensuring that the source code flows through the correct sequence of steps.

```
function main(code) {
  try {
    // 1. Lexing
    const tokens = lexer(code);

    // 2. Parsing
    const ast = parser(tokens);

    // 3. Analyzing
    analyzer(ast);

    // 4. Evaluation (Execution)
    const environment = {}; // This holds our variables during
runtime
    evaluate(ast, environment);

  } catch (err) {
    console.error(`COMPILER ERROR: ${err.message}`);
  }
}

const code = `
input a = 5 // initiate
input b = 2.3
/* multiline comment */
input sum = a + b * 4 / 3 //precedence order followed

print sum
`;

main(code);
```

Tokenization (Lexer)

Tokenization is the first step in interpreting or compiling a program. In SimpleScript, tokenization involves reading the source code and breaking it into meaningful units called tokens, which the interpreter can understand. Each token represents a basic element of the language, such as a keyword, identifier, literal, operator, symbol, or comment.

Tokenization will break our SimpleScript program code into tokens like:

Token	Lexeme
Keywords	input, print
Identifiers	[a-zA-Z][a-zA-Z0-9]*
Numbers	22, 1.23
Strings	"Teeth"
Operators	+, -, *, /, %

Steps for tokenization:

1. Define token types. Decide what types of tokens exist in SimpleScript.
2. Read the source code and process the program as a string, line by line or character by character.
3. Use regular expressions to detect each type of token.
4. Ignore whitespace and comments.
5. Store each recognized token as an object with its type and value in a token list for later processing.
6. If an unrecognized character or sequence is found, raise an error to indicate an invalid token.

Tokenization code:

```
function lexer(code) {
  const tokens = [];
  let cursor = 0;

  const tokenSpecs = [
    { type: "COMMENT", regex: /^\/\/.*|^\/\/*[\s\S]*?\*\/ / },
    { type: "WHITESPACE", regex: /\s+/ },
    { type: "KEYWORD", regex: /^(input|print)\b/ },
    // This Regex ensures identifiers DON'T start with a digit
    { type: "IDENTIFIER", regex: /^[a-zA-Z_][a-zA-Z0-9_]* / },
    { type: "NUMBER", regex: /^[0-9]+(\.[0-9]+)? / },
    { type: "OPERATOR", regex: /^[=+*/-]/ },
  ]
```

Decoding Compilers

```
];

while (cursor < code.length) {
  const remainingCode = code.slice(cursor);
  let matched = false;

  for (const { type, regex } of tokenSpecs) {
    const match = remainingCode.match(regex);

    if (match) {
      const value = match[0];

      // LOGIC CHECK: If we just matched a NUMBER,
      // check if the very next character is a letter.
      if (type === "NUMBER") {
        const nextChar = remainingCode[value.length];
        if (nextChar && /[a-zA-Z_]/.test(nextChar)) {
          throw new Error(
            `Lexical Error: Variable names cannot start with a
number ("${value}${nextChar}...")`,
          );
        }
      }

      if (type !== "WHITESPACE" && type !== "COMMENT") {
        tokens.push({ type, value });
      }

      cursor += value.length;
      matched = true;
      break;
    }
  }

  if (!matched) {
    throw new Error(
      `Unexpected character at position ${cursor}:
"${code[cursor]}"`,
    );
  }
}

return tokens;
}
```

Parser

After tokenization, the next step is parsing the tokens into a structured representation called an Abstract Syntax Tree (AST). The AST is a tree-like structure that represents the logical hierarchy of the program. Each node in the tree represents a construct in the language, such as:

1. Variable declaration (input a = 13)
2. Assignment expressions (sum2 = sum + a)
3. Print statements (print "Sum is: \${sum}")
4. Literals, identifiers, and operators

Using the AST, the interpreter can analyze the program, respect operator precedence, and execute statements in the correct order.

Steps to create AST:

1. Start with the token list produced by the lexer.
2. Iterate through tokens to identify statements:
 - a. Variable declarations
 - b. Assignments
 - c. Print statements
3. Build nodes for each construct and store them in a tree-like array.
4. Handle expressions recursively to respect operator precedence.
5. Return the AST, which will be passed to the execution stage.

Code:

```
function parser(tokens) {
  let cursor = 0;

  // return current token without
  // moving cursor
  function peek() {
    return tokens[cursor];
  }

  // consume a token if it matches the expected type
  // (and optional value)
  function consume(expectedType, expectedValue = null) {
    const token = tokens[cursor];
    if (
      !token ||
      token.type !== expectedType ||
      (expectedValue && token.value !== expectedValue)
    )
      return null;
    cursor++;
    return token;
  }
}
```


Decoding Compilers

```
) {
  throw new Error(
    `Expected ${expectedType}${expectedValue ? " " +
expectedValue : ""}, got ${token?.type} ${token?.value}`,
    );
}
cursor++;
return token;
}

// parse the entire program
function parseProgram() {
  const body = [];
  while (cursor < tokens.length) {
    body.push(parseStatement());
  }
  return { type: "Program", body };
}

// Parse a single statement
function parseStatement() {
  const token = peek();
  if (token.type === "KEYWORD" && token.value === "input") {
    return parseInputStatement();
  } else if (token.type === "KEYWORD" && token.value ===
"print") {
    return parsePrintStatement();
  } else {
    throw new Error(`Unexpected statement at ${cursor}:
${token.value}`);
  }
}

// parse an input statement: input <id> = <expr>
function parseInputStatement() {
  consume("KEYWORD", "input");
  const id = consume("IDENTIFIER");
  consume("OPERATOR", "=");
  const expr = parseExpression();
  return {
    type: "InputStatement",
    id: { type: "Identifier", name: id.value },
    expression: expr,
  };
}
```

Decoding Compilers

```
// parse a print statement: print <expr>
function parsePrintStatement() {
  consume("KEYWORD", "print");
  const expr = parseExpression();
  return {
    type: "PrintStatement",
    expression: expr,
  };
}

// parse expressions with precedence (lowest level starts here)
function parseExpression() {
  return parseAdditive();
}

// handle lower precedence (+, -)
function parseAdditive() {
  let node = parseMultiplicative();
  while (
    peek() &&
    peek().type === "OPERATOR" &&
    (peek().value === "+" || peek().value === "-")
  ) {
    const operator = consume("OPERATOR").value;
    const right = parseMultiplicative();
    node = { type: "BinaryExpression", operator, left: node,
right };
  }
  return node;
}

// handle * and / operators (higher precedence)
function parseMultiplicative() {
  let node = parsePrimary();
  while (
    peek() &&
    peek().type === "OPERATOR" &&
    (peek().value === "*" || peek().value === "/")
  ) {
    const operator = consume("OPERATOR").value;
    const right = parsePrimary();
    node = { type: "BinaryExpression", operator, left: node,
right };
  }
}
```

Decoding Compilers

```
    return node;
  }

  // parse numbers or identifiers
  function parsePrimary() {
    const token = peek();
    if (token.type === "NUMBER") {
      consume("NUMBER");
      return { type: "NumberLiteral", value: Number(token.value) };
    } else if (token.type === "IDENTIFIER") {
      consume("IDENTIFIER");
      return { type: "Identifier", name: token.value };
    } else {
      throw new Error(`Unexpected token in expression: ${token.value}`);
    }
  }

  return parseProgram();
}
```

Semantic analysis

Semantic Analysis is the phase where we ensure the program is meaningful according to the rules of the language. While the AST tells us the structure of the program, semantic analysis checks for valid use of variables, expressions, and types.

Steps for Semantic Analysis:

1. Initialize a Symbol Table
2. Traverse the AST
3. Check for variable declaration and logics
4. Report error

Code:

```
function analyzer(ast) {
  // A Set to keep track of defined variable names
  const symbolTable = new Set();

  function visit(node) {
    switch (node.type) {
      case "Program":
        node.body.forEach(visit);
        break;
    }
  }
}
```

Decoding Compilers

```
    case "InputStatement":
        // 1. Check the expression being assigned first
        visit(node.expression);
        // 2. Register the variable in the symbol table
        symbolTable.add(node.id.name);
        break;

    case "PrintStatement":
        // Check the expression to ensure all variables used
        exist
        visit(node.expression);
        break;

    case "BinaryExpression":
        // Recursively check left and right sides
        visit(node.left);
        visit(node.right);
        break;

    case "Identifier":
        // CRITICAL CHECK: Is the variable defined?
        if (!symbolTable.has(node.name)) {
            throw new Error(
                `Semantic Error: Variable "${node.name}" is used
before it is defined.`
            );
        }
        break;

    case "NumberLiteral":
        // Numbers are always valid
        break;

    default:
        throw new Error(`Unknown AST node type: ${node.type}`);
}

// Start the analysis
visit(ast);
console.log("Semantic Analysis: Success (All variables
resolved).");
}
```

Evaluation

Evaluation, also known as execution or interpretation, is the phase where the program is actually run. At this stage, the AST has already been verified through syntax and semantic analysis, so it is guaranteed to be correct and meaningful.

In SimpleScript, the evaluator takes the AST and executes each statement step by step. It calculates expressions, assigns values to variables, and produces output using the print statement. This phase transforms the program from a static structure (AST) into actual results.

The evaluator works together with a runtime environment (symbol table), which stores variable names and their values during execution. Every computation and variable update happens inside this environment.

Steps for evaluation:

1. Initialize runtime environment (symbol table) which will store all variables and their values during execution.
2. Traverse the AST
3. Handle each part of the program such as variable declaration, print statement, expression.
4. Update symbol table
5. Handle runtime error (such as divide by zero)
6. Continue until all AST nodes are processed.
7. Display the final output

Code:

```
function evaluate(node, env = {}) {
  switch (node.type) {
    case "Program":
      // Execute each statement in order
      node.body.forEach((stmt) => evaluate(stmt, env));
      break;

    case "InputStatement":
      // Evaluate the expression and store it in the environment
      const value = evaluate(node.expression, env);
      env[node.id.name] = value;
      return value;

    case "PrintStatement":
      // Evaluate the expression and log the result
```

Decoding Compilers

```
const result = evaluate(node.expression, env);
console.log(`${result}`);
return result;

case "BinaryExpression":
  // Recursively evaluate left and right operands
  const left = evaluate(node.left, env);
  const right = evaluate(node.right, env);

  // Perform the actual math
  switch (node.operator) {
    case "+":
      return left + right;
    case "-":
      return left - right;
    case "*":
      return left * right;
    case "/":
      return left / right;
    default:
      throw new Error(`Unknown operator: ${node.operator}`);
  }

case "Identifier":
  // Look up the variable value in the environment
  if (node.name in env) {
    return env[node.name];
  }
  throw new Error(`Runtime Error: Variable "${node.name}" is
not defined.`);

case "NumberLiteral":
  return node.value;

default:
  throw new Error(`Evaluation Error: Unsupported node type
"${node.type}"`);
}
```

SimpleScript was designed as a small, simple, and educational programming language to demonstrate how a programming language works internally. The goal of SimpleScript is not to compete with existing programming languages, but to help understand the main phases of a compiler or interpreter in a clear and practical way.

Decoding Compilers

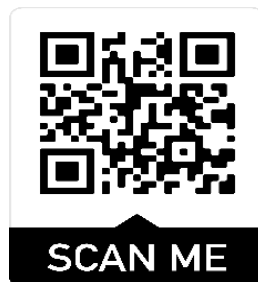
Throughout the design of SimpleScript, we implemented the complete language processing pipeline step by step. The process started with lexical analysis, where the source code was converted into tokens. Then, syntax analysis was performed to check whether the program follows the grammar rules and to build the Abstract Syntax Tree (AST). After that, semantic analysis ensured that variables were declared, types were used correctly, and expressions were meaningful. Finally, in the evaluation phase, the AST was executed using a runtime environment and symbol table to produce the final output.

This concludes our SimpleScript project. The implementation presented here provides a basic foundation, but it can be further improved and extended. You can enhance the language by adding more advanced features such as conditional statements, loops, functions, classes, and other modern programming constructs.

Another mini project is also provided below. This language is called “Spark”, and it is a web-based platform. It allows you to write and execute code in an environment similar to a terminal or an online compiler. You can also upload source code files with the “.spark” extension.

The Spark language supports additional features such as conditional statements, while loops, break and continue statements, boolean values, and more, making it a more advanced version compared to SimpleScript.

The complete source code, SimpleScript project, Spark project, and the full book are available in the GitHub repository provided below. Readers are encouraged to explore the repository to better understand the implementation details and experiment with the code:



Or: [Click Me](#)

Final Words

This marks the end of our journey into compiler design and language implementation. The best way to truly understand these concepts is to build, modify, and experiment with your own programming language.

This book, Decoding Compilers, was written to help students and beginners understand the basic concepts of compiler design in a simple and clear way. Throughout this book, we have explored how a compiler works, from lexical analysis to code generation, and learned how programming languages are translated into machine code.

Compiler design is an important part of computer science. It helps us understand how high-level languages are processed and executed by a computer. By learning these concepts, readers can improve their problem-solving skills and gain a deeper understanding of programming and software development.

We hope that this book has made compiler concepts easier to understand and has built a strong foundation for further learning. Readers are encouraged to practice more, explore advanced topics, and apply their knowledge in real-world projects.

Finally, we would like to thank all the readers for choosing this book. Your interest and effort in learning are the key to success. Keep learning, keep exploring, and never stop building.

END